

**O‘ZBEKISTON RESPUBLIKASI MUDOFAA VAZIRLIGI
AXBOROT-KOMMUNIKASIYA TEXNOLOGIYALARI VA ALOQA
HARBIY INSTITUTI**

**“KIBERXAVFSIZLIK” FAKULTETI
“AXBOROT TEXNOLOGIYALARI VA DASTURIY INJINIRING”
kafedrası**



SH.R. SAPAYEV

“PYTHON DASTURLASH TILI ASOSLARI”

O‘QUV QO‘LLANMA

Mazkur o‘quv qo‘llanma O‘zbekiston Respublikasi Oliy ta’lim, fan va innovatsiyalar vazirining 2023 yil 22-dekabrdagi 537-sonli buyrug‘iga asosan nashr etishga ruxsat berildi.

Toshkent-2023

UO‘K: 004.031.6.

KBK: 1132

Muallif: Sh.R. Sapayev, O‘zbekiston Respublikasi Mudofaa vazirligi Axborot-kommunikatsiya texnologiyalari va aloqa harbiy instituti, Axborot texnologiyalari va dasturiy injiniring kafedrası dotsenti // –T: 2023. –137 b.

“O‘rnatilgan tizimlarning apparat-dasturiy ta‘minoti” fanidan ishlab chiqilgan “Python dasturlash tili asoslari” nomli o‘quv qo‘llanma Axborot-kommunikatsiya texnologiyalari va aloqa harbiy institutida barcha ta‘lim yo‘nalishlari bo‘yicha tahsil olayotgan kursant va tinglovchilar uchun zamonaviy dasturlash tili-pythonning imkoniyatlaridan foydalanib sxemalar asosida yig‘ilgan qurilmalarga turli dasturlar tuzish, tayyor tuzilgan sxemalarning dasturlariga o‘zgartirishlar kiritish, ularni testlash, xizmat faoliyati davomida va xo‘jalik ishlarida foydalanish uchun turli dastur tuzish muhim ahamiyatga ega.

Shuningdek, mazkur o‘quv qo‘llanmadan ushbu ta‘lim yo‘nalishi bo‘yicha tahsil olayotgan barcha toifadagi oliy va o‘rta-maxsus bilim yurtlari talabalari ham foydalanishlari mumkin.

Ushbu o‘quv qo‘llanma o‘quv fanining o‘quv dasturiga muvofiq ishlab chiqilgan.

O‘quv qo‘llanma 4 ta bob, 21 ta mavzu, 21 ta rasm va 4 ta jadvaldan iborat bo‘lib, har bir mavzuga doir turli dasturlar tuzilgan va ularning tuzilishini tahlil qilish bilan tushuntirib ketilgan. Shuningdek har bir mavzu so‘ngida mavzuga doir nazorat savollari berilgan.

Taqrizchilar:

Sh.Sh. Safarov - Muhammad al-Xorazmiy nomidagi Toshkent Axborot Texnologiyalari universiteti “Multimediya texnologiyalari” kafedrası dotsenti

A.A. Raximov - Dotsent, Axborot-kommunikatsiya texnologiyalari va aloqa harbiy instituti “Chegara qo‘shinlari informatsion texnologiya mutaxassislarini tayyorlash bo‘yicha intellektual tizimlar” kafedrası katta o‘qituvchisi.

Mazkur o‘quv qo‘llanmasi AT kafedrasining 2023-yil 7-apreldagi 17 – sonli yig‘ilish bayonnomasida ko‘rib chiqilgan.

Ushbu o‘quv qo‘llanma O‘R MV AKT va AHI ilmiy kengashining 2023-yil 27-iyun № 23-sonli bayonnomada muhokama etildi va nashr etishga tavsiya qilindi.

ISBN: 529350

M U N D A R I J A

KIRISH.....	4
I BOB. PYTHON DASTURLASH TILIGA KIRISH	6
1.1.§. Python dasturlash tili va uning rivojlanish tarixi	6
1.2.§. Python va PyCharm dasturini Windowsga oʻrnatish va birinchi dastur	17
II BOB. PYTHON ASOSLARI	27
2.1.§. Pythonda dastur strukturasi	27
2.2.§. Oʻzgaruvchilar va maʼlumotlar turlari	30
2.3.§. Pythonda kiritish va chiqish funksiyalari	36
2.4.§. Arifmetik operatorlar	39
2.5.§. Mantiqiy ifodalar	44
2.6.§. Shart operatori – if, elif	48
2.7.§. Pythonda takrorlanuvchi jarayonlarni dasturlash	53
2.8.§. Funksiyalar	61
2.9.§. Parametrli funktsiya	65
2.10.§. Qiymat qaytaruvchi funktsiya	70
2.11.§. Lambda funktsiya	73
2.12.§. Oʻzgaruvchilarning tiplarini oʻzgartirish	75
III BOB. SATRLAR BILAN ISHLASH	80
3.1.§. Satrlar va ularni eʼlon qilish	80
3.2.§. Satr bilan ishlovchi operatorlar va metodlardan foydalanish	88
3.3.§. Format metodining qoʻllanilishi	94
IV BOB. ROʻYXAT, KORTEJ VA LUGʻATLAR BILAN ISHLASH	100
4.1.§. Pythonda roʻyxat tushunchasi va u bilan ishlovchi metodlar bilan tanishish	100
4.2.§. Pythonda kortej tushunchasi va u bilan ishlovchi metodlar bilan tanishish	116
4.3.§. Pythonda lugʻat tushunchasi va u bilan ishlovchi metodlar bilan tanishish	120
4.4.§. Toʻplamlar bilan ishlash	129
FOYDALANILGAN ADABIYOTLAR ROʻYXATI	137

KIRISH

O‘zbekiston Respublikasi Prezidenti – Qurolli Kuchlari Oliy Bosh Qo‘mondoni Sh.M. Mirziyoyev ta’kidlab o‘tganidek, oliy harbiy ta’lim muassasalarining intellektual saviyasi yuksak, dunyoqarashi keng, uzoqni ko‘ra oladigan ofitser kadrlarni tayyorlash borasidagi faoliyatini tubdan qayta ko‘rib chiqish talab etiladi.

2019 yilning 8-oktabrida O‘zbekiston Respublikasi Prezidentining “O‘zbekiston Respublikasi oliy ta’lim tizimini 2030-yilgacha rivojlantirish konsepsiyasini tasdiqlash to‘g‘risida”gi PF -5847-sonli farmonining imzolanishi oliy ta’lim tizimidagi islohotlarni yanada shiddat bilan, sifatli amalga oshirilishiga turtki bo‘ldi.

O‘zbekiston Respublikasi Qurolli Kuchlari tashkil etilganligining 30 yilligi munosabati bilan Vatan himoyachilariga yo‘llagan bayram tabrigida Prezidentimiz quyidagilarni alohida ta’kidlab o‘tdilar: “...harbiy ta’lim va ilm-fan tizimida boshlangan islohotlarni izchil amalga oshirish, soha uchun kadrlar tayyorlashda nazariya va amaliyotni imkon qadar yaqinlashtirishga alohida e’tibor qaratish lozim”.

Davlatimiz rahbari ta’kidlaganidek, “Biz yashayotgan bugungi zamon hech qachon bir joyda to‘xtab turmasligini barchamiz o‘zimizga doimo yaxshi tasavvur qilganmiz va tasavvur qilamiz. Zamon talablariga javob bera oladigan, yangicha fikrlaydigan, zamonaviy bilimlarga ega bo‘lgan, yuksak malakali harbiy mutaxassislarni tayyorlash bugungi kunning dolzarb vazifasidir”.

Bugungi kunda kursant va tinglovchilarga ta’lim sohasida tashkil etilgan zamonaviy o‘quv-moddiy-texnik bazadan samarali foydalanishni ta’minlash, oliy harbiy ta’lim tizimidagi yo‘nalishlar va mutaxassisliklar bo‘yicha davlat ta’lim standartlari, mutaxassislik bo‘yicha malakaviy talablar, o‘quv dasturlari, o‘quv-metodik adabiyotlarni qayta ko‘rib chiqishi va takomillashtirish borasida ko‘plab ishlar amalga oshirilmoqda.

Ushbu o‘quv qo‘llanmaning asosiy maqsadi – kursant va tinglovchilarga fanning yuqorida ko‘rsatib o‘tilgan talablardan kelib chiqib qayta ishlangan o‘quv dasturiga asosan fanning dasturlash qismini nazariya va amaliyotdan foydalangan holda yetkazib berish, hozirgi zamonaviy dasturlash tili bo‘lmish python dasturlash tilida turli dasturlar tuzish, turli sxemalar asosida tuzilgan qurilmaga dasturiy ta’minot qismini kodini yozish va ularga xizmat ko‘rsatish kabi bilim va ko‘nikmalarni o‘rgatishdan iborat.

“Python dasturlash tili asoslari” nomli o‘quv qo‘llanmaning asosiy maqsadi kursant va tinglovchilarda python dasturlash tilining asosiy negizi haqida bilimlarni egallash va ushbu bilimlardan foydalanib, turli crossplatformali ilovalarni mustaqil ravishda ishlab chiqish ko‘nikmalarini shakllantirish.

Ushbu o‘quv qo‘llanmada python tilining quyidagi bo‘limlari to‘liq yoritilib ketilgan: *python dasturlash tilining kelib chiqishi, python dasturlash tili asoslari,*

satrlar bilan ishlovchi metodlar bilan ishlash, python tilida ro'yxat, kortej va lug'atlar bilan ishlash.

O'z-o'zidan ma'lumki, davr talabidan va shiddat bilan rivojlanib borayotgan fan-texnika yutuqlaridan kelib chiqqan holda fanning o'quv dasturi muntazam ravishda yangilanib borishi talab etiladi. Shu sababli ushbu o'quv qo'llanma tarkibiy tuzilishi, mavzularning yoritilish darajasi, ularni bayon etish usuli bo'yicha muayyan kamchiliklarga ega bo'lishi mumkin. Shu munosabat bilan muallif bu haqida o'z fikr-mulohazalarini bildirgan o'quvchilariga minnatdorchilik bildiradi.

I-BOB. PYTHON DASTURLASH TILIGA KIRISH

1.1.§. Python dasturlash tili va uning rivojlanish tarixi.

Python dasturlash tili. Python yuqori darajadagi dasturlash tili bo'lib, u AT ning turli sohalarida, masalan, mashinani o'rganish, turli ilovalarni ishlab chiqish, web illovalari ishlab chiqish, tahlil qilish va boshqalarda keng qo'llaniladi.

2019-yilda Python Java tilini 10 foizga ortda qoldirib, eng ommabop dasturlash tiliga aylandi. Bu juda ko'p sabablarga bog'liq bo'lib, ulardan biri malakali mutaxassislarga to'lanadigan ish haqining yuqoriligidir (yiliga taxminan 100 ming dollar).

Hozirgi kunda Turli xil dasturlash tillari mavjud bo'lib, odatda ular ma'lum bir sohada (yoki bir nechta) liderlik qilishadi. Python dasturlash tili bo'lsa, turli xil sohalarda ilovalar yaratish imkomiyatiga egaligi, dasturchilarni o'ziga jalb qilmasdan qolmaydi.

Web ilovalar ishlab chiqish bozorining kichik qismini Python tili yaratilgan web illovalari egallagan. Python tilini desktop ilovalarini yozish uchun ham ishlatilsa, mashinani o'rganish (machine learning) sohasida to'liq yetakchilik qiladi.

Python nomining kelib chiqish tarixi.

Gido van Rossum python dasturlash tilini yaratganidan keyin, uni nomlamasdan turib tarqatishni hohlamadi. Ammo mukammal nom tanlash uchun vaqt sarflash Gvido uchun vaqtni behuda o'tkazish hisoblanardi. Shunda uning xayoliga kelgan birinchi fikr "Python" bo'ldi va dasturlash tilining nomini "Python"deb nomlab qo'yaqoldi. Chunki 1970-yillarning boshida mashhur bo'lgan, "Monty Python's Flying Circus" komediya seryali uning sevimli shoularidan biri edi.

Yangi til nomi bilan bir qatorda logotip o'ylab topish kerak edi va Guido tasodifiy shrift tanladi va "Python" so'zini shu shriftda yozdi. Gvido bu nomning ilonlarga aloqasi yo'qligini qayta-qayta ta'kidlagan bo'lsada, ommaning fikriga ta'sir qilishning ilojisi yo'q edi.

Ko'pchilik dasturchilar Python tilini ilonlar bilan bog'lab, uning nomini emas turli turdagi ilonlar rasmlari bilan kitoblar chop qilishar edi. Bu holat 2006 yilgacha davom etdi.

Payton (python) yoki Piton (ПИТОН) – bu qanday talaffuz qilinadi?

Bu ingliz teleko'rsatuvining nomi bo'ladimi yoki “ilon” so'zining inglizcha tarjimasi bo'ladimi, “Python” nomi Pyton kabi to'g'ri talaffuz qilinadi. Biroq, rus dasturchilarining taxminan 80% “Piton” (ПИТОН) deb talaffuz qilishga o'rganib qolishgan.

Ushbu variantlardan birini aniq to'g'ri ikkinchisini xato deb aytish qiyin. Biroq, “Piton” deb talaffuz qilish varianti faqat rus tilida so'zlashuvchi suhbatdoshlar bilan suhbatda foydalanish uchun mos keladi, chunki har qanday xalqaro konferentsiyada “Piton” so'zi tushunarsiz bo'ladi. Chunki ingliz tilida bunday so'z mavjud emas. Bunday konferensiyalarda faqat “Python (Python)” deb talaffuz qilish kerak bo'ladi.

Logotip. Logotipda ikkita ilon kvadrat shaklda tasvirlangan, bu esa ko'pincha foydalanuvchilarni til nomini sudralib yuruvchi bilan bog'lashga undaydi.



1-rasm. Python dasturlash tili logotipi

Bu logotip muallifning ukasi, dasturchi va tip dizayneri Joost van Rossum tomonidan yaratilgan. U logotip dizaynini ishlab chiqshda kvadrat shaklini egallagan ikkita ilon va Flux Regular matn shriftida yozilgan Python so'zidan foydalandi.

Yaratilish tarixi. Til 1980-yillarning oxirida dasturchi Guido van Rossum tomonidan ishlab chiqilgan. O'sha paytda u Niderlandiyadagi matematika va informatika markazida ishlayotgan edi.

Gvido van Rossum maktab yillaridanoq apparat vositalari bilan ishlashni yaxshi ko'ardi va u tengdoshlari tomonidan qo'llab-quvvatlanmasa ham, ma'qullamasa ham, bu uning mustaqil ravishda dasturlash tilini rivojlantirishiga to'sqinlik qilmadi.

Rossum bo'sh vaqtlarida Pythonda ishlagan va u bir paytlar rivojlanishiga yordam bergan ABC dasturlash tiliga asoslanib ishlagan.

Python dasturlash tili tarixidagi bosqichlar :

•**1991-yilning fevralda** python tilining kodlari *alt.sources* saytida chop etildi. Til ob'ektga yo'naltirilgan yondashuvga amal qildi va obyekt, class, metodlar, merosxorlik, funktsiyalar, istisnolarni qayta ishlash va barcha asosiy ma'lumotlar tuzilmalari bilan ishlash imkoniyatiga ega edi.

•**2000 yilda Pythonning ikkinchi versiyasi chiqdi.** Unga ko'plab muhim vositalar, jumladan Unicode yordami va qoldiqlarni yig'uvchi funksiyasi qo'shildi.

•**2008-yil 3-dekabrda Pythonning uchinchi versiyasi ishlab chiqildi.** Bu versiyasi hozircha oxirgi versiya hisoblanadi. Bu versiyada tilning ko'pgina xususiyatlari qayta ishlangan va oldingi versiyalar bilan mos kelmaydigan holga keltirildi. Uchinchi versiyaning funktsionalligi ikkinchisidan kam bo'lmasada, tilning rivojlanishi ikki tarmoqqa bo'lingan. Kimdir eski loyihalarni qo'llab-quvvatlash uchun Pythonning 2-versiyasida foydalanishni davom ettirsa, kimdir butunlay uchinchi versiyada ishlashni tanladi.

Ikkinchi versiyaning tugatilish vaqti 2015 yilga belgilangan edi. Biroq barcha mavjud kodlarni Python 3 ga o'tkazishga ulgurmaslikdan qo'rqib, Python 2 ning ishlash muddati 2020 yilgacha uzaytirilgan.

Pythonning sintaksisi uni har doim boshqa dasturlash tillaridan ajratib turadi. U ortiqcha operatorlardan xalos qilingan. Oddiy ingliz tili bilan sintaksisning o'xshashligi hatto oddiy foydalanuvchiga ham kodni tushunishga imkon beradi. Bundan tashqari, dasturchi kamroq kod yozadi, chunki ortiqcha belgilarni ishlatishning hojati bo'lmaydi. Masalan ; { }....

Pythonning oddiyligi qisman tilning ABC tiliga asoslanganligi bilan bog‘liq bo‘lib, u dasturlashni va dasturchi bo‘lmaganlarning kundalik ishlarini o‘rgatish uchun ishlatilgan.

Python kod yozishni soddalashtiradi va rivojlanishni tezlashtiradi, chunki u quyidagi xususiyatlarga ega:

- **Dinamik tip.** Dasturchi o‘zgaruvchilar tipini ko‘rsatishi shart emas, tilning o‘zi o‘zgaruvchiga qanday ma’lumot yuklanganiga qarab tipini aniqlab oladi.

- **Funktsiya orqali bir nechta qiymatlarni qaytarish.** Ushbu qiymatlarni vergul bilan ajratilgan holda to‘rtburchak qavslar ichiga yoziladi va avtomatik ravishda ro‘yxatga aylantiriladi. Funktsiyadan massivni qaytarish uchun `“return ro‘yxat_nomi”` yozish kifoya.

- **Avtomatik xotiradan joy ajratish.** Dasturchi o‘zgaruvchilarga xotiradan joy ajratish uchun ortiqcha kod yozishi talab qilinmaydi. Bu, bir tomondan, dasturchining dastur ustidan nazoratini kamaytiradi, ikkinchi tomondan, rivojlanish sezilarli darajada tezlashadi.

- **Chiqindilarni yig‘uvchi.** Agar ob‘ekt keraksiz bo‘lib qolsa, u chiqindi yig‘uvchi tomonidan avtomatik ravishda o‘chiriladi. Chiqindi yig‘uvchi sizga xotiradan foydalanishni optimallashtirish va keraksiz narsalarni qo‘lda o‘chirmaslik imkonini beradi.

- **a, b = b, a ifodasi.** Bu ifoda o‘zgaruvchilarning qiymatlarini o‘zaro almashtiradi. Endi **a** o‘zgaruvchining qiymati **b** o‘zgaruvchisiga yuklandi va aksincha. Shunday qilib, siz nafaqat ikkita o‘zgaruvchining, balki uch va undan ortiq ob‘yektlarning qiymatlarini mos ravishda bir-biriga almashtirishingiz mumkin. Faqat taraflardagi o‘zgaruvchilar soni teng bo‘lishi talab qilinadi.

- **O‘zgaruvchi tipining qiymatga bog‘liqligi.** O‘zgaruvchining qiymati - bu uning turini va boshqa xususiyatlarini belgilaydigan atributlarga ega bo‘lgan qandaydir ob‘ekt. O‘zgaruvchi esa bu ob‘ektning nomidir. Ushbu yondashuv qat’iy tipdagi ta’riflarga bo‘lgan ehtiyojni yo‘qqa chiqaradi va o‘zgaruvchilarga boshqa turdagi qiymatni qayta tayinlashni sezilarli darajada soddalashtirdi.

•**For tsikli** . Pythonda massivlar, ro'yxatlar va boshqa konteynerlar bilan ishlash oson va qulay. Uning barcha elementlarini takrorlash zarur bo'lganda, konstruktsiya quyidagicha ko'rinadi: **for x in konteyner:** (*iteratsiya 0 dan oxirgi -1 elementgacha elementlarni x ga qaytaradi*). Agar siz tsiklni aniq bir miqdorda takrorlanishini amalga oshirish uchun **range()** funksiyasidan foydalanish kerak bo'ladi: **for x in range(1,9):** (*tsikl 1 dan 8 gacha bo'lgan qiymatlarini x ga uzatadi*).

•**Machine learning sohasidagi yetakchiligi.** Python dasturlash tili "Mashina o'rganish" (Machine learning) sohasida yaqqol yetakchilik qilib kelmoqda. Ilm-fan bilan u yoki bu tarzda shug'ullanadigan odamlar kod yozish kabi narsalarga ko'p vaqt sarflamaslikni afzal ko'radilar, shuning uchun Python ularga yuklangan vazifalarni amalga oshirish uchun juda mos keladi.

Python ham soddalik, ham kuchli vositalarni o'zaro birlashtiradi. U deyarli har qanday dasturning prototipini yaratish uchun ishlatilishi mumkin.

Kod misoli :

```
def kattasi(a, b):
    if a > b:
        print(a, "katta", b, "dan")
    else:
        print(b, "kichik", a, "dan")
def max_mass (massiv):
    max = 0
    for x in massiv:
        if massiv[x-1] > max:
            max = massiv[x]
    return max

def 2mass(massiv):
    massiv = massiv * 2
```

```

        return massiv

print("Oddiy Python dasturi")

a = [1,2,3,6,1,6]
kattasi (1,5)
r1 = max_mass(a)
r2 = 2mass (a)

print("Massiv max elementi -", r1)
print("Ikkilangan massiv - ", r2)

```

Dastur natijasi:

```

Oddiy Python dasturi
5 katta 1 dan
Massiv max elementi - 6
Ikkilangan massiv - [1, 2, 3, 6, 1, 6, 1, 2,3,6,1,6]

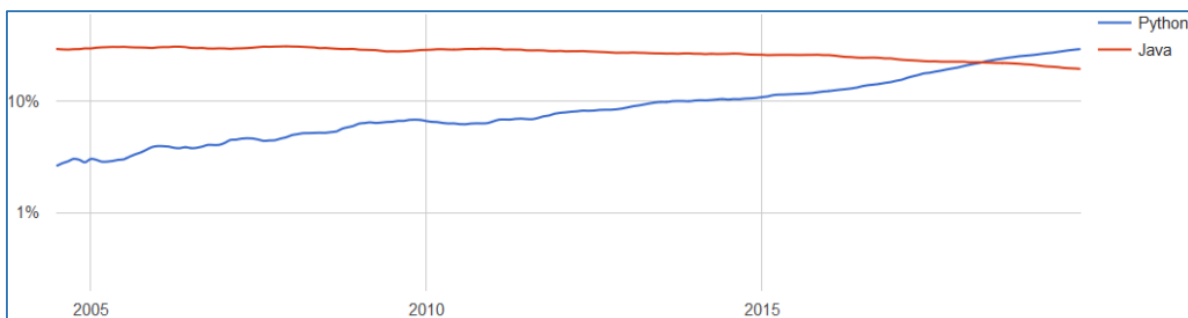
```

Ommalashganligi. Til 30 yoshdan oshgan bo'lsada, butun dunyodagi dasturchilar orasida juda keng tarqalgan. Python deyarli har bir o'rta yoki katta loyihada, asosiy ishlab chiqish vositasi sifatida bo'lmasada, prototiplash yoki uning bir qismini yozish uchun vosita sifatida ishlatiladi.

U o'z atrofida juda katta dasturchilar guruhini to'pladi.

Stackoverflow saytida o'tkazilgan so'rov natijalariga ko'ra, Python deyarli 39% ovoz bilan 7-o'rinni egalladi.

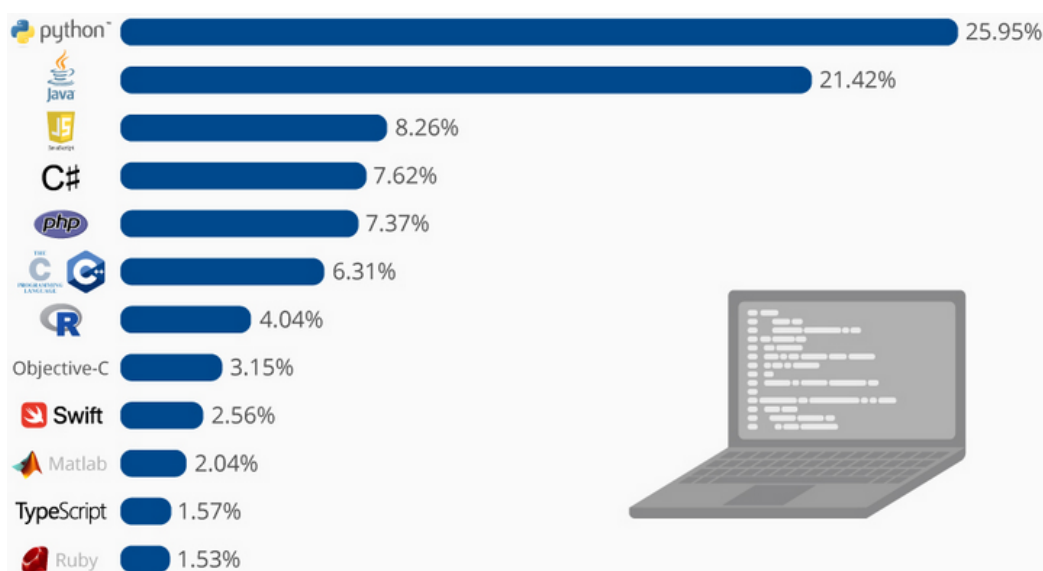
PYPL. Ushbu indeks tilni o'rganish materiallari bilan bog'liq qidiruv so'rovlari soniga asoslanadi.(2-rasm)



2-rasm. PYPL indeksi bo'yicha solishtirma sxemasi

PYPL ma'lumotlariga ko'ra, Python mashhurligi bo'yicha Javadan 10% ga oldinda.

statista.com. Xizmat turli xil statistik ma'lumotlarni taqdim etadi, ular orasida dasturlash tillarining mashhurligi bo'yicha statistikalari ham mavjud. (3-rasm)



3-rasm. **Statista.com** saytidagi solishtirish diagrammasi

Ish tezligi. Dasturchilar ko'pincha o'zlariga savol berishadi: "Python-dan foydalanish ishlashning pasayishiga olib keladimi?". Batafsil tekshiruvsiz hech qanday xulosa chiqarmang.

Agar biz faqat kodni bajarish tezligini hisobga olsak, Python C kabi boshqa dasturlash tillaridan past ekanligi ayon bo'ladi. Darhaqiqat, dinamik terish, izohlash va dasturchi ishini osonlashtiradigan boshqa xususiyatlar ishlashning pasayishiga olib keladi. Biroq, zamonaviy IT-da nafaqat dasturlarning tezligi, balki ularni ishlab chiqish tezligi ham muhimdir. Ishlab chiqish, sinovdan o'tkazish, disk raskadrovka va qo'llab-

quvvatlash - bularning barchasi juda ko‘p pul talab qiladi. Agar Python dasturlarining tezligi bo‘yicha past bo‘lishi mumkin, ammo rivojlanish tezligida bo‘yicha unga teng keladigani yo‘q.

Dasturchilar Pythonda yaratilgan dasturlarni bajarilish tezligini oshirish uchun turli yo‘llardan foydalanadilar:

•**C kodini o‘rnatish** . Ushbu texnikadan foydalanib, siz ishlashni sezilarli darajada yaxshilashingiz mumkin, odatda vaqt birligida ko‘p so‘rovlarni qayta ishlaydigan kod bo‘limlari C tilida yoziladi. Masalan, bitta ma’lumotlar bazasidan ma’lumotlarni qabul qiladigan, uni qayta ishlaydigan va boshqasiga yuboradigan funksiyalar, agar o‘tadigan ma’lumotlarning miqdori yetarlicha katta bo‘lsa, C tilida yozilgani afzal xisoblanasi.

•**ifodalarni optimallashtirish**. Python dasturlarining tezligi ifodalarning ishlash tezligiga juda bog‘liq. Quyidagi jadvalda tezroq va sekinroq ishlovshi ifodalar keltirilgan.

1-jadval

Tezroq	Sekinroq
$a, b = c, d$	$a = c; b = d$
$a < b < c$	$a < b \text{ and } b < c$
not not a	bool(a)
$a = 5$	$a = 2 + 3$

•**Dasturni testlash uchun tayyor modullar**. Kodning qaysi bo‘limlari umumiy ish faoliyatini sezilarli darajada kamaytirishini aniqlash uchun dasturchi testlash uchun maxsus modullardan foydalanishi mumkin. Shunday qilib, bu modullar yordamida qaysi kodni optimallashtirish yoki C kodi bilan almashtirish kerakligini bilib olishi mumkin.

•**Tayyor asboblari.** Aksariyat vazifalar uchun samarali echimlar allaqachon ishlab chiqilgan. O'z yechimingizni noldan yozishdan ko'ra, ba'zi kutubxonaning tayyor, tuzatilgan kodini ishlatish yaxshiroqdir, bu 100% samarali bo'lmaydi.

Pythonda qanday dasturlar yozish mumkin?

Python tilida turli sferalarda dasturlar tuzish mumkin.

Back-end – saytning dasturiy qismi. Saytning server tomonini rivojlantirish uchun turli frameworklar qo'llaniladi: **Django** va **Flask**. Bu frameworklar Pythonni boshqa mashhur vositalar bilan raqobatlashadigan imkoniyatlarga ega server tomonidagi dasturlash tiliga aylantiradilar.

PHP tili server tomonidagi web ishlab chiqish bozorining ko'p qismini nazorat qilsada, tobora ko'proq dasturchilar Pythonda ishlab chiqishni afzal ko'rishmoqda.

Blokcheyn. Blokcheyn - bu ketma-ket bloklar zanjiri bo'lib, unda har bir blok ma'lumotni o'z ichiga oladi va har doim oldingisiga ulanadi. Texnologiya har qanday sohada qo'llanilishi mumkin. Ayniqsa moliya sektorida va bitcoin kriptovalyutasi sohasida mashhurdir.

Blokcheyn axborot xavfsizligi va ochiqligini o'zida mujassamlashtiradi. U foydalanuvchiga dunyoning istalgan nuqtasidan ma'lumotlarga kirish imkonini beradi.

Bot. Bu ma'lum bir vaqtda yoki berilgan signalga javoban ba'zi harakatlarni avtomatik ravishda bajaradigan dasturdir. Botlar inson xatti harakatlarini bevosita taqlid qilishi mumkin. Shuning uchun ular ko'pincha texnik yordam ko'rsatishda (chat botlarida), Internetda ma'lumot qidirishda (qidiruv botlarida), virtual dunyoda odam yoki boshqa mavjudotning harakatlarini taqlid qilishda (kompyuter o'yinlari) zarur bo'ladi.

Python sizga tezkor xususiyatga ega va nisbatan aqlli botlarni yaratish imkonini beradi. Shuni tushunish kerakki, botlar 500 qatorli kodlardan iborat oddiy dastur emas. Biznes uchun bot yaratish buyurtmasi bir necha millionga tushishi mumkin. Narx insondan farqlash qiyin bo'ladigan botni loyihalash juda qiyin ekanligi

bilan bog'liq. Muloqotning ko'plab variantlarini ta'minlash, inson xatti-harakatlarining omillarini tahlil qilish va ularni dasturda amalga oshirish kerak. Oddiy qilib aytganda, faqat nol va birlarni tushunadigan mashinadan siz ibtidoiy "miya" qilishingiz kerak bo'ladi.

Malumotlar bazasi. Ma'lumotlar bazasi umumiy xususiyatlar va maxsus qoidalarga muvofiq tizimlashtirilgan ma'lumotlardir. Har qanday yirik loyihada ma'lumotlar bazalaridan foydalaniladi. Ular foydalanuvchilar haqidagi ma'lumotlarni, dasturdagi o'zgarishlarni va hokazolarni saqlaydi.

Ma'lumotlar bazasini boshqarish tizimi Pythonda yozilishi mumkin.

Kengaytirilgan haqiqat (Дополненная реальность). "To'ldirilgan reallik" virtual texnologiyalar yordamida jismoniy dunyoni to'ldiradi. Ya'ni, virtual ob'ektlar real muhitga proyeksiya qilinadi va oddiy jismoniy ob'ektlarning xususiyatlari va xatti harakatlariga taqlid qiladi.

"Kengaytirilgan haqiqat"ni turli 3D filmlarda guvohi bo'lish mumkin. Haqiqiy dunyoda u, masalan, jangovar jangchilarda (maqsad tizimi) ishlatiladi.

"Kengaytirilgan haqiqat" ishi teglar bilan o'zaro ta'sirga asoslangan. Elektron qurilma ma'lumot oladi va atrofdagi makonni tahlil qiladi, kompyuter ko'rish yordamida u odamning oldida nimani ko'rayotganini "Tushunadi". Keyin qurilma real dunyoda "Virtual qatlam"ni qoplaydi.

"Kengaytirilgan haqiqat" sferasida yaratilgan mukammal dasturlar taxminan 7-10 ming AKSH dollarini tashkil qiladi. Ularni loyihalash va yozish oson emas, 3D-dizaynerlardan dasturchilargacha turli mutaxassislar ishlab chiqish jarayonida tinmasdan mehnat qilishadi.

Python – bunday sferadagi loyihalarni yaratish uchun ajoyib tanlov.

BitTorrent mijozi. BitTorrent - bu Internet orqali katta hajmdagi ma'lumotlarni tezlikda uzatish, qabul qilish imkonini beruvchi noyob texnologiya.

BitTorrentning 6-versiyasidan keyingi versiyalari butunlay Pythonda yozilgan. Keyinchalik u C++ da butunlay qayta yozilgan bo'lsa-da, bu Python tilidan xuddi shunga o'xshash vazifalarni bajarish uchun foydalanish mumkinligini anglatadi.

Neyron tarmoq. “Neyron tarmoq” tushunchasi dasturlashga biologiyadan kirib kelgan. Biologiyada neyron tarmoq bir-biriga bog'langan neyronlar ketma-ketligidir. Dasturiy ravishda yaratilgan neyron tarmoqlar nafaqat ma'lumotni tahlil qilish va saqlash, balki uni xotira qayta ishlab chiqishga qodir.

Ular inson miyasi tomonidan bajariladigan hisob-kitoblarni talab qiladigan murakkab masalalarni hal qilish uchun ishlatiladi. Odatda, neyron tarmoqlar biror narsani xususiyatlari bo'yicha tasniflash, bashorat qilish, masalan, fotosurat yoki videodan odamni tanib olish kabi funksiyalarni amalga oshirish uchun foydalaniladi.

Python neyron tarmoqlarni rivojlantirishda yaqqol yetakchi hisoblanadi. Standart vositalardan tashqari, u “*Mashinani o'rganish*” sohasida juda ko'p kutubxonalar mavjud. Buning yordamida hatto katta va murakkab loyihalar nisbatan tez yozilishi mumkin.

Parser - bu ma'lumotlarni yig'ish va qayta ishlash uchun mo'ljallangan dastur. Foydalanuvchi valyuta kursi kabi ma'lumotlarni tahlil qilishi yoki turli kompaniyalar aksiyalaridagi o'zgarishlarni kuzatishi va tahlil qilishi mumkin.

Parser turli xil tillarda yozilishi mumkin. Ya'ni parser dasturlar boshqa dasturlash tillari yordamida ham yaratish mumkin. Lekin parser (ma'lumot yig'uvchi) dasturlarni python tilida boshqa dasturlash tillariga qaraganda tez va samarali yozish mumkin.

O'yin yaratish. Pythonda katta o'yinlar yaratilmaydi. U prototipni (demo versiyasini) ishlab chiqish yoki ba'zi bir qismini amalga oshirish uchun ishlatiladi (masalan, o'yinning server qismidagi logik qismi).

Kichik loyihani yozish uchun siz kichik 2D o'yinni yaratish uchun barcha kerakli vositalarni taqdim etadigan Pygame kutubxonasidan foydalanishingiz mumkin.

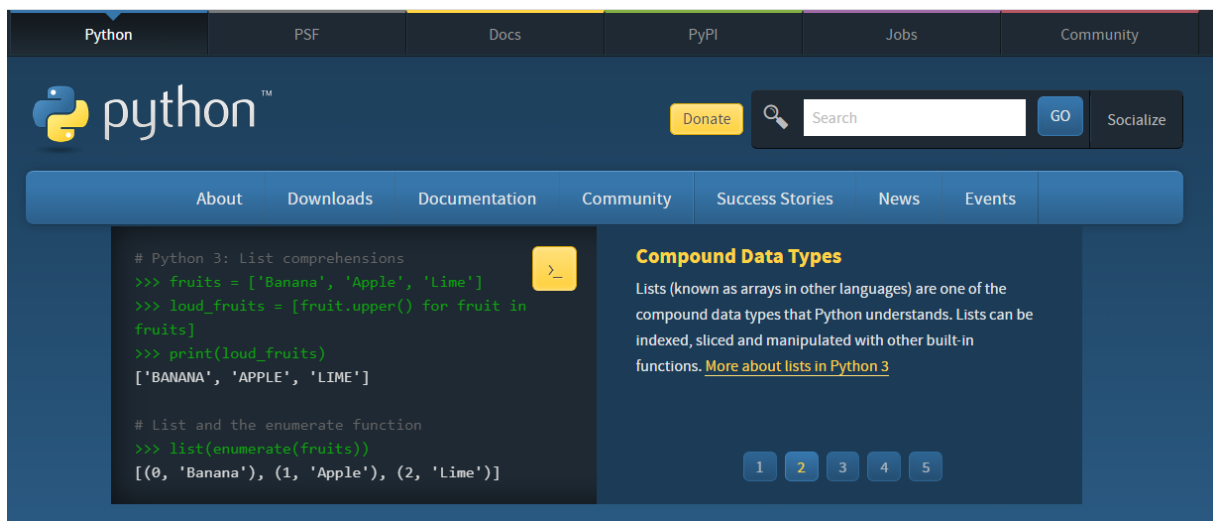
Nazorat savollari:

1. Python qanday dasturlash tillari sirasiga kiradi. Interpretatsiya qilinuvchimi yoki kompilyatsiya qilinuvchi?
2. Python nomi qanday tarixga ega?
3. Python logotipi kelib chiqishi qanday?
4. Python tili bilan qanday dasturlar tuzish mumkin?
5. Pythonning reytingi qanday?
6. Python tilining boshqa tillardan qanday ustunliklari bor?

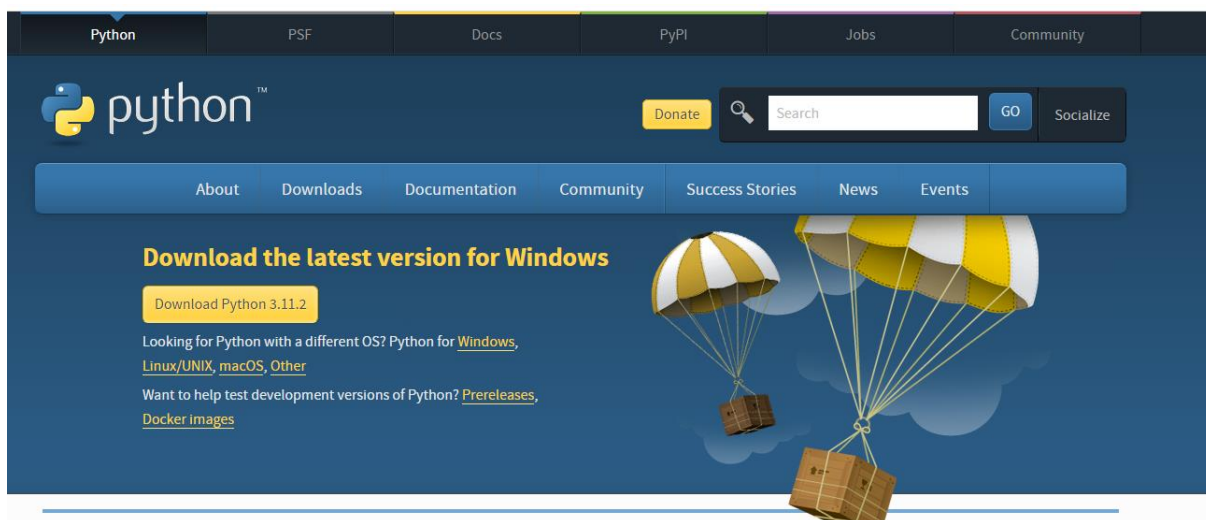
1.2.8. Python va PyCharm dasturini Windowsga o'rnatish va birinchi dastur

Python dasturini windows operatsion tizimiga o'rnatish.

1-qadam. Python-ni yuklab olish va o'rnatish uchun Pythonning rasmiy veb sayti python.org ga kiriladi (4-rasm) va **Downloads** bo'limidan pythonning oxirgi versiyasini tanlanadi (5-rasm).



4-rasm. [Python.org](https://python.org) saytining ko'rinishi



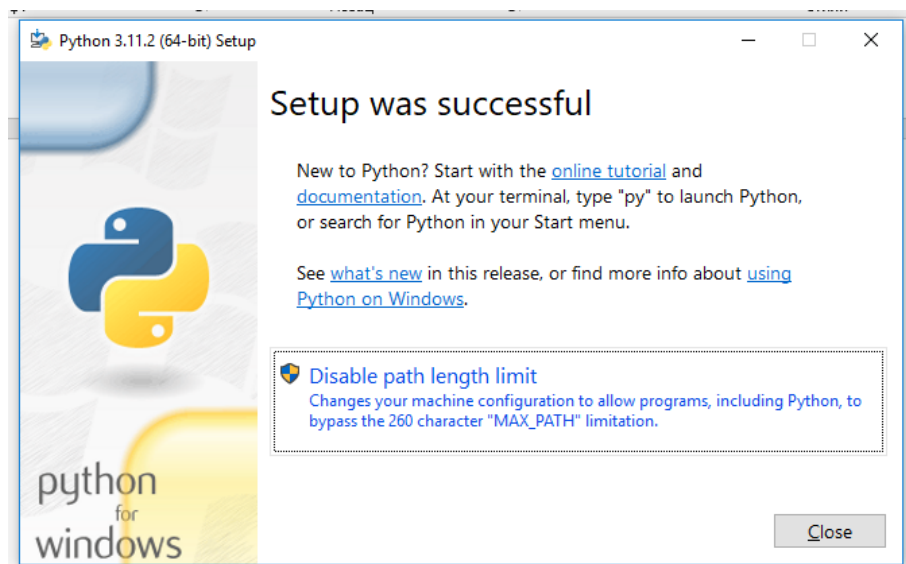
5-rasm. [Python.org](https://python.org) saytidan python dasturini ko‘chirib olish oynasi

2-qadam. Pythonning tegishli oxirgi versiyasiga doir, o‘rnatiluvchi faylni yuklab olish tugallangach, Pythonni o‘rnatish uchun yuklab olingan, yuklanuvchi faylni (.exe faylini) sichqoncha tugmasi bilan ikki marta bosish orqali ishga tushiriladi. Keyin paydo bo‘ldan oynadan "Install now" tugmasini bosiladi. (6-rasm) Shundan so‘ng Python o‘rnatilishi jarayonini ko‘rishingiz mumkin.



6-rasm. Python.exe faylini ishga tushirishdan hosil bo‘ladigan o‘rnatish oynasi ko‘rinishi

3-qadam. O‘rnatish tugallangach, o‘rnatish muvaffaqiyatli amalga oshirilgan bo‘lsa, quyidagi panel ekranda paydo bo‘ladi va “Close” tugmasini bosilib paneldan chiqiladi.



7-rasm. O‘rnatilish muvoffaqiyatli tugaganligi to‘g‘risidagi oyna

Bilamizki barcha dasturlash tillari uchun o‘zining kodlarini kiritish uchun qulay kompilyator yoki interpretatori mavjud bo‘ladi. Masalan C++ va C# uchun VisualStudio bo‘lsa, web bilan ishlovchilar uchun VisualStudioCode, SublimeText3 va shu kabi dasturlardan foydalaniladi.

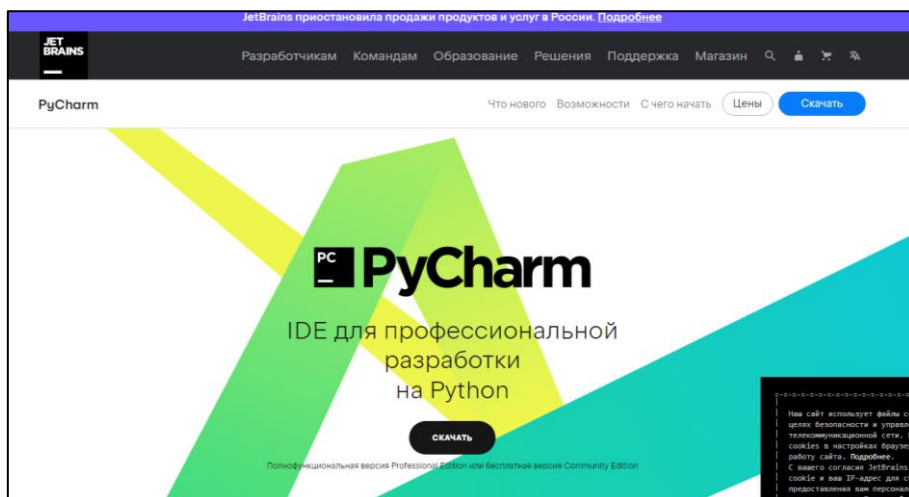
Python tili bilan ishlashda dasturchi dastur kodini interpretatsiya qilib beruvchi, o‘ziga qulay dasturni o‘rnatib olishi kerak bo‘ladi. Bunday dasturlar turi juda ko‘p. Masalan: VSCode, SublimeText, PyCharm, VisualStudio, Atom bu ro‘yxatni yana davom ettirish mumkin. Ammo bulardan bittasini tanlash kerak bo‘lganda PyCharm dasturini tanlash maqsadga muvofiq bo‘ladi.

PyCharm - JetBrains tomonidan ishlab chiqilgan kross-platformali interpretator dasturi. Pycharm dasturi Python uchun zarur bo‘lgan barcha vositalarni taqdim eta oladi. Quyida Windows muhiti uchun Python va PyCharmni o‘rnatish bo‘yicha batafsil ko‘rsatmalar keltirib o‘tilgan.

PyCharm dasturini windows operatsion tizimiga o‘rnatish

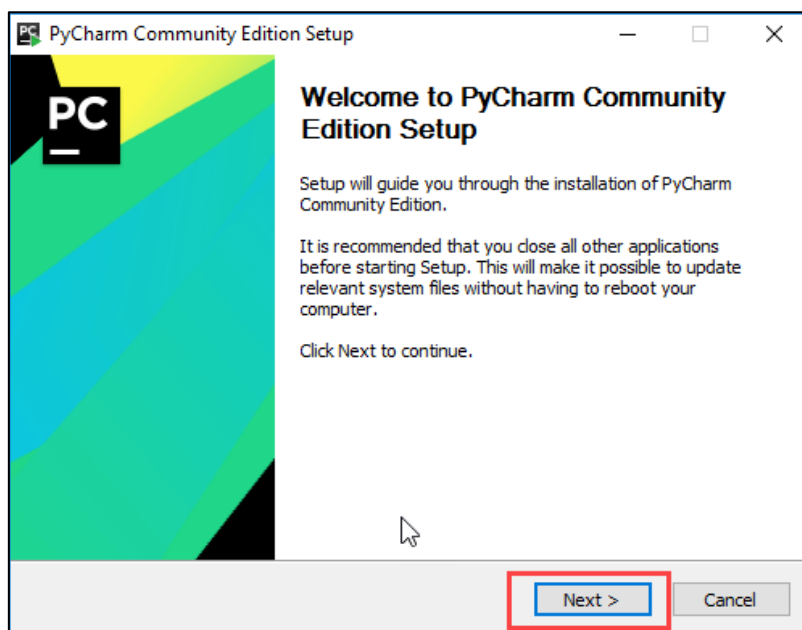
Dasturni o‘rnatish judayam oddiy. Boshqa o‘rnatiluvchi dasturlar kabi o‘rnatiladi. Buni qadamma qadam ko‘rib chiqamiz:

PyCharmni yuklab olish uchun jetbrains.com/ru-ru/pycharm (8-rasm) veb saytiga tashrif buyurish kerak va “**Скачать**” tugmasini bosiladi.



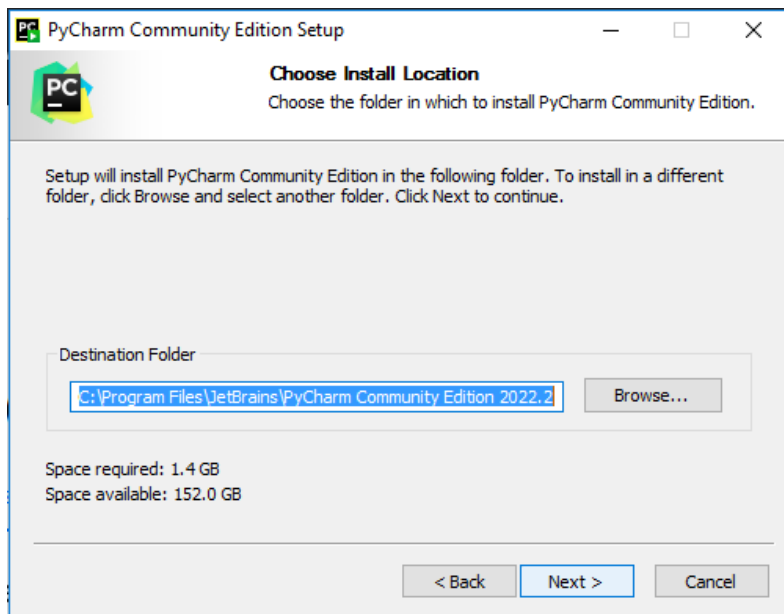
8-rasm. [Jetbrains.com/ru-ru/pycharm](https://jetbrains.com/ru-ru/pycharm) saytining ko‘rinishi

Yuklab olish tugallangach, PyCharm-ni o‘rnatish uchun ko‘chirib olingan, yuklanuvchi faylni (.exe faylini) ishga tushiriladi. Installer ishga tushadi va “**Next**” tugmasini bosish orqali keyingi oynasiga o‘tiladi.



9-rasm. PyCharm dasturini o‘rnatish jarayoni

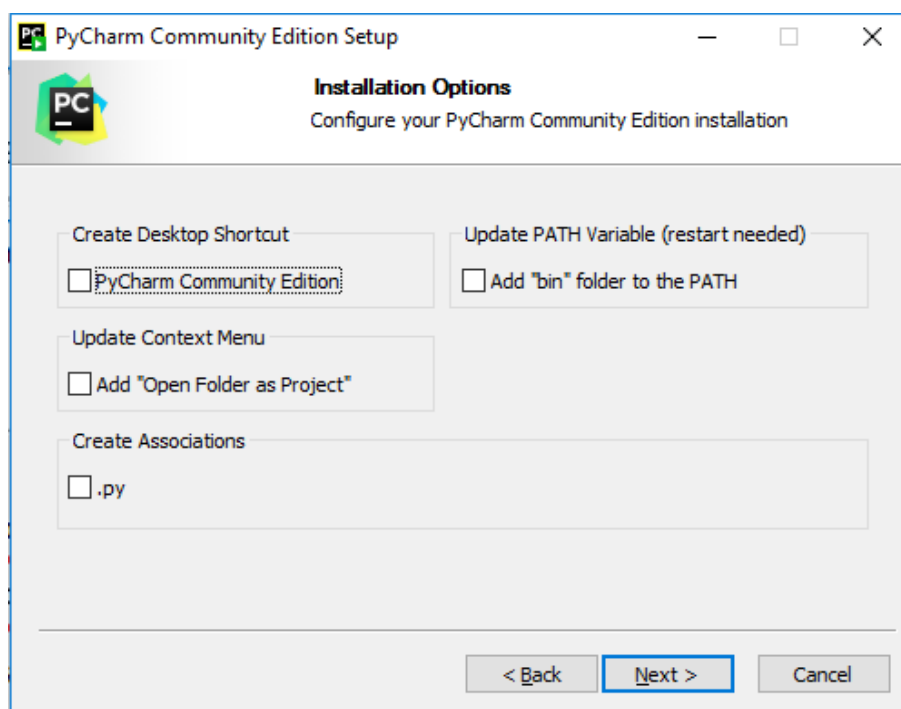
Agar dasturni o‘rnatilish yo‘lini ko‘rsatilmasa, unda dastur kompyuterning operatsion tizimi ko‘rsatgan joyga o‘rnatiladi. Kimdir dasturning o‘rnatilish joyini o‘zgartirishni istasa unda, o‘rnatilish joyini “**Browse..**” tugmasini bosib ko‘rsatib ketish kerak bo‘ladi.



10-rasm. PyCharm dasturini qaysi URL ga o‘rnatishni tanlash oynasi

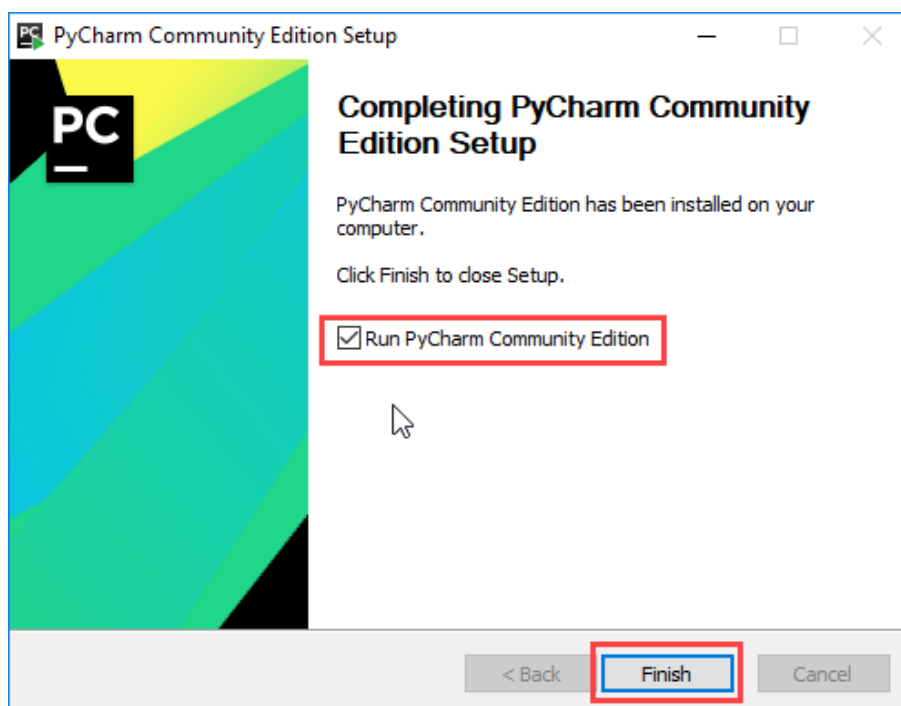
Keyingi bosqichda, ish stolida dastur yorlig‘ini yaratish mumkin bo‘ladi. Buning uchun “Create desktop Shortcut” bo‘limini belgilab, so‘ngra “Next” tugmasini bosiladi.

Add “bin” folder to the PATH bo‘limini tanlab qo‘yish maqsadga muvofiq bo‘ladi (11-rasm). Chunki bu hozir belgilanmasdan o‘rnatiladigan bo‘lsa, keyinchalik bu Path ni qo‘lda ko‘rsatib o‘rnatish talab qilinadi. Bu esa noqulaylik tug‘dirishi mumkin.



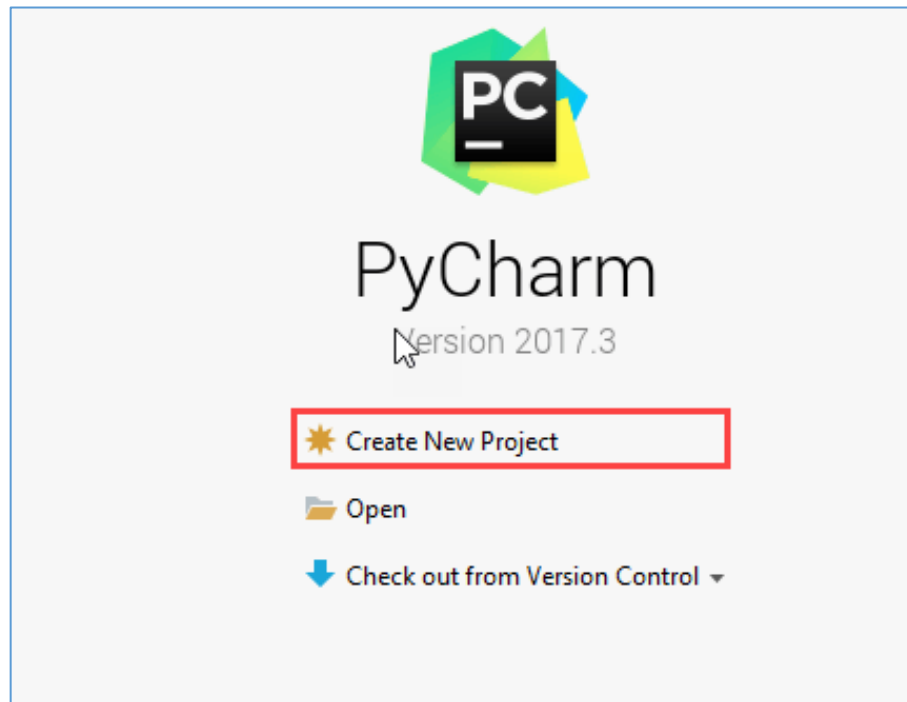
11-rasm. "Installeion option" oynasi

O‘rnatishni tugatish. O‘rnatish tugallangandan so‘ng PyCharm o‘rnatilganligi haqida xabar paydo bo‘ladi. Agar dasturchi uni ishga tushirishni xohlasa, avval "Run PyCharm Community Edition" katagiga belgi qo‘ying va "Finish" tugmasini bosishi kerak.



12-rasm. O‘rnatish yakunlanganligi haqidagi habar oynasi

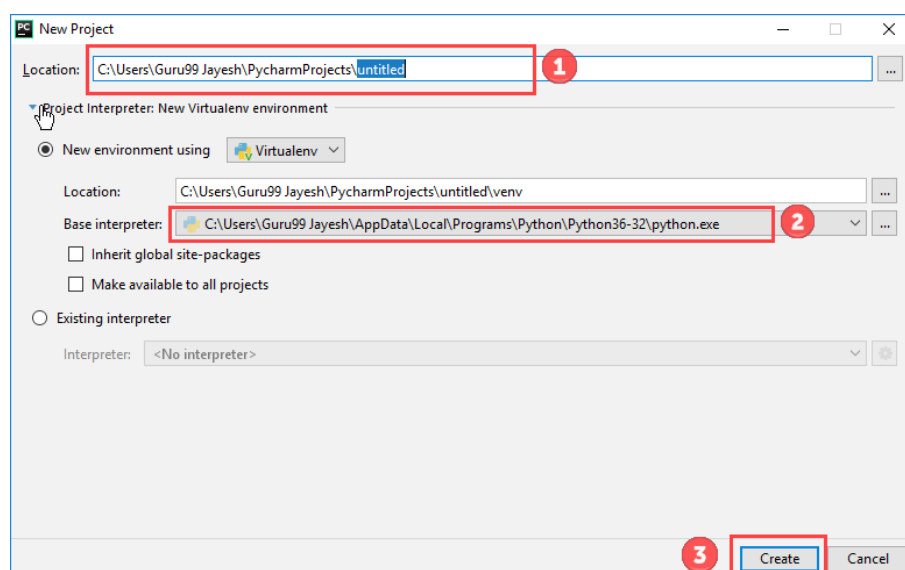
PyCharm dasturini ishga tushirilgandan keyin quyidagi “Welcome to PyCharm” oynasi paydo bo‘ladi. Yangi loyiha yaratish uchun "Create New Project" tugmasini bosiladi.



13-rasm. Yangi loyiha yaratish oynasi

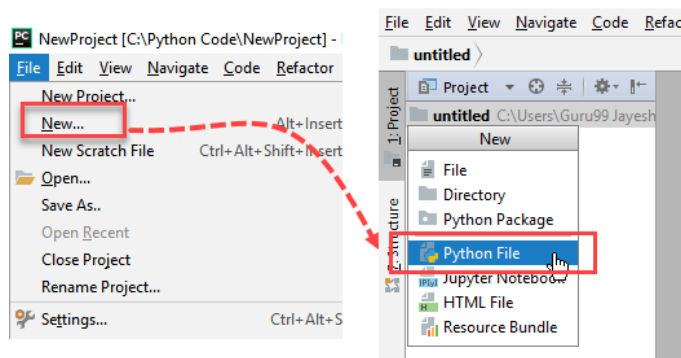
Loyiha yaratilayotganda uning nomi va joyini tanlanadi. Agar joylashish joyini o‘zgartirilmasa, unda operatsion tizim python qayerga o‘rnatilgan bo‘lsa, o‘sha joyda proyekt yaratiladi.

Nom berish va joy tanlangandan keyin "Create" tugmasini bosiladi.



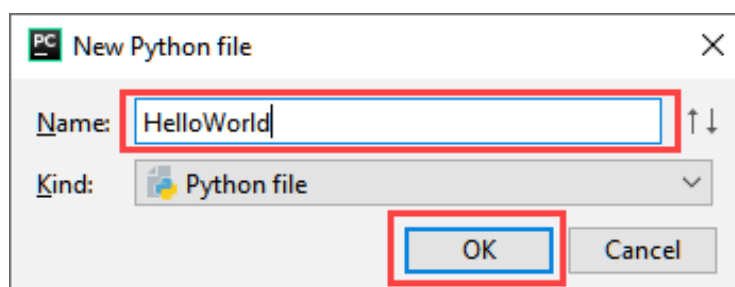
14-rasm. Yangi loyiha yaratish uchun loyihaga nom berish oynasi

Hosil bo‘lgan oynadan **File** menyusiga o‘tiladi va **New** buyrug‘ini tanlangandan keyingi kontekst menyudan **Python file** bo‘limi tanlanadi.



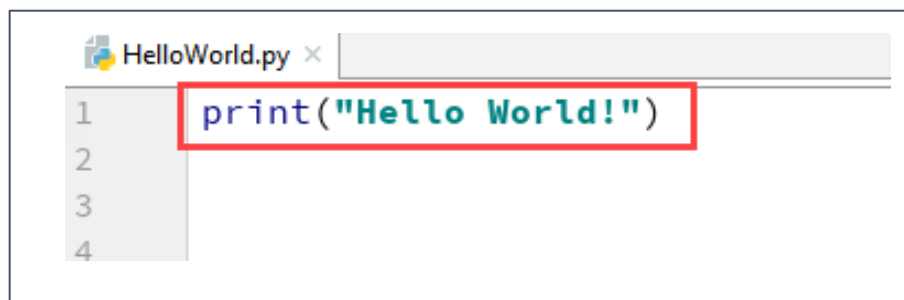
15-rasm. Loyiha ichida python fayl yaratish oynasi

Yangi paydo bo‘lgan oynadan fayl nomini kiritiladi va **OK** yoki klaviaturadan **Enter** tugmasini bosish kerak bo‘ladi. (16-rasm)



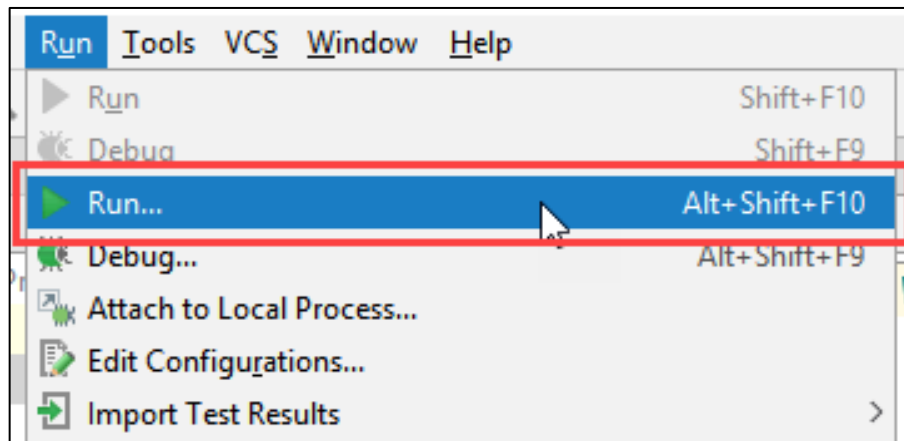
16-rasm. Loyiha ichida yaratilgan python faylga no berish oynasi

Shundan so‘ng Yangi hosil bo‘lgan oynada “Salom dunyo” dastur yozish uchun quyidagi kodni kiritish kifoya: `print("Salom Dunyo!")`



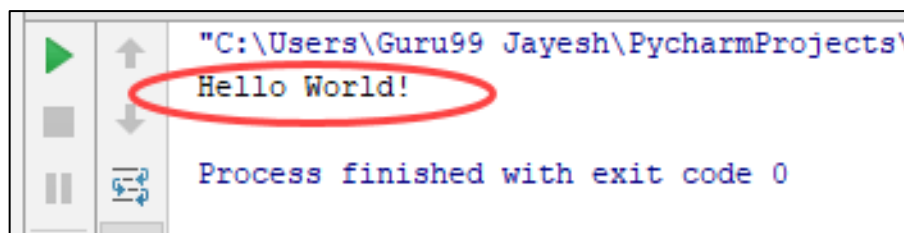
17-rasm. Salom dunyo dasturi

“Salom dunyo” dasturini ishga tushirish uchun menular satridan “RUN” menyusini tanlanadi va u yerdan “Run” buyrug‘i tanlanadi. Yoki oddiygina dastur kodining ustiga yoki bo‘sh joyga sichqoncha o‘ng tugmasini bosiladi va hosil bo‘lgan konteks menudan “Run” buyrug‘ini tanlash orqali ishga tushirish mumkin.



18-rasm. Salom dunyo dasturini ishga tushurish jarayoni

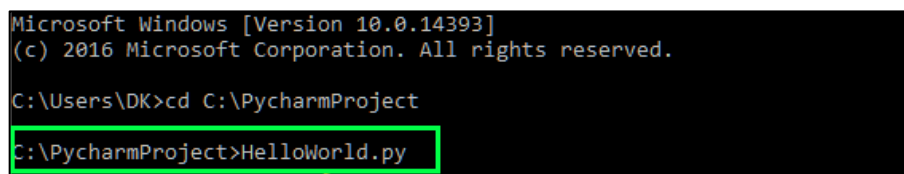
Dastur natijasi ekranning pastki qismida, konsolda paydo bo‘ladi.



19-rasm. Salom dunyo dasturining natijasi

Python faylni konsoldan ishga tushirish

Agar kompyuterda python faylni ochish uchun interpretatorlar o‘rnatilmagan bo‘lsa, python faylni “**Buyruq satri**”dan ishga tushirish mumkin. Dasturni ishga tushirish uchun faylga to‘g‘ri yo‘lini ko‘rsatish kerak.



20-rasm. Salom dunyo dasturini konsoldan ishga tushirish

Kodning natijasi quyidagicha bo‘ladi (21-rasm):

```
C:\Users\DK>cd C:\PycharmProject  
C:\PycharmProject>HelloWorld.py  
Hello World  
C:\PycharmProject>
```

21-rasm. Salom dunyo dasturini konsoldan ishga tushurishdan olingan natijasi

Nazorat savollari:

1. Python kodlari qaysi dasturlarga yozish mumkin?
2. PyCharm dasturi qaysi kompaniyaning dasturiy maxsuloti hisoblanadi?
3. Krosplatformali dastur deganda nimani tushunasiz?
4. Pythonni o'rnatilish jarayonini tushuntirib bering.
5. PyCharmni o'rnatilish jarayonini tushuntirib bering.
6. Python faylning formati qanday bo'ladi?

II-BOB. PYTHON ASOSLARI

2.1.§. Pythonda dastur strukturasi

Python dasturlash tilida yozilgan dastur ifodalar jamlanmasidan iborat. Har bir ifoda yangi qatorga joylashtiriladi. Masalan:

```
print(2 + 3)
print("Hello")
```

Python dasturlash tili kodida ifodalarning oldidan qoldirilgan oraliq masofalar katta ro‘l o‘ynaydi. Agar dasturchi tomonidan bilmasdan yoki chiroy uchun qoldirilgan har qanday oraliq masofa xatolik kelib chiqishiga sabab bo‘lishi mumkin. Masalan, garchi kod yuqoridagi bilan deyarli bir xil bo‘lsada, quyidagi holatda dasturda oraliq masofalar bilan bog‘liq xatolik sodir bo‘ladi,:

```
print(2 + 3)
print("Hello")
```

Shuning uchun, python dasturlash tilida dastur kodining boshi satrning boshidan boshlanishi kerak. Bu Python bilan C# yoki Java kabi boshqa dasturlash tillari o‘rtasidagi muhim farqlardan biridir.

Ammo shuni yodda tutish kerakki, tilning ba’zi operatorlarining sintaksisi bir necha qatordan iborat bo‘ladi. Masalan, shart operatori if:

```
if 1 < 2:
    print("salom")
```

Bu dasturda, agar 1 soni 2 dan kichik bo‘lsa, u holda “**salom**” so‘zi ekranga chiqariladi. Bu dasturda print(“Salom”) ifodasining oldida bo‘sh joy bo‘lishi kerak, chunki bu ifoda if shartli konstruktsiyasining bir qismi sifatida ishlatiladi.

Bundan tashqari, kodni yozish mobaynida qoldirib ketiladigan bo‘shliqlar 4 ga karrali probellar soniga (ya’ni 4, 8, 16 va boshqalar) teng bo‘lishi maqsadga muvofiqdir. Ammo 4 ga karrali bo‘lmay qolishi xatolik keltirib chiqarmaydi. Ifoda

oldida qoldiriladigan masofalarning 4 ta probelga yoki 4ga karrali probellar soniga teng bo'lishining sababi shundaki, 1 ta tab 4 ta probelga tengligida.

Harflar katta-kichikligini farqlanishi **(Регистрозависимость)**

Python katta-kichik harflarni turli belgilan sifatida qabul qiladigan tildir. Shuning uchun **print** va **Print** yoki **PRINT** ifodalari turli ifodalarni anglatadi. Shunday ekan agar konsolga "Salom dunyo" xabarini Print funksiyasi orqali chiqarmoqchi bo'lsak, hech qanday narsa chiqara olmaymiz, aksincha xatolik sodir bo'ladi:

```
Print("Hello World")
```

Izohlar

Izohlar – kodning ma'lum bir qismida nima amalga oshirilayotganligini ifodalab foydalanuvchi uchun tushunchalar berib ketish uchun foydalaniladi. Dastur interpretatsiya qilinganda (ishga tushirilganda) interpretator izohlarni e'tiborsiz qoldiradi, shuning uchun ular dasturning ishlashiga ta'sir qilmaydi. Pythondagi izohlar blok va inline izohlarda keladi.

Bitta satrda joylashuvchi izohlardan oldin panjara belgisi- # qo'yiladi:

```
# Konsolga chiqish  
# Salom Dunyo xabarlari  
print("Salom Dunyo ")
```

belgisidan keyingi har qanday belgilar to'plami izohni bildiradi. Ya'ni, yuqoridagi misolda kodning birinchi ikki qatori izohlar hisoblanadi.

Shuningdek izohlar ifodalardan so'ng, til ifodalari bilan bir qatorda joylashgan bo'lishi ham mumkin:

```
print("Salom dunyo")    # Xabarni konsolga chop etish
```

Blok izohlarida sharh matnidan oldin va keyin uchta bitta tirnoq qo'yiladi: *“‘sharh matni’”*. Masalan:

```
'''
    Konsol chiqishi
    Salom dunyo xabarлари
'''
print ("Salom dunyo")
```

Asosiy funktsiyalari

Python ko'plab o'rnatilgan, standart funktsiyalarni taqdim etadi. Ulardan ba'zilari, ayniqsa, til o'rganishning dastlabki bosqichlarida juda tez-tez qo'llaniladi, shuning uchun ularni ko'rib chiqaylik.

Konsolga ma'lumot chiqarishning asosiy funktsiyasi - bu `print()` funktsiyasi hisoblanadi. Biz chiqarmoqchi bo'lgan satr ushbu funktsiyaga argument sifatida uzatiladi:

```
print ("Salom Python")
```

Agar konsolga bir nechta qiymatlarni chop etish kerak bo'lsa, ularni vergul bilan ajratib, `print()` funktsiyasiga beriladi:

```
print("To'liq ismi:", "Tom", "Smit")
```

Natijada, vergul bilan ajratilgan joylarga probel bilan ajratilgan holatda, barcha berilgan qiymatlar, bir qatorga joylashtirilib konsolga chiqariladi:

```
To'liq ismi: Tom Smit
```

Agar **`print()`** funktsiyasi chiqarish uchun javobgar bo'lsa, u holda **`input()`** funktsiyasi axborotni kiritish uchun xizmat qiladi. **`input()`** funktsiyasi kiritilgan ma'lumotni satr tipida qaytaradi va bu satrni o'zgaruvchiga saqlanadi va shu qiymatga o'zaruvchi orqali murojaat qilinadi:

```
name = input("Ismni kiriting:")  
print ("Salom", ism)
```

Konsol chiqishi:

```
Ismni kiriting: Eugene  
  
Salom Eugene
```

Nazorat savollari:

1. Pythonda dastur strukturasi qanday tuzilgan?
2. Satrning oldida qoldirilgan oraliq masofalar (indents) nimani anglatadi?
3. Pythonning sintaksisi boshqa dasturlash tillarining sintaksisidan eng katta farqli jihati nimada?
4. Pythonda harflarning katta kichikligiga sezgirliги (Регистрозависимость) qanday?
5. Pythonda kommentariyalar qanday amalga oshiriladi?
6. Ko‘p satrlik kommentariyalar qanday bajariladi?
7. Pythonda kiritish, chiqarish funksiyalari qanday?

2.2.§. O‘zgaruvchilar va ma’lumotlar turlari

O‘zgaruvchilar

O‘zgaruvchilar qiymatlarni saqlash uchun mo‘ljallangan dastur elementi. Python tilidagi o‘zgaruvchi nomi alifbo belgisi yoki pastki chiziq bilan boshlanishi kerak va alfanumerik belgilar va pastki chiziqni o‘z ichiga olishi mumkin. Bundan tashqari, o‘zgaruvchining nomi Python tilidagi kalit so‘zlarning nomi bilan bir xil bo‘lmasligi kerak. Kalit so‘zlar ko‘p emas, ularni eslab qolish mumkin:

```
False  await  else  import  pass
None   break  except in    raise
True   class  finally is    return
and    continue for    lambda try
as     def    from    nonlocal while
assert del    global not    with
async  elif   if     or     yield
```

O‘zgaruvchilarni e‘lon qilish quyidagicha amalga oshiriladi:

```
foydalanuvchi = "Tomson"
```

Bu yerda “foydalanuvchi” nomli o‘zgaruvchiga "Tom" satri o‘zlashtirilmoqda.

Dasturlash tillarida o‘zgaruvchilarni nomlashning ikki xil usuli mavjud: **camel case** va **underscore notation**.

Camel case o‘zgaruvchi nomidagi har bir keying so‘z bosh harf bilan boshlanadi va oldingi so‘zga biriktirib yoziladi. Masalan:

```
userName = "Tomson"
```

Underscore notation – bu usulda o‘zgaruvchi nomidagi so‘zlar pastki chiziq bilan ajratiladi. Masalan:

```
user_name = "Tomson"
```

Shuningdek, siz katta-kichik registr sezgirligini hisobga olishingiz kerak, shuning uchun **ism** va **Ism** o‘zgaruvchilari turli ob’ektlarni ifodalaydi.

```
# ikki xil o‘zgaruvchi
```

```
ism = "Jenifer"
Ism = "Loktfud"
```

O'zgaruvchilarning o'ziga xos xususiyati shundagi dastur davomida uning qiymati va tipi o'zgartirishi mumkin.

Quyida o'zgaruvchini aniqlab, uni dastur davomida qiymatidan foydalanishni ko'rib chiqamiz:

```
name = "Tomson"    # o'zgaruvchi nomi "Tomson" ga teng
print(name)        # print: Tomson
name = "Bobbi"     # qiymatni "Bobbi" ga o'zgartiring
print(name)        # chiqish: Bobbi
```

Ma'lumotlar turlari

O'zgaruvchi ma'lumotlar turlaridan birining ma'lumotlarini saqlaydi. Pythonda juda ko'p turli xil ma'lumotlar turlari mavjud. Shulardan eng asosiy turlarni ko'rib chiqamiz: **bool**, **int**, **float** va **str**.

bool (Mantiqiy qiymatlar). bool turli o'zgaruvchilar ikkita mantiqiy qiymatni qabul qiladi: True (rost yoki 1) yoki False (yolg'on yoki 0). True qiymati biror narsaning haqiqat ekanligini ko'rsatish uchun ishlatiladi. False qiymati esa, aksincha biror fikrning yoki narsa noto'g'ri ekanligini ko'rsatadi. Ushbu turdagi o'zgaruvchilardan foydalanishga misol:

```
isMarried = False
print(isMarried)    # False
isAlive = True
print(isAlive)      # True
```

int (Butun sonlar). int turi 1, 4, 8, 50 kabi butun sonlar to'lamidagi sonlarni ifodalaydi. Ya'ni minus cheksizlikdan plus cheksizlikkacha bo'lgan barcha butun sonlar to'plamiga tushuvchi biror bir sonni qabul qiladi. dasturda foydalanishga misol:


```
age = 21
print("Yosh:", age)      # Yosh: 21
count = 15
print("Hisob:", count) # Hisob: 15
```

Odatiy bo'lib, standart raqamlar o'nlik sonlar sifatida qabul qilinadi. Ammo Python ikkilik, sakkizlik va o'n oltilik tizimlardagi raqamlarni ham qo'llab-quvvatlaydi.

Raqam **ikkilik sanoq sistemasida** ekanligini ko'rsatish uchun raqam oldiga **0b** prefiksi qo'yiladi:

```
a = 0b11
b = 0b1011
c = 0b100001
print(a)      # 3 o'nlik sanoq sistemasida
print(b)      # 11 o'nlik sanoq sistemasida
print(c)      # 33 o'nlik sanoq sistemasida
```

Raqam **sakkizlik sanoq sistemasini** ifodalashini ko'rsatish uchun raqamga **00** prefiksi qo'yiladi:

```
a = 007
b = 0011
c = 0017
print(a)      # 7 o'nlik sanoq sistemasida
print(b)      # 9 o'nlik sanoq sistemasida
print (c)     # 15 o'nlik sanoq sistemasida
```

Raqam **o'n oltilik sanoq sistemasini** ifodalashini ko'rsatish uchun raqamga **0x** prefiksi qo'yiladi:

```
a = 0x0A
b = 0xFF
```

```
c = 0xA1
print(a)      # 10 o'nlik sanoq sistemasida
print(b)      # 255 o'nlik sanoq sistemasida
print(c)      # 161 o'nlik sanoq sistemasida
```

Shuni ta'kidlash kerakki, qaysi tizimda biz konsolga chiqarish uchun bosib chiqarish funktsiyasiga raqam o'tkazsak, u sukut bo'yicha o'nli kasrda chiqariladi.

float (Kasr sonlar). float turi vergulli sonini ifodalaydi va haqiqiy sonlar to'pamiga kiruvchi biror-bir sonni qabul qiladi. masalan: 1,2 yoki 34,76 va hokazo. Butun va kasr qismlar orasidagi ajratuvchi sifatida nuqta ishlatiladi.

```
uzunligi = 1.68
p = 3.14
vazn = 68.
print (uzunligi)      # 1.68
print(p)              # 3.14
print(vazn)           # 68.0
```

kasr sonini eksponensial belgida aniqlash mumkin:

```
x = 3.9e3
print(x)      # 3900.0

x = 3.9e-3
print(x)      # 0.0039
```

Bu yerda $x = 3.9e3$ ifodadagi $e3 = 10^3$ ni anglatadi va $3.9 \cdot 10^3$ ko'payma x o'zgaruvchisiga yuklanmoqda. O'z navbatida $e-3 = 10^{-3}$

str (satrli tip). str turi satrlarni ifodalaydi. Satr "salom" va 'dunyo' kabi bitta yoki qo'sh tirnoq ichiga olingan belgilar ketma-ketligini ifodalaydi. Python 3.x da satrlar Unicode belgilar to'plamini ifodalaydi

```
hi = "Salom dunyo!"
```

```
print (hi )      # Salom dunyo!  
  
ism = "Tomson"  
  
print (ism)      # Tomson
```

O‘zgaruvchan tip. Python dasturlash tilida o‘zgaruvchilarning tipi dinamik tip hisoblanadi. Bu o‘zgaruvchining tiplari dastur davomida qiymatning tipiga bog‘liq tarzda o‘zgarishi mumkinligini anglatadi.

Shunday qilib, ikki yoki bitta tirnoq ichida satr tayinlanganda, o‘zgaruvchi str turida bo‘ladi. Butun sonni o‘zlashtirilganda Python avtomatik ravishda o‘zgaruvchining turini int sifatida aniqlaydi. O‘zgaruvchini float ob’ekti sifatida aniqlash uchun unga kasr son beriladi, bu yerda kasr va butun qismlarni ajratuvchi sifatida nuqtadan foydalaniladi. Quyida bunga misol keltirilgan:

```
f_Id = "abc"      # satr turiga mansub  
print (f_Id)  
  
f_Id = 234        # butun sonlar turiga mansub  
print (f_Id)
```

Standart **type()** funksiyasidan foydalanib, o‘zgaruvchining joriy tipini bilib olishingiz mumkin:

```
f_Id = "abc"      # tip str  
print(type(f_Id)) # <class 'str'>  
  
f_Id = 234        # tip int  
print(type(f_Id)) # <class 'int'>
```

Nazorat savollari:

1. Pythonda o‘zgaruchi qanday e’lon qilinadi?
2. Pythonda o‘zgaruvchilarga nom berishda nimalarga e’tibor berish kerak bo‘ladi?
3. O‘zgaruvchilarga nom berishning CamelCase usuli qanday?
4. O‘zgaruvchilarga nom berishning “Underscore notation” usulini tushuntirib bering.

5. Pythonda o'zgaruvchilarning qaday turlari ishlatiladi?
6. Satr tipli o'zgaruvchiga bir nechta satrni yuklash uchun qanday usuldan foydalaniladi?
7. Satr hosil qilinayotganda maxsus simvollar text ichida mavjud bo'lsa nima sodir bo'ladi? Misollar bilan tushuntirib bering.
8. Satr tarkibiga o'zgaruvchilarning qiymatini chaqirish uchun qanday amal bajarish kerak?

2.3.§. Pythonda kiritish va chiqish funksiyalari

Konsolga xabar chiqarish funksiyasi - print()

`print()` pythonning standart funksiyasi ma'lumotni konsolga chiqarish uchun ishlatiladi. Ushbu funktsiya chaqirilganda, chiqish qiymati funksiyaga parametr sifatida beriladi:

```
print("Salom, QALAYSIZ?")
```

Ushbu kod konsolda "Salom, QALAYSIZ?" qatorini chop etadi.

Ushbu funktsiyaning ajralib turadigan xususiyati shundaki, u standart holatda qiymatni alohida satrda chiqaradi. Masalan:

```
print ("Salom dunyo")  
print("Salom, QALAYSIZ?")  
print ("Salom Python")
```

Har bir chaqirilgan `print()` funksiyasi xabarni alohida satrga joylashtiriladi:

```
Salom Dunyo  
Salom qalaysiz?  
Salom Python
```

Bu har doim ham qulay bo'lavermaydi. Masalan, biz barcha qiymatlarni bitta satrda ko'rsatishni xohlaymiz. Buning uchun **end** parametri yordamida har bir satrning oxirida qanday belgi bo'lishini aniqlash mumkin. End parametridan quyidagicha foydalaniladi:

```
print (qiymat, end=oxirgi_belgilar)
```

Odatda, end parametrining qiymati, keyingi qatorga o'tishni ko'rsatadigan "\n" belgisiga teng bo'ladi. Shuning uchun print() funksiyasi standart bo'yicha xabarni alohida satrda chop etadi.

Endi shart bo'yicha barcha xabarlar yonma-yon chiqishini ko'rib chiqaylik. Buning uchun end parametriga bitta probel - " " ni yuklab qo'yish kerak:

```
print ("Salom dunyo", end=" ")
print("Salom, QALAYSIZ?", end=" ")
print ("Salom Python")
```

Bu holda har bir qiymatlar bo'sh joy bilan ajratiladi:

```
Salom Dunyo Salom, QALAYSIZ? Salom Python
```

End parametri nafaqat bitta belgi, balki belgilar to'plamini qabul qilishi mumkin:

```
print("Salom dunyo", end=" va ")
print("Salom, QALAYSIZ?", end=" va ")
print("Salom Python")
```

Bunday holda, chiqish xabarlar " va " belgilari bilan ajratiladi:

```
Salom Dunyo va Salom, QALAYSIZ? va Salom Python
```

Print funksiyasining yana bir **sep** nomli ajoyib parametri mavjud. **Sep** separator so'zidan olingan bo'lib, vazifasi print funksiyasiga berilgan bir nechta qiymatlar orasidagi ajratuvchi belgini o'rnatib beradi:

```
A = "Salom do'stlar"
B = "Bu menman"
print(A, B, sep=" - ")
```

Bunday holda, chiqish xabarlar " - " belgilari bilan ajratiladi:

```
Salom do'stlar - Bu menman
```

Kiritish operatori - input()

Konsolga chiqish bilan bir qatorda biz konsoldan foydalanuvchi ma'lumotlarini olishimiz, kirish ma'lumotlarini olishimiz ham kerak. Buning uchun Pythonda input() funksiyasi mavjud. Ushbu funktsiyaga kirish so'rovi uzatiladi. Va biz kiritilgan natijani o'zgaruvchida saqlashimiz mumkin bo'ladi. Masalan, foydalanuvchi nomini kiritish uchun kodni aniqlaylik:

```
ism = input("Ismingizni kiriting:")  
print(f" Sizning Ismingiz : {ism}")
```

Bunday holda, input() funktsiyasiga "Ismingizni kiriting:" qatori ko'rinishidagi kiritish so'rovi yuboriladi. Funktsiya natijasi - foydalanuvchi kiritgan natija `ism` o'zgaruvchisiga uzatiladi. Keyinchalik print() funksiyasidan foydalanib, ushbu o'zgaruvchining qiymatini konsolga chop etishimiz mumkin. Kod natijasi:

```
Ismingizni kiriting: Sanjarbek  
  
Sizning Ismingiz: Sanjarbek
```

Bir nechta qiymatlar kiritishga misol:

```
ism = input("Ismingiz:")  
yosh = input("Yoshingiz:")  
print(f"Sizning ismingiz: {ism}, yoshingiz esa:  
{yosh} da")
```

Dasturning ishlashiga misol:

```
Ismingiz: Anjilika  
Yoshingiz: 20  
Sizning ismingiz : Anjilika, yoshingiz esa: 20 da
```

Shuni yodda tutish kerakki, kiritilgan barcha qiymatlar str tipida qabul qilinadi, ya'ni satrlar sifatida qabul qilinadi. Va agar biz yuqoridagi koddagi ikkinchi holatda bo'lgani kabi raqamni kiritsak ham, Python kiritilgan qiymatni raqam sifatida emas, balki satr sifatida ko'rib chiqadi.

Nazorat savollari:

1. Ma'lumotni konsoldan kiritish qanday amalga oshiriladi?
2. Ma'lumotni konsol ekraniga chiqarish qanday amalga oshiriladi?
3. `print()` funksiyasining ishlash prinsipi va qabul qiladigan parametrlar haqida gapirib bering.
4. `input()` funksiyasining qabul qiladigan parametrlari haqida gapirib bering.
5. `print()` funksiyasi orqali stringlarning ichida o'zgaruvchining qiymatini chiqarish uchun qanday amallar bajarilishi kerak?

2.4.§. Arifmetik operatorlar

Python barcha turdagi arifmetik operatsiyalarni qo'llab-quvvatlaydi :

Qo'shish (+):	<code>print(6 + 2)</code>	# 8
Ayirish (-):	<code>print(6 - 2)</code>	# 4
Ko'paytirish (*):	<code>print(6 * 2)</code>	# 12
Bo'linish (/):	<code>print(6/2)</code>	# 3.0
Bo'lib butunni olish (//):	<code>print(7 // 2)</code>	# 3
Darajaga oshirish (**):	<code>print(6 ** 2)</code>	# 6 ² natija 36
Bo'lib qoldig'ini olish (%):	<code>print(7 % 2)</code>	# 1

Agar bitta arifmetik ifodada bir nechta arifmetik amallar ketma-ket foydalanilsa, amallarning prioritetiga muvofiq ketma-ketlikda bajariladi. Birinchi navbatda yuqori ustuvorlikdagi operatsiyalar amalga oshiriladi. Arifmetik amallarning prioritetlarining kamayish tartibida quyidagi 2-jadvalda ko'rsatilgan.

Operatsiya yo'nalishi:

2-jadval

1	<code>**</code>	o'ngdan chapga
2	<code>*</code> , <code>/</code> , <code>//</code> , <code>%</code>	Chapdan o'ngga
3	<code>+</code> , <code>-</code>	Chapdan o'ngga

Keling, quyidagi ifodani ko‘rib chiqamiz:

```
raqam = 3 + 4 * 5 ** 2 + 7
print(raqam)    # 110
```

Bu yerda birinchi navbatda darajaga oshirish amali ($5 ** 2$) bajariladi, keyin natija 4 ga ($25 * 4$) ko‘paytiriladi, keyin qo‘shish ($3 + 100$) amali bajariladi va keyin yana qo‘shish ($103 + 7$) amalga oshiriladi. Shunday qilib natija 110 ga teng bo‘ladi.

Qavslar amallar tartibini qayta aniqlash uchun ishlatilishi mumkin:

```
raqam = (3 + 4) * (5 ** 2 + 7)
print(raqam)    # 224
```

Shuni ta’kidlash kerakki, arifmetik amallarda ham butun, ham kasr sonlar ishtirok etishi mumkin. Agar butun son (int) va vergulli (kasr) son (float) bir xil amalda ishtirok etsa, u holda butun son float turiga o‘tkaziladi.

Arifmetik amallarning tenglik belgisi bilan ishlatilishi

Bu maxsus operatsiyalar amal bajarilgandan keyin natijani birinchi operandga yuklash imkonini beradi:

3-jadval

+=	Qo‘shishdan hosil bo‘lgan natijani o‘zlashtirish
-=	Ayirishdan hosil bo‘lgan natijani o‘zlashtirish
*=	Ko‘paytirishdan hosil bo‘lgan natijani o‘zlashtirish
/=	Bo‘linishdan hosil bo‘lgan natijani o‘zlashtirish
//=	Bo‘lib butunni olishdan hosil bo‘lgan natijani o‘zlashtirish
**=	Raqam darajaga oshirishdan hosil bo‘lgan natijani o‘zlashtirish
%=	Bo‘lib qoldig‘ini oshirishdan hosil bo‘lgan natijani o‘zlashtirish

Operatsiyalarni dasturda qo‘llanilishi:

```
raqam = 10

raqam += 5
```



```

print(raqam)      # 10 + 5 = 15

raqam -= 3
print(raqam)      # 15 - 3 = 12

raqam *= 4
print(raqam)      # 12 * 4 = 48

raqam /= 2
print(raqam)      # 48 / 2 = 24.0

raqam //= 5
print(raqam)      # 24 // 5 = 4

raqam **= 3
print(raqam)      # 4 ** 3 = 64

raqam %= 6
print(raqam)      # 64 % 6 = 4

```

Yaxlitlash va “round” funksiyasi

Dasturlashda float tipli raqamlar bilan operatsiyalar bajarayotganda, natijalar aniq biz o‘ylaganimiz kabi bo‘lmasligi mumkin. Masalan :

```

bir_son = 2.0001
ikki_son = 5
uch_son = bir_son / ikki_son
print(uch_son) # 0.400020000000000004

```

Bunday holda, dasturchi 0,40002 kabi natijani olishni kutdi, lekin oxirida bir qator nollar orqali, qandaydir to‘rt paydo bo‘lmoqda. Yoki boshqa ifoda:

```

print(2.0001 + 0.2)      # 2.10020000000000003

```

Yuqoridagi holatda, natijani yaxlitlash uchun o'rnatilgan round() funksiyasidan foydalanishimiz mumkin:

```
bir_son = 2.0001
ikki_son = 0.1
uch_son = bir_son + ikki_son
print(round(uch_son)) # 2
```

Yaxlitlanadigan raqam round() funksiyasiga o'tkaziladi. Agar yuqoridagi misoldagi kabi funksiyaga bitta raqam uzatilsa, u butun songa yaxlitlanadi.

round() funksiyasi ikkinchi raqamni ham olishi mumkin, natijada olingan sonda nechta o'nli kasr bo'lishi kerakligini ko'rsatadi:

```
bir_son = 2.0001
ikki_son = 0.1
uch_son = bir_son + ikki_son
print(round(uch_son, 4)) # 2.1001
```

Bu holda uch_son verguldan keying 4 ta raqamgacha yaxlitlanadi.

Agar funksiyaga faqat bitta qiymat o'tkazilsa - faqat yaxlitlanadigan raqam, u eng yaqin butun songa yaxlitlanadi.

Yaxlitlash misollari:

```
# butun songacha yaxlitlash
print(round(2.49))
print(round(2.51))
```

Biroq, agar yaxlitlanadigan qism ikkita butun sondan bir xil masofada bo'lsa, yaxlitlash eng yaqin juft songa o'tadi:

```
print(round(2.5)) # 2
print(round(3.5)) # 4
```

round() funksiyasining ikkinchi parametrini beradigan bo'lsak, verguldan keyin shuncha xonagacha yaxlitlaydi. Yaxlitlagandayam tushirib qoldiriladigan raqam 5 dan katta yoki teng bo'lsa, unda oldingisiga birni qo'shib yaxlitlaydi:

```
# verguldan keyingi 2 ta raqamgacha yaxlitlash
print(round (2.554, 2))          # 2.55
print(round (2.5551, 2))        # 2.56
print(round (2.554999, 2))      # 2.55
print(round (2.499, 2))         # 2.5
```

Biroq, round() funksiyasi ideal vosita emasligini yodda tuting. Masalan, yuqorida, butun sonlarga yaxlitlashda, agar yaxlitlanadigan qism ikki qiymatdan bir xil masofada bo'lsa, yaxlitlash eng yaqin juft qiymatgacha amalga oshiriladi, degan qoida qo'llaniladi. Pythonda, sonning kasr qismini suzuvchi raqam sifatida to'g'ri ko'rsatib bo'lmasligi sababli, bu butunlay kutilmagan natijalarga olib kelishi mumkin. Masalan:

```
# verguldan keyingi 2 ta raqamgacha yaxlitlash

print(dumaloq (2.545, 2))      # 2.54
print(dumaloq(2.555, 2))      # 2.56
print(dumaloq (2.565, 2))      # 2.56
print(dumaloq (2.575, 2))      # 2.58
print( dumaloq(2.655, 2))      # 2.65
print(dumaloq (2.665, 2))      # 2.67
print(dumaloq (2.675, 2))      # 2.67
```

Nazorat savollari:

1. pythonda qanday arifmetik amallar mavjud?
2. % - qanday amalni bajarish uchun ishlatiladi?
3. // - qanday amalni bajarish uchun ishlatiladi?
4. ** - qanday amalni bajarish uchun ishlatiladi?
5. Kvadrat ildiz qanday ishlatiladi?
6. Oddiy bo'lish amalini bajarilganda qanday tipli qiymat qaytaradi?

7. += , -=, *=, /=, **= va %= - operatorlarining vazifasi qanday?
8. Round() funksiyasining ishlash prinsipini tushuntirib bering.

2.5.8. Mantiqiy ifodalar

Python dasturlash tilida ham bir qancha mantiqiy ifodalar mavjud. Ushbu mantiqiy ifodalarning barchasi ikkita o'zgaruvchi bilan ishlatiladi va umumiy natija sifatida bool turidagi mantiqiy qiymat qaytaradi. Mantiqiy qiymatlar ikkita bo'lib, quyidagilar: **True** (rost) va **False** (yolg'on).

Solishtirish operatorlari. Eng oddiy shartli ifodalar ikki qiymatni taqqoslaydigan taqqoslash amallaridir. Python quyidagi taqqoslash operatsiyalarini qo'llab-quvvatlaydi:

4-jadval

==	Ikkala operand teng bo'lsa, True qiymatini qaytaradi. Aks holda False qaytaradi.
!=	Ikkala operand teng bo'lmasa, True qiymatini qaytaradi. Aks holda False qaytaradi.
> (katta)	Agar birinchi operand ikkinchisidan katta bo'lsa, True qiymatini qaytaradi.
< (kichik)	Agar birinchi operand ikkinchidan kichik bo'lsa, True qiymatini qaytaradi.
>= (katta yoki teng)	Birinchi operand ikkinchidan katta yoki teng bo'lsa, True qiymatini qaytaradi.
<= (kichik yoki teng)	Birinchi operand ikkinchidan kichik yoki teng bo'lsa, True qiymatini qaytaradi.

Taqqoslash operatsiyalariga misollar:

```
a = 6
b = 9
res = 6 == 9 # natijani o'zgaruvchiga yuklash
```

```
print(res)    # False
print(a > b)   # False
print(a != b)  # True
print(a < b)   # True

boolean1 = True
boolean2 = False
print(boolean1 == boolean2) # False
```

Taqqoslash operatsiyalari turli ob'ektlarni - satrlarni, raqamlarni, mantiqiy amallarni solishtirishi mumkin, ammo operatsiyaning ikkala operandlari ham bir xil turni ifodalashi kerak.

Mantiqiy amallar. Turli murakkablikka ega bo'lgan mantiqiy ifodalarni yaratish uchun mantiqiy operatorlar ishlatiladi. Python quyidagi mantiqiy operatorlarni qo'llab quvvatlaydi:

and (mantiqiy ko'paytirish) operatori ikkita operandga qo'llaniladi:

```
x and y
```

Birinchidan, and operatori x ifodasini baholaydi va agar u False bo'lsa, uning qiymati qaytariladi. Agar u True bo'lsa, ikkinchi operand y baholanadi va y ning qiymati qaytariladi.

```
yosh = 20
vazn= 65
res = yosh > 19 and vazn == 65
print(res)    # True
```

Bu holda **and** operatori ikkita ifoda natijalarini taqqoslaydi: yosh > 19 hamda vazn == 65. Agar bu ifodalarning ikkalasi ham True qiymatini qaytarsa, unda **and** operatori ham res o'zgaruvchisiga True qiymatini qaytaradi (rasmiy ravishda oxirgi operandning qiymati qaytariladi). .

Lekin **and** operatorining operandlari True va False bo'lishi shart emas. Bu har qanday qiymat bo'lishi mumkin. Masalan:

```
natija = 14 va "s"
print(natija)    # s, chunki 14 soni True ga ekvivalent,
#shuning uchun oxirgi operandning qiymati qaytariladi

natija = 0 va "s"
print(natija)    # 0 , chunki 0 False ga ekvivalent
```

Bunday holda, 0 raqami va bo'sh "" qatori noto'g'ri deb hisoblanadi, qolgan barcha raqamlar va bo'sh bo'lmagan satrlar "True" ga ekvivalentdir.

or (mantiqiy qo'shish) ikkita operandga ham tegishli:

```
x or y
```

Or operatori avval x ifodasini baholaydi va agar u True bo'lsa, uning qiymati qaytariladi. Agar u False bo'lsa, ikkinchi operand y baholanadi va y ning qiymati qaytariladi. Masalan

```
yosh = 22
natija = yosh > 21 or False
print(natija) # True
```

Shuningdek, **OR** operatori har qanday qiymatlarga qo'llanilishi mumkin. Masalan:

```
res = 4 or "f"
print(res)    # 4, chunki 4 rost ga teng, shuning uchun birinchi
#operandning qiymati qaytariladi
natija = 0 or "f"
print(natija)    # f, chunki 0 False ga ekvivalent, shuning
uchun #oxirgi operandning qiymati qaytariladi
```

not (mantiqiy inkor - !)

Ifoda False bo'lsa, True qiymatini qaytaradi

```
yosh = 25
isSingle = False
print(not yosh > 21)    # False
print(not isSingle)    # True
print(not 45)          # False
print(not 0)           # True
```

in operatori

Agar ba'zi qiymatlar to'plamida ma'lum qiymat mavjud bo'lsa, **in** operatori "True" ni qaytaradi. U quyidagi shaklga ega:

```
qiymat in [qiymat_to'plamidagi]
```

Masalan, satr belgilar to'plamini ifodalaydi. Va in operatori bilan biz unda biron bir qiymat mos kelishini va mavjudligini tekshirishimiz mumkin:

```
xabar = "hello world!"
hi = "hello"
print(hi in xabar)
zol= "golden"
print(zol in xabar)
```

Agar qiymatlar to'plamida qiymat mavjud masligini tekshirishimiz kerak bo'lsa, unda biz **not in** modifikatsiyasidan foydalanishimiz kerak bo'ladi. Agar qiymat [qiymatlar to'plami] da mavjud bo'lmasa, u True qiymatini qaytaradi:

```
xabar = "hello world!"
hi = "hello"
print(hi not in xabar)    # False

golden = " golden "
print(golden not in xabar) # True
```

Nazorat savollari:

1. Pythonda shart operatorining sintaksisi qanday?
2. Pythonda qanday taqqoslash operatorlar mavjud?
3. Pythonda qanday mantiqiy operatorlar mavjud?
4. Mantiqiy ko'paytirish va mantiqiy qo'shish operatorlarini solishtirib bering?
5. Shart operatorining ichida **in** operatorining ishlash prinsipi qanday?
6. **not in** operatiri qanday ishlaydi?
7. **in** va **not in** operatorlarining farqi nimada?

2.6.§. Shart operatori – IF

Shartli operatorlari shartli ifodalardan foydalanadi va ularning qiymatiga qarab, dasturning bajarilishini yo'llardan biri bo'ylab yo'naltiradi. Shunday operatorlardan biri **if** operatoridir. U quyidagi quyidagicha sintaksisga ega:

```
if mantiqiy_amal:
    [if bloki]
elif mantiqiy_amal:
    [elif bloki]
else:
    [else bloki]
```

Eng oddiy shaklda if kalit so'zidan keyin mantiqiy ifoda keladi. Va agar bu mantiqiy ifoda "True" ni qaytarsa, u holda ko'rsatmalarning keyingi bloki bajariladi, ularning har biri blokga tegishliligini oldiga 4 ta probel yoki 1 ta tab tashlanishi bilan aniqlanadi (4 bo'sh joy yoki bo'shliqlar sonini cheklash maqsadga muvofiqdir) bu 4 ga karrali):

```
til = "uzbekcha"
if til == "uzbekcha":
    print("Assalomu alaykum")
print("Xayr")
```


Bu holda `til` o'zgaruvchisining qiymati `"uzbekcha"` bo'lganligi sababli, `if` bloki bajariladi, unda faqat bitta operator mavjud - `print("Assalomu alaykum")`. Natijada, konsol quyidagi qatorlarni ko'rsatadi:

```
Assalomu alaykum
Xayr
```

Kodning oxirgi qatoriga e'tibor bering, unda `"Xayr"` xabari ko'rsatiladi. Chunki `print("Xayr")` oldidan tab tashlanmagan, shuning uchun u `if` blokiga tegishli emas va `if` konstruktsiyasidagi ifoda `False` qiymatini qaytarsa ham bari bir `print("Xayr")` amali bajariladi.

Ammo agar biz uni cheklasak, u `if` konstruktsiyasiga ham tegishli bo'ladi:

```
til = "uzbekcha"
if til == "uzbekcha":
    print("Assalomu alaykum")
    print("Xayr")
block else:
```

Mabodo `if` ifodasi `False` qiymatini qaytarsa, unda `else` bloki ishga tushadi:

```
til = "uzbekcha"
if til == "uzbekcha":
    print("Assalomu alaykum")
else:
    print("Hi")
print("Xayr")
```

Agar `til == "uzbekcha"` iborasi `True` qiymatini qaytarsa, u holda `if` bloki bajariladi, aks holda `else` bloki bajariladi. Va bu holda shart `til == "uzbekcha"` `False` qiymatini qaytarganligi sababli, `else` blokidagi operator bajariladi.

Bundan tashqari, **else** bloki tarkibiga kiruvchi operatorlarning oldidan ham satr boshidan tab (4ta probelga teng bo'sh joy) tashlanishi kerak. Misol uchun, yuqoridagi misolda `print("Xayr")` ning oldidan tab tashlanmagan, shuning uchun u **else** bloki tarkibiga kiritilmagan va `til` o'zgaruvchisining qiymati qaysi tilda bo'lishidan qat'iy nazar bajariladi. Ya'ni, konsol quyidagi qatorlarni ko'rsatadi:

```
Assalomu alaykum
Xayr
```

else bloki tarkibi bir nechta qatorlardan tashkil topgan bo'lishi ham mumkin:

```
til = "uzbekcha"
if til == "uzbekcha":
    print("Salom")
    print("Dunyo!")
else:
    print("Hello")
    print("World!")
```

Agar siz bir nechta aniq shartlarni kiritishingiz kerak bo'lsa, unda siz qo'shimcha **elif** bloklaridan foydalanishingiz kerak bo'ladi:

```
til = "uzbekcha"
if til == "uzbekcha":
    print("Assalomu")
    print("Alaykum")
elif til == "german":
    print("Hallo")
    print("Welton")
else:
    print("What's")
    print("up")
```

Python bu yerda birinchi, if ning shartini tekshiradi. Agar rost bo'lsa, if blokidagi operatorlar bajariladi. Agar bu shart False qiymatini qaytarsa, Python elif shartini tekshiradi.

Agar elifning sharli ifodasi True bo'lsa, u holda elif blokidagi opertorlar bajariladi. Ammo agar u ham False qaytarsa, else blokidagi bajariladi.

Shartlarni elif operatoridan foydalangan holda ixtiyoriy davom ettirishingiz mumkin. Masalan :

```
til = "Uz"
if til == "english":
    print("Hello")
elif til == "german":
    print("Hallo")
elif til == "french":
    print("Salut")
else:
    print("Salom")
```

Ichma-ich joylashgan if operatori. if operatorining sharti bajarilganidan keyin yanayam aniqlashtirish uchun ichida yana bir necha shartga tekshiriladi:

```
til = "uzbekcha"
payt == "ertalabki vaqt"
if til == "uzbekcha":
    print("Uzbek")
    if payt == "ertalabki vaqt":
        print("Unda hayrli tong!")
    elif payt == "kechgi vaqt":
        print("Unda sizga hayrli kech!")
```

Bu yerda if konstruktsiyasi ichki oʻrnatilgan if/elif konstruktsiyalarini oʻz ichiga oladi. Yaʼni, agar til oʻzgaruvchisi "uzbekcha" ga teng boʻlsa, u holda ichki oʻrnatilgan if/elif konstruktsiyalarisi kunduzgi oʻzgaruvchining qiymatini qoʻshimcha ravishda tekshirishadi - `payt` oʻzgaruvchisining qiymati `"ertalabki vaqt"` ga tengmi yoki `"kechgi vaqt"` ga tengligini tekshiradi va shunga qarab javob qaytariladi. Va bu holda, biz quyidagi konsol chiqishini olamiz:

```
Uzbek
```

```
Unda hayrli tong
```

Esda tutingki, ichki oʻrnatilgan if ifodalarining oldidan `tab` qoldirish kerak, ichki oʻrnatilgan konstruktsiyalardagi operatorlarning oldidan ham `tab` (boʻsh joy) qoldirilishi kerak. Toʻgʻri joylashtirilmagan tabulyatsiya dastur mantigʻini oʻzgartirishi mumkin.

Xuddi shunday, ichki oʻrnatilgan if/elif/else konstruktsiyalari elif va else bloklariga joylashtirilishi mumkin:

```
til = "uzbek"
payt = "ertalab"

if til == "uzbek":
    if payt == "ertalab":
        print("Hayrli tong")
    else:
        print("Hayrli kech")
else:
    if payt == "ertalab":
        print("Good morning")
    else:
        print("Good evening")
```

Nazorat savollari :

1. If operatori qanday vazifasi bajaradi?
2. If operatorining strukturasi qanday?

3. Elif operatorining vazifasi qanday?
4. Ichma – ich foydalaniladigan shart operatoriga misollar keltiring.
5. Bloklarning if ifodaga tegishliligini qanday bilish mumkin?

2.7.§. Pythonda takrorlanuvchi jarayonlarni dasturlash.

For, while, break, continue va else operatorlari bilan ishlash.

Sikllar biror bir shartning bajarilishiga qarab ba'zi harakatlarni bajarishga imkon beradi. Pythonda quyidagi turdagi sikllar mavjud: **while** va **for**.

WHILE sikli. while sikli qandaydir bir shartning to'g'riligini tekshiradi va agar shart True bo'lsa, u holda sikl tanasidagi blokni bajaradi. U quyidagicha sintaksisga ega:

```
while <shartli_ifoda>:  
    [sikl tanasi]
```

while kalit so'zidan keyin shartli ifoda keladi va bu ifoda True qiymat qaytarsa, sikl tanasiga kirib u yerdagi operatorlar ketma-ket bajariladi.

while sikliga tegishli barcha bloklar oldidan bo'sh joy qoldirilib ketishi va boshlanish qismlari bir ustunga joylashgan bo'lishi kerak va while kalit so'zidan bir xilda chekingan bo'lishi kerak.

```
son = 1  
while son < 5:  
    print(f"son = {son}")  
    son += 1  
print("Dastur tugatildi")
```

Bunday holda, son o'zgaruvchisi 5 dan kichik bo'lsa, while sikli ishlaydi. Aks holda sikldan keyingi amal bajariladi.

Bu sikl tanasi ikkita ifodadan iborat:

```
print(f"son = {son}")  
son += 1
```

Shuni ham yodda tutingki, oxirgi `print("Dastur tugallandi")` satr boshidan chekinmaydi, shuning uchun u `while` siklining bir qismi emas.

Siklning butun jarayonini quyidagicha ifodalash mumkin: Birinchi, **son** o'zgaruvchisining qiymati 5 dan kichik yoki kichik emasligini tekshiriladi. Va o'zgaruvchi dastlab 1 ga teng bo'lganligi sababli, bu shart `True` ni qaytaradi va shuning uchun sikl operatorlari bajariladi.

Sikl ko'rsatmalari satr **son**=1 ni konsolga chiqaradi. Va keyin raqam o'zgaruvchisining qiymati bittaga oshiriladi - endi u 2 ga teng bo'ladi. sikl ko'rsatmalari blokini bir marta bajarish iteratsiya deb ataladi. Ya'ni, shu tarzda, birinchi takrorlash siklda amalga oshiriladi.

son < 5 sharti yana tekshiriladi. Yana `True` qaytariladi, chunki son 2 ga teng, shuning uchun sikl operatorlari yana bir marta bajariladi.

Sikl ko'rsatmalari konsolga satr **son** = 2 ni chiqaradi. Va keyin **son** o'zgaruvchisining qiymati yana bittaga oshiriladi - endi u 3 ga teng bo'ldi. Shunday qilib, ikkinchi takrorlash amalga oshiriladi.

son < 5 shart yana tekshiriladi. Yana `True` qiymat qaytariladi, chunki son = 3, shuning uchun sikl operatorlari bajariladi.

Sikl ko'rsatmalari konsolga satr **son** = 3 ni chiqaradi. Va keyin **son** o'zgaruvchisining qiymati yana bittaga oshiriladi - endi u 4 ga teng. Ya'ni uchinchi iteratsiya amalga oshiriladi.

son < 5 sharti yana tekshiriladi. Yana `True` qiymat qaytariladi, chunki son = 4, shuning uchun sikl ko'rsatmalari bajariladi.

Sikl ko'rsatmalari konsolga son= 4 qator raqamini chiqaradi. Va keyin son o'zgaruvchisining qiymati yana bittaga oshiriladi - endi u 5 ga teng. Ya'ni to'rtinchi takrorlash amalga oshiriladi.

Va shart $son < 5$ yana tekshiriladi. Lekin endi u False qiymat qaytaradi, chunki son o'zgaruvchisining qiymati endi 5 ga teng, shuning uchun sikl tugatiladi. Shundan so'ng sikldan keyin operatorlar bajariladi. Shunday qilib, bu sikl to'rtta o'tish yoki to'rtta takrorlashni amalga oshiradi

Natijada, kodni bajarishda biz quyidagi konsol chiqishini olamiz:

```
son = 1
son = 2
son = 3
son = 4

Dastur tugatildi
```

While sikli bilan yana bir operator – else ham ishlatiladi. U while siklidagi shart False qiymat qaytarganida ishga tushadi va tarkibidagi blok bajariladi:

```
raqam = 1
while raqam <5:
    print(f"raqam = {raqam}")
    raqam += 1
else:
    print(f"raqam = {raqam}. sikl tugallandi")
print("Dastur tugatildi")
```

Ya'ni, bu holda birinchi navbatda shart tekshiriladi va while operatorlari bajariladi. Keyin shart False ga aylanganda else blokidagi gaplar bajariladi. E'tibor bering, else blokidagi iboralar ham sikl konstruktsiyasining boshidan chekinadi. Natijada, bu holda biz quyidagi konsol chiqishini olamiz:

```
raqam = 1
```

```
raqam = 2
raqam = 3
raqam = 4
raqam = 5. sikl tugallandi

Dastur tugadi
```

For sikli. Sikl ning yana bir turi for konstruksiyasidir. Ushbu sikl qiymatlar to‘plamini takrorlaydi, har bir qiymatni o‘zgaruvchiga qo‘yadi va keyin siklda biz ushbu o‘zgaruvchi bilan turli harakatlarni bajarishimiz mumkin. For siklining strukturasi quyidagicha:

```
for [o‘zgaruvchi] in [qiymatlar_to‘plami]:
    [sikl tanasi]
```

For kalit so‘zidan keyin qiymatlar joylashtiriladigan o‘zgaruvchining nomi keladi. Keyin in operatoridan keyin qiymatlar to‘plami va ikki nuqta qo‘yiladi.

Va keyingi qatordan sikl tanasidagi operatorlar boshlanadi. Bu operatorlar siklning bir qismi ekanligi uchun ular for kalit so‘zi boshidan ichkariga kiritilishi kerak.

Siklni bajarishda Python ketma-ket barcha qiymatlarni to‘plamdan oladi va ularni o‘zgaruvchiga uzatadi. To‘plamdagi barcha qiymatlar takrorlangandan keyin sikl tugaydi.

Masalan, qiymatlar to‘plami sifatida siz aslida belgilar to‘plamini ifodalovchi satrni ko‘rib chiqishingiz mumkin. Keling, bir misolni ko‘rib chiqaylik :

```
string = "Hello"
for char in string:
    print(char)
```

char o‘zgaruvchisi siklda aniqlanadi, **in** operatoridan keyin **string** o‘zgaruvchisi "Hello" satrini saqlaydigan takrorlangan to‘plam sifatida

ko'rsatiladi. Natijada, for sikli `string` tarkibidagi barcha belgilarni ketma-ketlikda takrorlaydi va ularni **char** o'zgaruvchisiga o'zlashtiradi. Sikl blokining o'zi **char** o'zgaruvchining qiymatini konsolga chop etadigan bitta funksiyadan iborat. Dasturning konsol chiqishi:

```
H
e
l
l
o
```

For siklida sikl tugagandan keyin bajariladigan qo'shimcha `else` bloki ham bo'lishi mumkin:

```
string = "Hello"
for char in string:
    print(char)
else:
    print(f"oxirgi simvol: {char}")
print("Dastur yakunlandi")
```

Bunday holda, biz quyidagi konsol chiqishini olamiz:

```
H
e
l
l
o
oxirgi simvol: o
Dastur yakunlandi
```

Shuni ta'kidlash kerakki, `else` bloki for siklida aniqlangan barcha o'zgaruvchilarga kirish huquqiga ega.

Ichma-ich sikllar. Ba'zi sikllar o'zlarida boshqa sikllarni o'z ichiga olishi mumkin. Ko'paytirish jadvalining chiqishi misolini ko'rib chiqing:

```
a = 1
b = 1
while a < 10:
    while b < 10:
        print(a * b, end="\t")
        b += 1
    print("\n")
    b = 1
    a += 1
```

Tashqi sikl `while a < 10:` `a` o'zgaruvchisi 10 ga teng bo'lgunga qadar 9 marta ishlaydi. Bu sikl ichida ichki sikl `while b < 10:` ishlaydi. `b` o'zgaruvchisi 10 ga teng bo'lgunga qadar ichki sikl ham 9 marta ishlaydi. Bundan tashqari, ichki siklning barcha 9 ta takrorlanishi tashqi siklning bir iteratsiyasi ichida ishga tushiriladi.

Ichki halqaning har bir iteratsiyasida konsolda `a` va `b` raqamlarining mahsuloti ko'rsatiladi. Keyin `b` o'zgaruvchining qiymati bittaga oshiriladi. Ichki sikl o'z ishini tugatgandan so'ng, `b` o'zgaruvchining qiymati 1 ga o'rnatiladi va `a` o'zgaruvchining qiymati bittaga oshiriladi va tashqi siklning keyingi takrorlanishiga o'tish sodir bo'ladi. Va `a` o'zgaruvchisi 10 ga teng bo'lgunga qadar hamma narsa takrorlanadi. Shunga ko'ra, ichki sikl tashqi siklning barcha iteratsiyasi uchun faqat 81 marta ishlaydi. Natijada, biz quyidagi konsol chiqishini olamiz:

1	2	3	4	5	6	7	8	9
2	4	6	8	10	12	14	16	18
3	6	9	12	15	18	21	24	27

4	8	12	16	20	24	28	32	36
5	10	15	20	25	30	35	40	45
6	12	18	24	30	36	42	48	54
7	14	21	28	35	42	49	56	63
8	16	24	32	40	48	56	64	72
9	18	27	36	45	54	63	72	81

for siklini xuddi shunday aniqlanishi mumkin:

```
for c1 in "ab":
    for c2 in "ba":
        print(f"{c1}{c2}")
```

Bunday holda, tashqi sikl "ab" qatoridan o'tadi va har bir belgini c1 o'zgaruvchisiga qo'yadi. Ichki sikl "ba" satrini takrorlaydi, satrdagi har bir belgini c2 o'zgaruvchisiga qo'yadi va ikkala belgi kombinatsiyasini konsolga chiqaradi. Ya'ni, oxirida biz a va b belgilarning barcha mumkin bo'lgan kombinatsiyalarini olamiz:

```
ab
aa
bb
ba
```

Break va continue operatorlari. Biz siklni boshqarish uchun maxsus break va continue operatorlaridan foydalanishimiz mumkin. Break operatori siklni to'xtatadi. continue operatori esa siklning keyingi iteratsiyasiga o'tkazadi.

Agar siklda uning keyingi bajarilishiga mos kelmaydigan shartlar hosil bo'lsa, break operatoridan foydalanish mumkin. Quyidagi misolni ko'rib chiqing :

```
raqam = 0
while raqam <5:
    raqam += 1
    if raqam == 3: # agar raqam = 3 bo'lsa, sikldan chiqamiz
        break
    print(f"raqam = {raqam}")
```

Bu yerda while sikli raqam<5 shartini rostlikka tekshiradi. Va agar raqam o'zgaruvchisining qiymati hozircha 5 ga teng bo'lmasa, raqamning qiymati konsolda chop etiladi. Ammo ungacha sikl ichida yana bir shart ham tekshiriladi: if raqam == 3. Ya'ni **raqam**ning qiymati 3 ga teng bo'lsa, u holda break operatori yordamida sikldan chiqamiz. Va oxirida biz quyidagi konsol chiqishini olamiz:

```
raqam = 1
raqam = 2
```

Break operatoridan farqli o'laroq, **continue** operatori siklning keyingi iteratsiyasiga uni tugatmasdan o'tadi. Misol uchun, oldingi misolda break ni continue bilan almashtiramiz:

```
raqam = 0
while raqam <5:
    raqam += 1
    if raqam == 3: # agar raqam = 3 bo'lsa, siklning yangi
iteratsiyasiga o'ting
        continue
    print(f" raqam = {raqam}")
```

Va bu holda, agar raqam o'zgaruvchisining qiymati 3 ga teng bo'lsa, continue operatori siklni keyingi aylanishga o'tkazib yuboradi va navbatdagi qadam amalga oshiriladi:

```
raqam = 1
raqam = 2
raqam = 4
raqam = 5
```

Nazorat savollari :

1. Dasturlashda takrorlanuvchi jarayonlarni dasturlash qanday ro‘l egallaydi?
2. Takrorlanuvchi jarayonlarni qanday operatorlar yordamida dasturini tuziladi?
3. **for** operatorining ishlash prinsipi qanday?
4. **while** operatorining sintaksisi va ishlash prinsipi qanday?
5. **for** va **while** operatorlarining bir biridan asosiy farqlari nimada?
6. **else** operatorini siklik jarayonlarda nima maqsadda foydalaniladi?
7. **range** operatorining vazifasi va ishlash prinsipi qanday?
8. **break** operatorining vazifasi nima?
9. **continue** operatori qanday holatlarda ishlatiladi?

2.8.§. Funktsiyalar

Funktsiyalar ma’lum bir vazifani bajaradigan va dasturning boshqa qismlarida qayta ishlatilishi mumkin bo‘lgan kod blokini ifodalaydi. Funktsiyalardan oldingi mavzularni tushuntirish davomida allaqachon foydalaib o‘tildi. Masalan, konsolga qiymatlarni chop etadigan **print()** funksiyasidan deyarli har bir mavzuda foydalanildi. Python ko‘plab standart o‘rnatilgan funktsiyalarga ega bo‘lgan dasturlash tilidir. Bunday standart funktsiyalardan tashqari foydalanuvchi o‘ziga kerakli funktsiyalarni yaratish imkoni mavjud. Funktsiya yaratish uchun quyidagi sintaksisdan foydalaniladi:

```
def function_name([parametrlar]):
    [funksiya tanasi]
```

Pythonda funksiya boshqa dasturlash tillaridan farqli ravishda `def` kalit soʻzi bilan boshlanadi. `Def` kalit soʻzidan keyin funksiya nomi va agar qabul qiluvchi parametrlari mavjud boʻlsa, ularni qavs ichida, vergul bilan ajratgan holda nomlari keltiriladi va qavslardan keyin osti-usti ikki nuqta qoʻyiladi. Va keyingi qatordan funksiya bajaradigan koʻrsatmalar bloki boshlanadi. Barcha funksiya tanasidagi operatorlar satr boshidan ichkariga chekingan holda bir ustundan boshlanishi kerak.

Masalan, eng oddiy funktsiyaning koʻrinishi:

```
def say_hello():  
    print ("Salom")
```

Funktsiya nomi: `say_hello` deb ataladi. U hech qanday parametrlarga ega emas va konsolga "Salom" satrini chop etadigan bitta operatorni oʻz ichiga oladi.

Eʼtibor bering, funksiya iboralari funksiya boshidan chekinishi kerak. Masalan:

```
def say_hello():  
    print("Salom")  
print("Xayr")
```

Bu yerda `print("Xayr")` buyrugʻi `say_hello` funksiyasi bilan bir ustunda va shuning uchun bu funktsiyaga tegishli boʻlmagan operator hisoblanadi. Funktsiya tarkibiga kiruvchi blok va funktsiyaga tegishli boʻlmagan operatorlar bir-biri bilan oldidagi qoldirilgan tab (boʻsh joy) bilan farq qiladi.

Funktsiyaga murojaat. Funktsiyaning chaqirish uchun funksiya nomi va uning qabul qiluvchi parametrlariga mos qiymatlar qavs ichida koʻrsatiladi:

```
funktsiya_nomi ([parametrlar])
```

Masalan, salom tekstini konsolga chiqaruvchi bitta oddiy funksiya yaratamiz va uni chaqiramiz:

```
def hello():  
    print ("Salom")
```

```
hello()  
hello()  
hello()
```

Bu yerda hello funksiyasi ketma-ket uch marta chaqirilmoqda. Natijada, konsolda quyidagi natijalar paydo bo‘ladi:

```
Salom  
Salom  
Salom
```

Shuni esda tutish kerakki, funksiya yuqorida yaratiladi va keyin chaqiriladi.

Agar funksiya tarkibida faqat bitta operator mavjud bo‘lsa, bu operatorni funksiya ta’rifining davomidan ikki nuqtadan keyin joylashtirilishi mumkin:

```
def hello(): print("Salom")  
hello ()
```

Boshqa funktsiyalarni ham xuddi shunday aniqlash va chaqirish mumkin. Masalan, bir nechta funktsiyalarni aniqlaymiz va bajaramiz:

```
def say_hello():  
    print ("Salom")  
def say_goodbye():  
    print ("Xayr")  
say_hello ()  
say_goodbye ()
```

Natijaning konsolda ko‘rinishi :

```
Salom  
Xayr
```

Lokal funksiya. Ba’zi funktsiyalarni boshqa funktsiyalar ichida aniqlash mumkin. Bunday funktsiyalarni ichki funktsiyalar ya’ni **lokal funktsiyalar** deb

ataladi. Lokal funksiyalar faqat o'zi tegishli bo'lgan funksiya doirasida ishlatilishi mumkin. Masalan :

```
def print_msg():  
    def say_hello(): print("Salom")  
    def say_bye(): print("Xayr")  
    say_hello()  
    say_bye()  
print_msg()
```

Bu yerda `say_hello()` va `say_bye()` funksiyalari `print_msg()` funksiyasi ichida aniqlanadi va shuning uchun u uchun lokal fuksiya hisoblanadi. Shunga ko'ra, ular faqat `print_msg()` funksiyasi ichidagina ishlatilishi mumkin.

main funksiyasi va undan qo'llanilishi. Dasturda ko'plab funktsiyalar ishtirok etishi mumkin. Va ularning barchasini birlashtirish uchun asosiy bitta funksiya ishlatiladi. Odatda bu funksiya `main` deb ataladi va unda boshqa funktsiyalar chaqiriladi:

```
def main_f ():  
    say_hello ()  
    say_bye ()  
  
def say_hello():  
    print("Salom")  
  
def say_bye():  
    print("Xayr")  
  
# Asosiy funksiyani chaqirish  
main_f()
```

Nazorat savollari:

1. Lokal funksiya deb qanday funksiyalarga aytiladi?

2. Funksiyalar dasturlashda qanday ro‘l o‘ynaydi?
3. Foydalanuvchi funksiyalari qanday yaratiladi?
4. Funksiya tanasi va funksiyaga tegishli bo‘lmagan operatorlar qanday farqlanadi?

2.9.§. Parametrli funktsiya

Funksiyalarni parametrli va parametrsiz funksiyalarga ajratishimiz mumkin. Parametrsiz funksiyalar tashqaridan hech qanday qiymat qabul qilmaydi. Parametrli funksiyalar esa funksiya chaqirilayotganda biror bir qiymatni berilishini kutadigan funksiyalardir. Ushbu parametrlar orqali funksiyaga turli ma’lumotlarni uzatish mumkin. Buni **print()** funksiyasi misolida ko‘rib chiqamiz. **print()** parametr sifatida konsolga chop etilishi kerak bo‘lgan xabarli qabul qiladi.

Endi bitta oddiy, ism paramrli funksiya yaratamiz va uni chaqiramiz:

```
def say_hello(ism):  
    print(f"Salom, { ism }")  
  
say_hello("Jamshid")  
say_hello("Boboy")  
say_hello("Asilbek")
```

Say_hello funksiyasi ism parametriga ega va funksiya chaqirilganda ushbu parametrba ba’zi qiymatlarni berish mumkin. Funksiya ichida parametrni oddiy o‘zgaruvchi sifatida ishlatish mumkin. Masalan, ushbu parametrning qiymatini print funksiyasi bilan konsolga chop etish. Shunday qilib, ifodada:

```
say_hello("Jamshid ")
```

"Jamshid" qatori ism parametriga uzatiladi. Natijada, dasturni bajarishda quyidagi natijani konsol chiqaradi:

```
Salom, Jamshid  
Salom, Boboy  
Salom, Asilbek
```

Funktsiya chaqirilganda, qiymatlar pozitsiya bo'yicha parametrlarga o'tkaziladi. Masalan, bir nechta parametrli funktsiyani quyidagicha yaratish va chaqirish mumkin:

```
def print_per (ism, yosh):  
    print(f"Ism: {ism}")  
    print(f"Yosh: {yosh}")  
  
print_per ("Jamshid", 25)
```

Bu yerda `print_per` funktsiyasi ikkita parametрни qabul qiladi: `ism` va `yosh`. Funktsiyani chaqirganda :

```
print_per("Jamshid", 25)
```

funktsiya chaqirilish vaqtidagi unga parametr sifatida beriladigan qiymatlar mos parametrlarga qabul qilinadi. Ya'ni birinchi qiymat – "Jamshid" birinchi parametrga, ya'ni `ism` parametriga qabul qilinsa, ikkinchi qiymat – 25 bo'lsa, `yosh` parametriga qabul qilinadi. Va funktsiya ichida parametr qiymatlari konsolga chop etiladi:

```
Ism: Tom  
Yosh: 37
```

Standart qiymatlar

Funktsiyani yaratish mobaynida, funktsiyaning mavjud parametrlari uchun boshlang'ich qiymatlarni belgilash orqali funktsiyaning ba'zi parametrlariga qiymat berishni ixtiyoriy qilinishi mumkin. Masalan :

```
def say_hello(ism="Jamshid") :  
    print(f"Salom, {ism}")  
  
say_hello()          # bu yerda ism = "Jamshid"  
say_hello("Bobo")    # bu yerda ism = "Bobo"
```

Bu yerda `ism` parametriga qiymat berish majburiy emas. Ya'ni, agar funktsiyani chaqirishda uning `ism` parametriga qiymat berilmasa standart qiymat – "Jamshid" qo'llaniladi. Ushbu dasturning konsolda ko'rinishi quyidagicha bo'ladi:

Salom, Jamshid

Salom, Bobo

Agar funktsiya bir nechta parametrlarga ega bo'lsa, qiymat berish ixtiyoriy bo'lgan parametrlar, qiymat berish majburiy bo'lgan qilinganlardan keyin kelishi kerak. Masalan :

```
def print_per(ism, yosh = 54):  
    print(f"Ism: {ism}    Yosh: {yosh}")  
print_per("Bobo")  
print_per("Jamshid", 25)
```

Bu yerda **yosh** parametri ixtiyoriy va standart bo'yicha 54 ga teng. Undan oldin qiymat berish majburiy bo'lgan **ism** parametri joylashgan. Shuning uchun funktsiyani chaqirishda **yosh** parametriga qiymat o'tkaza olmaymiz, lekin **ism** parametriga qiymat uzatish kerak.

Agar kerak bo'lsa, biz barcha parametrlarni ixtiyoriy qilishimiz mumkin:

```
def print_per(ism = "Jamshid", yosh = 18):  
    print(f"Ism: {ism}    Yosh: {yosh}")  
print_per()                # Ism: Jamshid Yosh: 18  
print_per("Bobo")          # Ism: Bobo    Yosh: 18  
print_per("Samat", 54)     # Ism: Samat   Yosh: 54
```

Nomlangan parametrlar. Yuqoridagi misollarda funktsiya chaqirilganda parametrlarga qiymatlar joylashish o'рни bo'yicha uzatildi. Ammo qiymatlarni parametrlarga nomi ko'ra uzatish ham mumkin. Buning uchun funktsiyani chaqirishda parametr nomi ko'rsatiladi va unga qiymat beriladi:

```
def print_per(ism, yosh):  
    print(f"Ism: {ism}    Yosh: {yosh}")  
  
print_per(yosh = 22, ism = "Jamshid")
```

Bunday holda, **yosh** va **ism** parametrlari nomiga ko'ra qiymatlarni topib olishadi. Funktsiya yaratilayotganda **ism** parametri birinchi o'rinda turibdi. Shunga qaramay, funktsiyani chaqirishda `print_per(yosh= 22, ism = "Jamshid")`

kabi chaqirilsa ham bo‘ladi va shu tariqa **yosh** parametriga 22 ni va **ism** parametriga "Jamshid" qiymatini berish mumkin.

Funksiya parametriga faqat nomi bo‘yicha qiymat berish. * belgisi qaysi parametrlarga nom berilishini belgilash imkonini beradi. Ya’ni qiymatlarni faqat nomi bo‘yicha uzatish mumkin bo‘lgan parametrlar. * belgisining o‘ng tomonida joylashgan barcha parametrlar qiymatlarni faqat nomi bilan qabul qiladi:

```
def print_per(ism, *, yosh, tashkilot):  
    print(f"Ism: {ism} Yosh: {yosh} Tashkilot: {tashkilot}")  
  
print_per("Bobo", yosh = 41, tashkilot = "Microsoft")
```

Bunday holda, **yosh** va **tashkilot** parametrlari nomi bilan qiymatlar qabul qilishadi.

Parametrlar ro‘yxatiga * bilan prefiks qo‘yish orqali barcha parametrlarni nomlash mumkin:

```
def print_person(*, ism, yosh, tashkilot):  
    print(f"Ism: {ism} Yosh: {yosh} Tashkilot: {tashkilot}")
```

Aksincha, siz qiymatlarni faqat pozitsiya bo‘yicha, ya’ni pozitsion parametrlar bo‘yicha qabul qilish mumkin bo‘lgan parametrlarni belgilashingiz kerak bo‘lsa, unda / belgisidan foydalanish mumkin.

Bunda / belgisidan olda joylashgan barcha parametrlar faqat joylashish o‘rniga ko‘ra qiymatlar va qabul qilishi mumkin.

```
def print_per(ism, /, yosh, tashkilot="Micros"):  
    print(f"Ism:{ism} Yosh: {yosh} Tashkilot: {tashkilot}")  
  
print_per("Jamshid", tashkilot="ShamsWeb", yosh = 11)  
print_per("Bobo", 41)
```

Bunday holda, **ism** parametri pozitsiondir.

Bitta funktsiyada bir vaqtning o‘zida pozitsion va nomli parametrlar mavjud bo‘lishi mumkin.

```
def print_per(ism, /, yosh = 18, *, tashkilot):
    print(f"Ism: {ism} Yosh:{yosh} Tashkilot: {tashkilot}")

print_per("Samat", tashkilot = "Google")
print_per("Tomson", 37, tashkilot = "JetBrains")
print_per("Bobo", tashkilot = "Micros", yosh= 42)
```

Bunday holda, **ism** parametri / belgisining chap tomonida joylashgan, shuning uchun u pozitsion, majburiydir hamda faqat pozitsiya bo'yicha qiymat berilishi mumkin.

Tashkilot parametri * belgisining o'ng tomonida joylashganligi sababli nomlangan parametr hisoblanadi. **Yosh** parametri nomi bo'yicha ham, joylashgan o'ri bo'yicha ham qiymat qabul qilishi mumkin.

Aniqlanmagan sondagi parametrlar. Funksiyaga noma'lum miqdordagi qiymatlarni uzatish uchun yulduzcha belgisidan foydalaniladi. Bu funktsiyaga nomalum miqdordagi, bir nechta qiymatlarni qabul qilishda qo'llaniladi. Masalan, sonlar yig'indisini hisoblash funksiyasini ko'rib chiqamiz:

```
def sum(*raqamlar):
    natija = 0
    for j in raqamlar:
        natija += j
    print(f"sum = {natija}")

sum(1, 2, 3, 4, 5)          # sum = 15
sum(3, 4, 5, 6)            # sum = 18
```

Bunday holda, **sum** funktsiyasi bitta parametrga oladi - * **raqamlar**, lekin parametr nomi oldidagi yulduzcha, aslida ushbu parametr o'rniga cheksiz miqdordagi qiymatlarni yoki qiymatlar to'plamini qabul qilish mumkinligini ko'rsatadi. Funktsiyaning o'zida for siklidan foydalanib, ushbu to'plamdan birma-bir elementlarni olish mumkin, ushbu to'plamdan har bir qiymatni j o'zgaruvchiga olish

va u bilan qandaydir amallarni bajarish mumkin. Misol uchun, bu dasturda uzatilgan raqamlarning yig'indisi hisoblandi.

Nazorat savollari:

1. Parametrli va parametrsiz funksiyalarning bir-biridan farqi nimada?
2. Funksiya parametr sifatida qanday turdagi ma'lumotlarni qabul qilishi mumkin?
3. Funksiya parametrining tinchlik qiymati qanday beriladi va bu qanday hollarda kerak bo'ladi?
4. Nomlangan parametrlar deb qanday parametrlarga aytiladi?
5. Aniqlanmagan sondagi parametrlar deb qanday parametrlarga aytiladi?

2.10.§. Qiymat qaytaruvchi funktsiya

Natijani qaytarish. Odatda funksiya bajargan amalning natjasidan dastur davomida foydalanishga zarurat paydo bo'ladi. Bunday hollarda natija qaytaruvchi funksiyalardan foydalaniladi. Funksiya natija qaytarishi uchun **return** operatoridan foydalaniladi, shundan so'ng qaytarish qiymati ko'rsatiladi:

```
def funksiya_nomi([parametrlar]):  
    [operatorlar bloki]  
    return qaytariladigan_qiymat
```

Qiymat qaytaruvchi eng oddiy funktsiyani yaratamiz:

```
def get_msg():  
    return "Salom, QALAYSIZ?"
```

Bu yerda, `get_msg()` funksiyasi qaytaradigan qiymat **return** operatoridan keyin kelgan - "Salom, QALAYSIZ?" xabaridir.

Ushbu funktsiya natijasi keyinchalik o'zgaruvchiga tayinlanishi yoki oddiy qiymat sifatida ishlatilishi mumkin:

```
def get_msg():
```

```

    return "Salom, QALAYSIZ?"

xabar = get_msg()
print(xabar)           # Salom, QALAYSIZ?
print(get_msg())       # Salom, QALAYSIZ?

```

Return operatoridan keyin murakkab hisoblash ifodasi ham kelishi mumkin. Uning natijasi funktsiyadan qaytariladi. Masalan, sonni ikki barobarga oshiradigan funktsiyani ko‘rib chiqaylik:

```

def ikkilantirish(son):

    return 2 * son

```

Bu yerda ikkilantirish funktsiya $2 * \text{son}$ ifodasi natijasini qaytaradi:

```

def ikkilantirish (son):
    return 2 * son

result1 = ikkilantirish (4)      # result1 = 8
result2 = ikkilantirish (5)      # result2 = 10
print(f"result1 = {result1}")    # result1 = 8
print(f"result2 = {result2}")    # result2 = 10

```

Yoki berilgan ikkita raqamning yig‘indisini hisblash funktsiyasi:

```

def summasi(a, b):
    return a + b

natija = summasi(4, 6) # natija = 10

print(f"summasi(4,6)={natija}") # summasi(4, 6) = 10
print(f"summasi(3,5)={summasi(3,5)}") #summasi(3,5)= 8

```

Funktsiyadan chiqish. Return operatori nafaqat qiymatni qaytaradi, balki funktsiyadan chiqishni amalga oshiradi. Agar return operatori bajarilganidan keyin yana qandaydir operator joyashgan bo‘lsa, unda bu operator bajarilmay qoladi. Chunki return qo‘llanilganidan so‘ng funktsiya tugatiladi. Shuning uchun return operatori barcha amallar bajarilgandan keyin qo‘llanilishi kerak. Masalan:

```
def get_msg():
    return "Salom, Qandaysan?"

    print("Funksiyaning tugatilishi")

print(get_msg())
```

Sintaksis nuqtai nazaridan, bu funktsiya to‘g‘ri, lekin uning `print("Funksiyaning tugatilishi")` amali mantiqiy emas - u hech qachon bajarilmaydi, chunki u bajarilishidan oldin `return` operatori qiymatni qaytaradi va funktsiyadan chiqadi.

Biroq, hech qanday qiymat qaytarmaydigan funktsiyalarda `return` operatoridan qiymatsiz foydalaniladi. Bunday holda, `return` operatoridan keyin hech qanday qaytarish qiymati qo‘yilmaydi. Odatiy holat - muayyan shartlarga qarab funktsiyadan chiqish:

```
def print_per(ism, yosh):

    if yosh > 120 or yosh < 1:
        print("Noto‘g‘ri yosh ")

        return

    print(f"Ism: {ism}   Age: { yosh }")

print_per("Jamshid", 22)
print_per("Bobo", -102)
```

Bu yerda `print_per` funktsiyasi parametr sifatida foydalanuvchi `ismi` va `yoshini` qabul qiladi. Biroq, funktsiyada birinchi navbatda `yoshni` ma’lum bir oraliqda (120 dan kam va 0 dan katta) ekanligini tekshiriladi. Agar `yosh` ushbu diapazondan tashqarida bo‘lsa, noto‘g‘ri `yosh` haqida xabarni ko‘rsatiladi va `return` operatori orqali funktsiyadan chiqiladi. Shundan so‘ng, funktsiya o‘z ishini tugatadi.

Biroq, agar `yosh` to‘g‘ri bo‘lsa, foydalanuvchi ma’lumotlarini konsolga chop etiladi. Konsol chiqishi:

```
Ism: Jamshid   Yosh: 22
Noto‘g‘ri yosh
```


Nazorat savollari:

1. Qiymat qaytaruvchi va qaytarmaydigan funksiyalarning bir-biridan farqi nimada?
2. Funksiyadan shartga asosan chiqib ketish uchun qaysi kalit soʻzdan foydalanish kerak?
3. Funksiya ichida return operatoridan keying operatorlar bloki yozilgan boʻlsa, qanday hodisa roʻy beradi?

2.11.§. Lambda funktsiya

Python dasturlash tilidagi lambda ifodalari - bu lambda operatori yordamida aniqlangan kichik anonim funksiyalar. Lambda ifodasining sintaksisi quyidagicha:

```
lambda [parametrlar]: operatorlar
```

Eng oddiy lambda ifodasini aniqlaymiz:

```
msg = lambda: print ("salom")  
msg()      # salom
```

Bu yerda **lambda** ifodasi **msg** oʻzgaruvchisiga yuklatilgan. Ushbu misolda **lambda** ifodasi hech qanday parametrga ega emas, hech narsa qaytarmaydi va shunchaki konsolga "salom" xabarini chop etadi. Va **msg** oʻzgaruvchisi orqali bu **lambda** ifodasini oddiy funktsiya kabi chaqirishimiz mumkin. Aslida, u quyidagi funktsiyaga oʻxshaydi:

```
def msg():  
    print("salom")
```

Agar **lambda** ifodasi parametrlarga ega boʻlsa, ular **lambda** kalit soʻzidan keyin aniqlanadi. Agar **lambda** ifodasi natijani qaytarsa, u holda u ikki nuqtadan keyin koʻrsatiladi. Masalan, sonning kvadratini qaytaruvchi **lambda** ifodasini yozaylik:

```
kvadrat = lambda a : a * a
```

```
print(kvadrat(4))      # 16
print(kvadrat(5))      # 25
```

Bunday holda, lambda ifodasi bitta a nomli parametrni qabul qiladi va undan keyingi Ikki nuqtaning o'ng tomonida qaytariladigan qiymat - $a * a$. Ushbu lambda ifodasi quyidagi funktsiyaning alternativi hisoblanadi:

```
def kvadrat2(a): return a * a
```

Xuddi shu usulda, bir nechta parametrlarni qabul qiluvchi lambda ifodalarini yaratish mumkin:

```
sum = lambda a, b: a + b

print(sum(4, 5))      # 9
print(sum(5, 6))      # 11
```

Lambda ifodalari funksiya ta'riflarini biroz qisqartirishga imkon bersada, ular faqat bitta operatsiyani bajarishi mumkinligi bilan cheklangan. Biroq, parametr sifatida o'tish yoki boshqa funktsiyaga qaytish uchun funktsiyadan foydalanish kerak bo'lgan hollarda ular juda qulay bo'lishi mumkin. Masalan, lambda ifodasini parametr sifatida o'tkazish:

```
def do_operatsiya(a, b, operatsiya):
    result = operatsiya(a, b)
    print(f"result = {result}")

do_operatsiya(5, 4, lambda a, b: a + b)  # result = 9
do_operatsiya(5, 4, lambda a, b: a * b)  # result = 20
```

Bunday holda, oxirgi mavzuda bo'lgani kabi, ularni parametr sifatida jo'natish uchun funktsiyalarni belgilashimiz shart emas.

Xuddi shu narsa funktsiyalardan lambda ifodalarini qaytarish uchun ham amal qiladi:

```

def tanlash_funk(tanlangan):
    if tanlangan == 1:
        return lambda a, b: a + b
    elif tanlangan == 2:
        return lambda a, b: a - b
    else:
        return lambda a, b: a * b

operation = tanlash_funk (1)  # operation = a+b
print(operation(10, 6))  # 16

operation = tanlash_funk (2)  # operation = a-b
print(operation(10, 6))  # 4

operation = tanlash_funk (3)  # operation = a*b
print(operation(10, 6))  # 60

```

Nazorat savollari:

1. Lambda funksiyasining vazifasi nima?
2. Lambda funksiyasining sintaksisi va ishlash prinsipi qanday?
3. Oddiy funksiyadan qanday afzallik va kamchiliklarga ega?

2.12.§. O‘zgaruvchilarning tiplarini o‘zgartirish

Ma’lumotlar bilan ishlashda har xil turdagi qiymatlardan foydalanish mumkin. Masalan, int, float yoki strng va hokazo tiplardan. Ikkita sonli har xil tiplarni bir-biri bilan qo‘shib yoki boshqa bir arifmetik amallarni bajarib ko‘rilsa, natijadagi hosil bo‘lgan son ularning umumiy bitta katta to‘lamining tipini qabul qilganini ko‘rishimiz mumkin. Masalan quyidagi misolda:

```

a = 2          # int turdagi raqam
b = 2,5        # float turdagi raqam

```

```
c = a + b
```

```
print(c)      # 4.5
```

Bunday holda, hech qanday xato bo'lmaydi. Biroq, Python har doim ham har xil turdagi ma'lumotlarni o'z ichiga olgan operatsiyalarni avtomatik ravishda bajara olmaydi. Keling, bu holatda qanday qoidalar qo'llanilishini ko'rib chiqaylik.

Yashirin konvertatsiyalar. Arifmetik amallardagi ikkala raqam ham bir xil turni ifodalashi kerak. Agar operatsiyaning ikkita operandlari turli xil ma'lumotlar turlarini ifodalasa, Python quyidagi qoidalariga muvofiq avtomatik ravishda turlardan biriga aylantirishga harakat qiladi:

- Agar operatsiya operandlaridan biri kompleks sonni (tur kompleksini) ifodalasa, boshqa operand tipi ham kompleks songa aylantiriladi.
- Agar, operandlardan biri float turiga ega bo'lsa, ikkinchi operand ham float tipiga o'tkaziladi. Aslida, bu yuqoridagi misolda sodir bo'ldi, bu yerda a o'zgaruvchisining qiymati float turiga aylantirildi.
- Agar, ikkala operand ham int tipida bo'lsa, tiplar o'zgarishsiz qoldiriladi va natija int tipida bo'ladi.

Aniq konvertatsiyalar. Ba'zi hollarda o'zgaruvchilarning tiplarini o'zgartirishni qo'lda bajarishga to'g'ri keladi. Masalan, bizda quyidagi kod bor deylik:

```
a = "2"
```

```
b = 3
```

```
c = a + b
```

Biz "2" + 3 ifoda 5 ga teng bo'lishini kutamiz. Biroq, bu kod istisno qiladi, chunki birinchi raqam satrni ifodalaydi. Va kodni bajarishda biz shunga o'xshash narsani ko'ramiz:

```
Traceback (most recent call last):
```

File

```
"C:/Users/SHAdmin/AppData/Local/Programs/Python/Python310/
/ssssssssssssssssss.py", line 3, in <module>
```

```
c = a + b
```

TypeError: can only concatenate str (not "int") to str
Python turini o'zgartirish uchun bir qator o'rnatilgan funktsiyalarni taqdim etadi:

int(): qiymatni butun songa aylantirish

float(): qiymatni floatga aylantiradi

str(): qiymatni satrga aylantirish

INT. Shunday qilib, oldingi misolda int() funksiyasi yordamida satrni raqamga aylantiramiz:

```
a = "2"
b = 3
c = int(a) + b
print(c)      # 5
```

int() yordamida konversiyalarga misollar:

```
a = int(15)      # a = 15
b = int(3.7)     # b = 3
c = int("4")     # c = 4
e = int(False)   # e = 0
f = int(True)    # f = 1
```

Biroq, agar qaysidir bir qiymatni int tipiga otirishni ilojisi bo'lmasa, u holda int() funksiyasi quyidagi xatolikni qaytaradi: *ValueError: invalid literal for int() with base 10*:

```
b = int("a1c")      # Xato
c = int("4.7")      # Xato
```

FLOAT. `float()` funksiyasi ham xuddi shunday ishlaydi, biroq vergulli sonlarni ya'ni kasr sonlarni raqamga aylantirish uchun xizmat qiladi. Masalan:

```
a = "2,7"
b = 3
c = float(a) + b

print(c) # 5.7
```

`float()` funksiyasi yordamida konversiya qilishga misollar:

```
a = float (15)      # a = 15,0
b = float (3,7)     # b = 3,7
c = float ("4,7")   # c = 4,7
d = float ("5")     # d = 5,0
e = float (False)   # e = 0,0
f = float (True)    # f = 1,0
```

Ammo yana, barcha qiymatlarni avtomatik ravishda floatga aylantirib bo'lmaydi. Shunday qilib, quyidagi holatda Python xatolik keltirib chiqaradi:

```
d = float ("abcd")  # False
```

STR. `str()` funksiyasi barcha qiymatlarni satrga aylantirish maqsadida foydalaniladi. Quyida unga doir misollar keltirilgan:

```
a = str (False)     # a = "False"
b = str (True)      # b = "True"
c = str (5)         # c = "5"
d = str (5.7)       # d = "5.7"
```

str() funksiyasi juda ko‘plab hollarda foydalaniladi. Masalan sonni satrga shunday holicha qo‘shishda xatolik berganligi uchun str funksiyasi orqali sonni satrga aylantirib keyin satr va son birlashtiriladi. Bunga qiyida misollar keltirilgan:

```
yosh = 22
msg = "Yoshi: " + yosh      # xato
print(msg)
```

Agar raqamga raqam qo‘shilsa, bu standart holat hisoblanadi. Agar satrga satr qo‘shilsa, u holda satrlarni birlashtirish operatsiyasi deb tushiniladi va birlashtirish amali bajariladi. Ammo python turli tipdagi ya’ni, satr va raqamga nisbatan qo‘shish amalini qanday bajarishni bilmaydi. Va agar bu holda biz satrlarni birlashtirish operatsiyasini bajarmoqchi bo‘lsak, u holda raqamni str () funksiyasi yordamida satrga aylantirish talab qilinadi:

```
yosh = 22
msg = "Yoshi: " + str(yosh)    # Yoshi: 22
print(msg)
```

Nazorat savollari:

1. O‘zgaruvchilarning tiplarini o‘zgartirish dasturlashda qanday holatlarda ishlatiladi?
2. O‘zgaruvchilarning turini qanday o‘zgartiriladi?
3. Arifmetik amallar bajarilayorganda o‘zgaruvchilarning turlari har xil bo‘ladigan bo‘lsa, operatsiyalar qanday amalga oshiriladi?
4. Yashirin konvertatsiya nima?
5. Aniq konvertatsiya nima?

III BOB. SATRLAR BILAN ISHLASH

3.1.§. Satrlar va ularni e'lon qilish

Satr deb mavjud belgilar ketma-ketligiga aytiladi. Satrlarni pythonda aniqlash uchun ham bittalik, ham qo'sh tirnoqlardan foydalaniladi. Masalan:

```
msg = 'Salom Dunyo!'
print(msg)    # Salom Dunyo!

nomi="Jeyson"
print(nomi)   # Jeyson
```

Agar satrdagi simvollar soni ko'p bo'lsa, uni qismlarga bo'lish va ularni turli xil kod satrlariga joylashtirish mumkin. Bunday holda, butun satr qavs ichiga olinadi va uning alohida qismlari tirnoq ichiga joylashtiriladi:

```
text = ("Salom hurmatli kursantlar"
        "Bugun biz sizlar bilan"
        "Satrlar ustida amallar bajarib ko'ramiz"
)
print(text)
```

Agar ko'p qatorli matnni hosil qilish kerak bo'lsa, unda bunday matn ikki tomondan uch qo'sh (""") yoki bittalik (``) tirnoq ichiga olinib yoziladi:

```
'''
Bu izoh
'''
text = '''Salom hurmatli kursantlar
Bugun biz sizlar bilan
Satrlar ustida amallar
bajarib ko'ramiz
'''
print(text)
```


Faqat shuni esda tutish kerakki uchta qo'shtirnoqdan izohlar uchun ham foydalaniladi. Agar uchta qo'shtirnoq ichidagi matn o'zgaruvchiga tayinlangan bo'lsa, unda bu izoh emas, satr hisoblanadi. Izoh qoldirish uchun esa izoh bo'lishi kerak bo'lgan matnni uchta qo'shtirnoqqa o'rab olinadi holos.

Satrdagi qo'llaniladigan maxsus simvollar. Satr bir qator maxsus belgilarni o'z ichiga olishi mumkin. Ulardan ba'zilar quyida keltirilgan:

- ❖ `\:` satr tarkibiga slesh simbolini qo'shish imkonini beradi
- ❖ `\'`: satr ichiga bitta tirnoq qo'shish imkonini beradi
- ❖ `\"`: qatorga qo'sh tirnoq qo'shish imkonini beradi
- ❖ `\n`: Yangi qatorga o'tadi
- ❖ `\t`: Tabulyatsiya qo'shadi (4 ta probelga teng bo'sh joy)

Quyida ba'zi maxsus belgilardan satr ichida foydalanish bo'yicha misollar keltirilgan:

```
text = "Xabar:\n\"Salom Dunyo\""
print(text)
```

Dasturning konsoldagi ko'rinishi:

```
Xabar:
"Salom Dunyo"
```

Garchi bunday maxsus belgilar satrga bittalik yoki qo'sh tirnoq qo'yish, jadval tuzish, boshqa qatorga o'tkazish kabi ko'plab amallarni bajarishga imkon bersada, ba'zi hollarda ular dasturda xatolik keltirib chiqaradi. Masalan:

```
URL = "D:\python_course\name.txt"
print(URL)
```

Bu yerda URL o'zgaruvchisi faylga yo'lni o'zlashtirib oladi. Biroq, satr ichida "\n" maxsus belgisi mavjud bo'lib, garchi dasturchi bu belgini faylga yo'lni ko'rsatish uchun foydalanayotgan bo'lsada, python interpretatori uni maxsus operator sifatida tarjima qiladi. Shunday qilib satrni ekranga chiqarishda natija quyidagicha bo'ladi:

```
c:\python_course  
ame.txt
```

Bunday vaziyatni oldini olish uchun satr oldiga r belgisi qo'yiladi:

```
URL = r"C:\python_course\name.txt"  
print(URL)
```

```
c:\python_course\name.txt
```

Qatorga qiymatlarni kiritish

Python boshqa o'zgaruvchilarning qiymatlarini satrga joylashtirish imkonini beradi. Buning uchun satr ichida o'zgaruvchilar jingalak qavslarga {} joylashtiriladi va satr oldidagi qo'shtirnoq belgisi oldiga f harfi qo'yiladi:

```
user_name = "Jeyson"  
user_age = 22  
user = f"Ismi: { user_name }   Yoshi: { user_age }"  
  
print(user_age)    # Ismi: Jeyson   Yoshi: 22
```

Bunday holda, user_name o'zgaruvchisining qiymati {user_name} o'rniga kiritiladi. Xuddi shunday, {user_age} o'rniga user_age o'zgaruvchisining qiymati kiritiladi.

Satr tarkibidagi belgilarga murojaat qilish. Python dasturlash tilida ham boshqa dasturlash tillaridagi kabi satrning tarkibidagi simvollarga murojaat qilib ular ustida turli amallar bajarish mumkin. Satr tarkibidagi belgilarga murojaat qilish uchun satr_nomi va kvadrat qavslar ichida simvolning indeksi kiritiladi. Indeks bu yerda

simvolning satrdagi joylashgan o'рни bo'lib, simvollar satrda 0 dan boshlab indexlangan bo'ladi. Demak satrdagi simvollar soni oxirgi simvolning indeksidan bittaga ortiq bo'ladi. Quyida satr tarkibidagi belgilarga murojaat qanday amalga oshirilishiga misol keltirilgan:

```
str = "hello world!"
belgi = str[0]    # h
print(belgi)

belgi = str[6]    # w
print(belgi)

belgi = str[11]   # error: IndexError: string index out of range
print(belgi)
```

Agar satrda mavjud bo'lmagan indeksdagi elementga murojaat qilinsa, `IndexError` xatoligi kelib chiqadi. Misol uchun, yuqoridagi holatda, `str` nomli satr 11 ta belgidan iborat va belgilari 0 dan 10 gacha indekslarga ega. Shuning uchun `str[11]` ifodasi xatolik qaytardi. Chunki satrda 11 ta element va oxirgi elementning indeksi 10 ga teng.

Satr oxiridan boshlab elementlarga murojlat qilish uchun manfiy indekslardan foydalanish mumkin. Shunday qilib, indeks -1 oxirgi belgini va indeks -2 oxirgi belgidan bitta oldingi ya'ni oxiridan 2- belgini ifodalaydi va hokazo. Masalan:

```
str = "salom dunyo!"
belgi = str[-1]

print(belgi)        #!

belgi2 = str[-5]

print(belgi2)       #u
```

Belgilar bilan ishlashda satr o'zgarmas (immutable) tip ekanligini hisobga olish kerak, shuning uchun satrning har qanday individual belgisini o'zgartirishga harakat qilsak, quyidagi holatda bo'lgani kabi xatoga duch kelamiz:

```
str = "Salom DASTURCHILAR!"  
str[1] = "R"
```

```
TypeError: 'str' object does not support item assignment
```

Faqat satr qiymatini unga boshqa qiymat belgilash orqali butunlay qayta o'rnatishimiz mumkin.

Satrning barcha elementlarini sikl yordamida olish. Satrdagi barcha belgilarni for tsiklini qo'llash orqali olish mumkin:

```
str = "salom dunyo"  
for belgi in str:  
    print(belgi)
```

```
s  
a  
l  
o  
m  
  
d  
u  
n  
y  
o
```

Matndan belgilar ketma-ketligini olish. Satr tarkibidan nafaqat bitta belgini, balki belgilar ketma-ketligini ham olish mumkin. Buning uchun quyidagi sintaksis qo'llaniladi:

str[:end] – ushbu ifoda 0 - indeksdan to end - indeksgacha belgilar ketma-ketligini chiqaradi. end – elementni o‘zini olmaydi. End o‘rnida son beriladi. str[:10]

str[start:end] – belgilar ketma-ketligi **start** dan to **end** - indeksigacha bo‘lgan belgilar ketma-ketligini qaytaradi. Ammo ikkinchi index – end xisobga olinmaydi.

string[start:end:step] – belgilar ketma-ketligini **start** dan to **end** - indeksigacha bo‘lgan belgilar ketma-ketligini **step** – qadami bilan qaytaradi.

Quyida **str** satrining tarkibidan belgilar ketma-ketligini olish uchun barcha variantlardan foydalanilgan dastur keltirilgan:

```
str = "salom dunyo"

# 0-indeksdan 5 - indeksigacha
sub_str1 = str[:5]
print(sub_str1)          # salom

# 2-indeksdan 5-indeksigacha
sub_str2 = str[2:5]
print(sub_str2)          #llo

# 2 - indeksdan 9-indeksigacha oraliqda 2 qadam bilan
sub_str3 = str[2:9:2]
print(sub_str3)          # lmdn
```

Satrlarni birlashtirish. Satrlar ustida eng keng tarqalgan amallardan biri ularni birlashtirish yoki qo‘shish (konkatinatsiya) qilish. Starlar ustida arifmetik qo‘shish amalini bajaradigan bo‘lsak, satrlarni birlashtirish amalini bajargan bo‘lamiz:

```
name = "Jeyson"
surname = "Stethom"

full_name = name + " " + surname

print(full_name)  # Jeyson Stethom
```

Ikki qatorni birlashtirish oson, lekin agar satr va raqam qo'shishi kerak bo'lsa-chi? Bunday holda `str()` funksiyasidan foydalanib raqamni satrga o'tkazish talab qilinadi:

```
name=" Jeyson "  
age = 38  
  
info_per = "Ism: " + name + " Yoshi: " + str(age)  
print(info_per)    # Ism: Jeyson Yoshi: 38
```

Satrni takrorlash. Satrlarga arifmetik ko'paytirish amalini qo'llash, ularning shuncha marta takrorlaydi:

```
print("char" * 3)    # charcharchar  
print("u" * 4)      # uuuu
```

Satrlarni taqqoslash. Satrlarni taqqoslashda asosiy diqqatni belgiga va ularning katta yoki kichik harfligiga qaratiladi. Demak, raqamli belgi shartli ravishda har qanday alifbo belgisidan kichik hisoblanadi. Katta harfdagi alifbo belgilari shartli ravishda kichik alifbo belgilaridan kichik hisoblanadi. Masalan:

```
str1 = "1a"  
str2 = "aa"  
str3 = "aa"  
print(str1>str2) #False, chunki str1 ni 1- belgisi raqam  
print(str2>str3) #Rost, chunki str2 ni 1- belgisi kichik harf
```

Shuning uchun **"1a"** qatori shartli ravishda **"aa"** qatoridan kichikdir. Avval birinchi belgi solishtiriladi. Agar ikkala satrning bosh belgilari raqamlarni ifodalasa, u holda kichikroq raqam kichikroq hisoblanadi, masalan, **"1a"** satri **"2a"** dan kichik.

Agar boshlang'ich belgilar bir xil holatda alifbo belgilarini ifodalasa, unda alifbo bo'yicha tartibiga qaraladi. Shunday qilib, **"aa"** **"da"** dan kichik va **"da"** **"fa"** dan kichikdir.

Agar birinchi belgilar bir xil bo'lsa, ikkinchi belgilar solishtiriladi va hokazo.

Satrlarni katta kichik harflarini bir biri bilan solishtirishdan qochish maqsadida ularning harflarini bir xilga keltirib olish mumkin. Ya'ni katta yoki kichik registrga o'tkazib olish mumkin.

lower() funksiyasi satrdagi barcha harflarni kichik harfga, **upper()** funksiyasi esa satrdagi barcha harflarni bosh harfga o'zgartiradi. Bu funkmsiyalar satrning o'ziga o'zgartirish kiritadi.

```
str1 = "Tomas"
str2 = "TOMAS"
print(str1 == str2)      # False - satrlar teng emas

print(str1.lower() == str2.lower())    # Rost
```

ord va **len** funktsiyalari. Satr Unicode belgilarni o'z ichiga olganligi sababli, Unicode belgisining raqamli qiymatini olish uchun **ord()** funktsiyasidan foydalanish mumkin:

```
print(ord("B"))        # 66
```

String uzunligini olish uchun **len()** funktsiyasidan foydalaniladi:

```
str = "salom dunyo!"
uzunlik = len(str)
print(uzunlik)        # 12
```

Satrdagi qidirish. **text in str** ifodasidan foydalanib, **str** qatoridan **text** satrini qidirib topish mumkin. Agar **str** satrida **text** topilsa, u holda ifoda **"True"** ni qaytaradi, aks holda **"False"** ni qaytaradi:

```
str = "salom dunyo"
is_true = "salom" in str
print(is_true)        # rost

is_true = "sdunyo" in string
print(is_true)        # False
```

Nazorat savollari:

1. Satr nima?
2. Satrlarni qanday e'lon qilinadi?
3. Satrning ichidan satr qidirish qanday amalga oshiriladi?
4. Satr uzunligini olish qanday amalga oshiriladi?

3.2.§. Satr bilan ishlovchi operatorlar va metodlardan foydalanish

Satrlar bilan ishlash uchun ko'p qo'llanladigan asosiy metodlarni ko'rib chiqaylik:

isalpha (): agar satr faqat alfavit belgilaridan iborat bo'lsa, True qaytaradi;

islower (): Agar satr faqat kichik harflardan iborat bo'lsa, True qiymatini qaytaradi;

isupper (): Agar satrdagi barcha belgilar katta harflardan iborat bo'lsa, True qiymatini qaytaradi;

isdigit (): agar satrning barcha belgilari raqam bo'lsa, True qiymatini qaytaradi;

isnumeric(): agar satr raqam bo'lsa, True qiymatini qaytaradi;

startswith(str): agar satr pastki qator str bilan boshlansa, True qiymatini qaytaradi;

endswith(str): agar satr oxiri **str** bilan tugasa, True qiymatini qaytaradi;

low(): satrni kichik harfga o'zgartiradi;

upper(): satrni katta harfga aylantirish;

title(): Satrdagi barcha so'zlarning birinchi harflarini bosh harfga o'zgartiradi;

capitalize(): satrning faqat birinchi so'zining birinchi harfini bosh harf bilan yozadi;

lstrip(): satr boshidagi probellarni o'chirib, satrni tozalaydi;

rstrip(): satr oxiridagi probellarni o‘chirib, satrni tozalaydi;

strip(): satrdan oldingi va keyingi bo‘shliqlarni olib tashlaydi;

ljust(width): agar satr uzunligi width parametridan kichik bo‘lsa, kenglik qiymatini to‘ldirish uchun satrning o‘ng tomoniga bo‘shliqlar qo‘shiladi va satrning o‘zi oqlanadi;

rjust(width): agar satr uzunligi width parametridan kichik bo‘lsa, kenglik qiymatini to‘ldirish uchun satrning chap tomoniga bo‘shliqlar qo‘shiladi va satrning o‘zi o‘ng tomonga asoslanadi;

center(width): agar satrning uzunligi kenglik parametridan kichik bo‘lsa, kenglik qiymatini to‘ldirish uchun satrning chap va o‘ng tomoniga bo‘shliqlar teng ravishda qo‘shiladi va satrning o‘zi markazlashtiriladi;

find(str[, start [, end]]): satrning ko‘rsatilgan intervaldan qism qatorini qidirish. Agar satrdan qism qator topilsa indeksini qaytaradi. Agar qator topilmasa, -1 raqami qaytariladi;

replace(old, new[, num]): satrdagi bir qism qatorni boshqasiga almashtiradi;

split([delimiter[, num]]): ajratuvchiga qarab qatorni qism qatorlarga ajratadi;

join(strs): bir nechta satrni ular orasiga ma’lum ajratuvchi qo‘yib, bir qatorga birlashtiradi.

Yuqorida keltirilgan metodlarni dasturda misollar bilan ko‘rib chiqaylik. Masalan, agar raqam klaviaturadan kiritilishi kerak bo‘lsa, kiritilgan satrni raqamga aylantirishdan oldin `isnumeric()` usuli yordamida haqiqatda raqam kiritilganligini tekshirishimiz mumkin va agar shunday bo‘lsa, raqamga o‘girish amalini bajariladi (***isnumeric()*** metodi):

```
str = input(" Raqamni kiriting :")
if str.isnumeric():
    raqam = int(str)
    print(raqam)
```

Satr ma'lum bir qism qator bilan boshlanishi yoki tugashini tekshirish (**startswith()** metodi):

```
f = "hello.py"

startsWithHello = f.startswith("hello") #True
endsWithExe = f.endswith("exe") # False
```

Satrning boshida va oxiridagi bo'shliqlarni olib tashlash (**strip()** metodi):

```
str = "    Salom    dunyo!    "
str = str.strip()
print(str)                # salom    dunyo!
```

Bo'shliqlar va hizalamalar bilan satrni to'ldirish (**rjust()** metodi)::

```
print("Redmi 7:", "500".rjust(10))
print("Nokia P10:", "360".rjust(10))
```

Konsol chiqishi:

```
Redmi 7:          500
Nokia N10:        360
```

Satrdagi qidirish

Satrdagi qism qatorni topish uchun Pythonda **find()** metodi mavjud, u satrda qism qatorning birinchi paydo bo'lish indeksini qaytaradi. **find()** metodining uchta ko'rinishi mavjud:

find(str): str qism satrini satr boshidan oxirigacha qidiriladi;

find(str, start): start parametri qidiriladigan boshlang'ich indeksni belgilaydi va start -indeksdan oxirigacha bo'lgan simvollar ketma-ketligidan qidiradi;

find(str, start, end): bu holda str qism satri satrning start va end indeksleri orasidan qidiradi.

Agar qism qator topilmasa, find metodi -1 qiymatni qaytaradi:

```

Hi_bye = "Salom dunyo! Xayr dunyo!"
indeks = Hi_bye.find("dun")
print(indeks)           #6

# 10 indeksdan qidirish
indeks = Hi_bye.find("dun",10)
print(indeks)           #21

# 10 dan 15 gacha indekslarni qidirish
indeks = Hi_bye.find("dun",10,15)
print(indeks)           # -1 - topilmaganini anglatadi

```

Satrd almashtirish (*replace*). Satrdagi bitta str satrini boshqasiga almashtirish uchun ***replace()*** metodidan foydalaning. Replace metodini bir nechta turda qo'llash mumkin:

***replace* (old, new):** satrdagi **old** satrlarni **new** satr bilan almashtiradi;

***replace*(old, new, num):** **num** parametri **old** satrlarning nechtasini **new** satri bilan almashtirishni belgilab beradi. Odatda **num** parametridan foydalanilmasa, u -1 qiymatni qabul qiladi, bu esa barcha **old** satrlarni **new** satri bilan almashtirilishini anglatadi.

```

tel_num = "+99-899-667-89-10"

# defislarni bo'sh joylar bilan almashtirish
tahrirlangan_telefon = tel_num.replace("-", " ")
print(tahrirlangan_telefon)           # +99 899 667 89 10

# defislarni olib tashlash
tahrirlangan_telefon = tel_num.replace("-", "")
print(tahrirlangan_telefon)           # +998996678910

```

```
# faqat birinchi defisni almashtirish
tahrirlangan_telefon = tel_num.replace("-", "", 1)
print(tahrirlangan_telefon)          # +99899-667-89-10
```

Qism qatorlarga ajratish. *Split()* metodi chegaralovchiga qarab qatorni qism qatorlar ro'yxatiga ajratadi. Ajratuvchi har qanday belgi yoki belgilar ketma-ketligi bo'lishi mumkin. Ushbu metod quyidagi ko'rinishlarda qo'llanilishi mumkin:

split(): ajratuvchi sifatida bo'sh joy qilinadi. Va probeldan probelgacha bo'lgan joylarni element qilib oladi.

split(delimiter): cheklovchi sifatida delimiter dan foydalaniladi.

split(delimiter, num): num parametri bo'linish uchun delimiter (ajratuvchi) ning necha marta ishlatilishini belgilaydi. Qatorning qolgan qismi qismlarga ajratilmasdan ro'yxatga qo'shiladi.

```
text = "Bu katta, ikki bo'yli eman, shoxlari singan
va po'stlog'i singan"
# probellardan ajratiladi

splitted_text = text.split()
print(splitted_text)
print(splitted_text[6])    #singan

# vergul ustida tanaffus
splitted_text = text.split(",")
print(splitted_text)
print(splitted_text[1])    # ikki bo'yli eman

# birinchi besh bo'shliqqa bo'lingan
splitted_text = text.split(" ", 5)
print(bo'lingan_matn)
print(bo'lingan_matn[5])

# shoxlari singan va po'stlog'i singan
```

Satrlarni birlashtirish. Satrlar ustidagi eng oddiy amallarni ko‘rib chiqa turib, qo‘shish amali yordamida qatorlarni birlashtirishni ko‘rsatdik. Satrlarni birlashtirishning yana bir imkoniyati - ***join()* usuli**: u bir nechta satrlarni birlashtirish uchun ishlatiladi.

Bundan tashqari, ushbu metod chaqirilgan joriy satr ajratuvchi sifatida ishlatiladi:

```
strings = ["Let's", "go", "speaking", "from", "my",  
"heart", "into", "My box"]
```

```
# ajratuvchi - bo'sh joy
```

```
sentence = " ".join(strings)
```

```
print(sentence)
```

```
# Let's go speaking from my heart into My box
```

```
# ajratuvchi - vertikal chiziq
```

```
sentence = " | ".join(strings)
```

```
print(sentence)
```

```
# Let's | go | speaking | from | my | heart | into | My box
```

Ro‘yxat o‘rniga oddiy satr qo‘shilish usuliga o‘tkazilishi mumkin, keyin ajratuvchi ushbu satr belgilari orasiga kiritiladi:

```
str = "hello"
```

```
joined_str = "\".join(str)
```

```
print(joined_str)           # h\e\l\l\o
```

Nazorat savollari:

1. satrlarning asosiy metodlariga misollar keltiring.
2. Satrda qidirish metodi qaysi?

3. Split() metodining vazifasi qanday va u parametr sifatida qanday qiymatlar qabul qiladi?
4. Len() – metodining vazifasi qanday?
5. Join() - metodining vazifasi qanday?
6. isalpha () - metodining vazifasi qanday?
7. islower () - metodining vazifasi qanday?
8. isupper () - metodining vazifasi qanday?
9. isdigit () - metodining vazifasi qanday?
- 10.startswith () - metodining vazifasi qanday?
- 11.endswith () - metodining vazifasi qanday?
- 12.replace () - metodining vazifasi qanday?

3.3.§. Format metodining qo'llanilishi

Oldingi mavzularda satr oldiga f belgisini qo'yish orqali ba'zi qiymatlarni qanday kiritish mumkinligi ko'rib chiqilgan edi. Masalan:

```
first_name="Jeyson"
text = f"Hello, {first_name}."
print(text)      # Hello, Jeyson.

name="Bobo"
age=73
info = f"Ismi: {name}\t Yoshi: {age}"
print(info)      # Ismi: Bobo  Yoshi: 73
```

Satrga o'zgaruvchining qiymatini kiritish uchun maxsus parametrlardan foydalaniladi, ular jingalak qavslar ({}) bilan o'rab olinadi.

Nomlangan parametrlar. Formatlanayotgan satrga chaqirilayotgan o'zgaruvchilarni **format()** metodiga parametr sifatida beriladi. Format metodi esa

ushbu parametrlarning qiymatlarini satr tarkibida jingalak qavslarda ko'rsatilgan mos nomlarga uzatadi:

```
txt = "Salom, {first_name}.".format(first_name="Jeyson")
print(txt)      # Salom, Jeyson.

info = "Ismi: {name}\t Yoshi: {age}".format(name="Bobo", age=73)
print(info)     # Ismi: Bobo  Yoshi: 73
```

Demak, formatlash metodida parametrlar qatordagi parametrlar bilan satr tarkibidagi jingalak qavslar ichidagi nomlar ham bir xil bo'llishi talab qilinadi. Shunday qilib, agar parametr yuqorida ko'rsatilga dasturda bo'lgani kabi `first_name` deb ataladigan bo'lsa, u holda qiymat tayinlangan argument ham `first_name` deb ataladi.

Joylashgan o'rni bo'yicha parametrlar. Shuningdek, ketma-ket argumentlar to'plamini formatlash usuliga o'tkazish va bu argumentlarni format satrining o'ziga, ularning indeksini jingalak qavslar ichida ko'rsatish mumkin (indekslash noldan boshlanadi):

```
info = "Ismi: {0}\t Age: {1}".format("Bobo", 73)
print(info)      # Ismi: Bobo  Age: 73
```

Bunday holda, argumentlar satrga bir necha marta chaqirilishi ham mumkin:

```
txt = "Salom, {0} {0} {0}.".format("Jeyson")
```

```
Salom, Jeyson Jeyson Jeyson
```

Almashtirishlar. Formatlangan qiymatlarni satrga o'tkazishning yana bir usuli - o'rniga ma'lum qiymatlar qo'shiladigan almashtirishlar yoki maxsus to'ldiruvchilardan foydalanish. Formatlash uchun quyidagi to'ldiruvchilardan foydalanish mumkin:

s: String tipli ma'lumot kiritish uchun;

d: Double tipli sonlarni kiritish uchun;

f: Float tipli sonlarni kiritish uchun. Bu tur uchun nuqta orqali kasr sonini aniqlash ham mumkin;

%: qiymat 100 ga ko'paytiriladi va foiz belgisini qo'shib qo'yiladi.

To'ldiruvchining umumiy sintaksisi quyidagicha:

```
{:[maxsus to'ldiruvchi] }
```

To'ldiruvchiga qarab qo'shimcha parametrlar qo'shilishi mumkin. Masalan, float raqamlarini formatlash uchun siz quyidagi variantlardan foydalanishingiz mumkin

```
{:[belgilar_soni][vergul][.sonning_verguldan_keyingi_qismi] to'ldiruvchi}
```

Format metodini chaqirganda, qiymatlar unga argumentlar sifatida uzatiladi, ular to'ldiruvchilar o'rniga kiritiladi. String almashtirish ({:s}) ga misol:

```
hi = "Salom sizga {:s}"  
name = "Jeyson"  
formatted_txt = hi.format(name)  
print(formatted_txt)           # Salom sizga Jeyson
```

Format() usuli natijada yangi formatlangan qatorni qaytaradi.

{:d} - butun son formatlashga misol:

```
txt = "{:d} ta simvol"  
number = 12  
formatted_txt = txt.format(number)  
print(formatted_txt)      # 12 ta simvol
```


Agar formatlash kerak bo'lgan raqam 999 dan katta bo'lsa, vergulni minglik ajratuvchi sifatida ishlatmoqchi ekanligimizni to'ldiruvchi ta'rifida belgilash (`{:,d}`) mumkin:

```
txt = "{:,d} belgi"
print(txt.format(300))    # 300 belgi
```

Bundan tashqari, to'ldiruvchilar f-satrlarida ham ishlatilishi mumkin:

```
n = 300
txt = f"{n:,d} belgi"
print(txt)    # 200 belgi
```

Kasr sonlar uchun, sonning kasr qismida qancha belgi ko'rinishini aniqlash mumkin. Buning uchun verguldan keyin ko'rinishi kerak bo'lgan sonlar soni, nuqtadan keyin to'ldiruvchidan oldin kiritiladi. Masalan:

```
num = 115.2548695
print("{:.2f}".format(num))    # 115.25
print("{:.3f}".format(num))    # 115.254
print("{:.4f}".format(num))    # 115.2548
print("{:,.2f}".format(12582.23664))    # 12,582.23
```

Yana bir parametr formatlangan qiymatning minimal kengligini belgilar bilan belgilash imkonini beradi:

```
print("{:10.2f}".format(115.2548695))    #      115.25
print("{:8d}".format(25))                #      25
```

format metodini o'rnini bosuvchi **f** usuli:

```
n1 = 115.2548695
print(f"{n1:10.2f}")    #      115.25
n2 = 26
print(f"{n2:8d}")        #      26
```

Foizlarni ekranga chiqarish uchun "%" kodini ishlatish yaxshiroqdir :

```
num = .123456
print("{:%}".format(num))          # 12.3456000%
print("{:.0%}".format(num))        # 12%
print("{:.1%}".format(num))        # 12.3%

print(f"{num:%}")                  # 12.3456000%
print(f"{num:.0%}")                # 12%
print(f"{num:.1%}")                # 12.3%
```

Format metodini ishlatmasdan formatlash. Quyidagi sintaksis bilan formatlashning yana bir usuli ham mavjud:

```
string%(param1, param2, ..paramN)
```

Ya'ni, boshida yuqorida ko'rib chiqilgan bir xil to'ldiruvchilarni o'z ichiga olgan qator mavjud (% to'ldiruvchidan tashqari) satrdan keyin foiz - % belgisi qo'yiladi va keyin qatorga kiritiladigan qiymatlar ro'yxati keltiriladi. Aslida, foiz belgisi yangi qatorga olib keladigan operatsiyani anglatadi:

```
info = " Ism : %s \t Yoshi : %d" % ("Jeck", 32)
print(info)      # Ism : Jeck      Yoshi : 32
```

To'ldiruvchidan keyin foiz belgisi qo'yiladi va format funksiyasidan farqli o'laroq, bu yerda jingalak qavslar kerak emas.

Bundan tashqari, bu yerda raqamlarni formatlash usullari ham qo'llaniladi:

```
raqam = 23.8689578
print("%0.2f    - %e" % (raqam, raqam))
# 23.87    - 2.386896 e +01
```

Nazorat savollari:

1. Format metodining qo'llanilishi qanday?

2. Format metodining “Nomlangan parametrlar” usulini tushuntirib bering.
3. Format metodining “Pozitsiya bo‘yicha parametrlar” usulini tushuntirib bering
4. Almashtirishlar – bu qanday vazifani bajaradi?
5. Format metodisiz formatlash qanday amalga oshiriladi?

IV BOB. RO‘YXAT, KORTEJ VA LUG‘ATLAR BILAN ISHLASH

4.1.8. Pythonda ro‘yxat tushunchasi va u bilan ishlovchi metodlar bilan tanishish

Ma’lumotlar to‘plamlari bilan ishlash uchun Python ro‘yxat, kortej va lug‘atlar kabi o‘rnatilgan turlarni taqdim etadi.

Ro‘yxat (list) - elementlar to‘plami yoki ketma-ketligini saqlaydigan ma’lumotlar turi. Ko‘pgina dasturlash tillarida massiv deb ataladigan o‘xshash ma’lumotlar tuzilmasi mavjud.

Ro‘yxat yaratish

Ro‘yxatni yaratish uchun kvadrat qavslar `[]` ishlatiladi. Ularning ichida ro‘yxat elementlari vergul bilan ajratilgan holda saqlanadi. Masalan, raqamlar ro‘yxatini yarataylik:

```
raqamlar = [0, 2, 12, 44, 51]
```

Xuddi shunday, boshqa turdagi ma’lumotlar bilan ro‘yxatlarni belgilash mumkin, masalan, satrlar (ya’ni string turdagi ma’lumotlar) ro‘yxatini yaratish quyidagicha amalga oshiriladi:

```
odamlar = ["Jasur", "Akmal", "Bobur"]
```

Ro‘yxat yaratish uchun **list()** konstruktor funksiyasidan ham foydalanish mumkin :

```
raqamlar1 = []  
raqamlar2 = list()
```

Ushbu ikkala ro‘yxat ta’riflari o‘xshash - ular bo‘sh ro‘yxat yaratadi.

Ro‘yxatda faqat bir xil turdagi ob’ektlar bo‘lishi shart emas. Bir vaqtning o‘zida satrlarni, raqamlarni va boshqa ma’lumotlar turlarining ob’ektlarini bir xil ro‘yxatga joylashtirish mumkin:

```
obyektlar = [1, 2.26, "Salom dunyo", True, False]
```

Ro'yxat elementlarini ekranga chiqarish uchun standart print funksiyasidan foydalanish mumkin. Bu ro'yxat tarkibini to'liq shaklda konsolga chop etadi:

```
raqamlar = [1, 2, 3, 4, 5]
odamlar = ["Jasur", "Akmal", "Bobur"]

print(raqamlar)    # [1, 2, 3, 4, 5]
print(odamlar)     # ["Jasur", "Akmal", "Bobur"]
```

list() konstruktori qiymatlar to'plamini qabul qilishi mumkin, ular asosida ro'yxat tuziladi:

```
raqamlar1 = [1, 2, 3, 4, 5]
raqamlar2 = list(raqamlar1)
print(raqamlar2)    # [1, 2, 3, 4, 5]

harflar = list("Salom")
print(harflar)      # ['S', 'a', 'l', 'o', 'm']
```

Agar bir xil qiymat bir necha marta takrorlanadigan ro'yxatni yaratish kerak bo'lsa, unda yulduzcha - * belgisidan foydalanish mumkin, ya'ni aslida mavjud ro'yxatga ko'paytirish amalini qo'llaniladi:

```
raqamlar = [15]*6      # 15 marta 6 takrorlash
print(raqamlar)        # [15, 15, 15, 15, 15, 15]

odamlar = ["Jasur"]*3  # "Tom" ni 3 marta takrorlash
print(odamlar)         # ["Jasur", "Jasur", "Jasur"]

talabalar = ["Bobur", "Saman"] * 2
# "Bobur", "Saman" ni 2 marta takrorlash

print (talabalar)
# ["Bobur", "Saman", "Bobur", "Saman"]
```

Ro'yxat elementlariga murojaat qilish. Ro'yxat elementlariga murojaat qilish uchun ro'yxatdagi element sonini ifodalovchi indekslardan foydalanish kerak. Indekslar noldan boshlanadi. Ya'ni, birinchi element indeksi 0, ikkinchi element indeksi 1 ga teng bo'ladi va hokazo. Elementlarga teskari tartibda ya'ni oxiridan boshiga qarab murojaat qilish uchun manfiy indeks tushunchasi kiritilgan. Bu manfiy indekslar -1 dan boshlanadi. Ya'ni, oxirgi element -1 indeksiga ega bo'ladi, oxirigidan bitta oldingisi esa -2 indeksiga ega bo'ladi va hokazo.

```
odamlar = ["Jasur", "Samat", "Bobur"]
# ro'yxat boshidan elementlarni olish

print(odamlar[0])      # Jasur
print(odamlar[1])      # Samat
print(odamlar[2])      # Bobur

# ro'yxat oxiridagi elementlarni olish
print(odamlar[-2])     # Samat
print(odamlar[-1])     # Bobur
print(odamlar[-3])     # Jasur
```

Ro'yxat elementini o'zgartirish uchun unga yangi qiymat yuklash kifoya:

```
odamlar = ["Jasur", "Samat", "Bobur"]

odamlar[1] = "Akmal"    # ikkinchi elementni o'zgartirish
print(odamlar[1])      # Akmal
print(odamlar)          # ["Jasur", " Akmal", "Bobur"]
```

Ro'yxat elementlarini ajratish. Python sizga ro'yxatni alohida elementlarga ajratish imkonini beradi:

```
odamlar = ["Jasur", "Bobur", "Samat"]

jasur, bobur, samat = odamlar

print (jasur)          # Jasur
print (bobur)          # Bobur
print (samat)          # Samat
```

Bunday holda, **jasur**, **bobur** va **samat** o'zgaruvchilari odamlar ro'yxatidan ketma-ket tayinlangan elementlardir. Biroq, o'zgaruvchilar soni tayinlangan ro'yxat elementlari soniga teng bo'lishi talab qilinadi. Aks xolda xatolik kelib chiqadi.

Ro'yxat elementlarini olish uchun **for** yoki **while** tsiklidan foydalanish mumkin.

For tsikli bilan takrorlash:

```
odamlar = ["Tomson", "Sems", "Bob"]
for odam in odamlar:
    print (odam)
```

Bu yerda odamlar ro'yxati takrorlanadi va har bir element shaxs o'zgaruvchisiga joylashtiriladi.

Iteratsiyani **while** tsikli bilan ham bajarish mumkin:

```
odamlar = ["Tomson", "Sems", "Bob"]
i = 0
while i < len(odamlar):
    print(odamlar[i])
    # elementni olish uchun indeksdan foydalanish
    i += 1
```

len() funksiyasidan foydalanib takrorlash uchun ro'yxat uzunligini olinadi. Bu dasturda **i** - hisoblagich qiymati ro'yxat uzunligiga teng bo'lguncha chop etadi. **i** ning qiymati esa har safar birga oshirilib boraveradi.

Ro'yxatni taqqoslash. Ikki ro'yxat, agar ular bir xil elementlar to'plamini o'z ichiga olsa, teng hisoblanadi:

```
raqamlar1 = [1, 2, 3, 4, 5]
raqamlar2 = list ([1, 2, 3, 4, 5])

if raqamlar1 == raqamlar2:
    print("1 - raqamlar 2 - raqamlarga teng")
else:
    print("1-raqamlar 2-raqamlarga teng emas")
```

Bunday holda, ikkala ro'yxat ham teng bo'ladi.

Ro'yxatlar bilan ishlovchi metodlar va funksiyalar

Ro'yxat elementlarini boshqarish uchun bir qancha metodlar mavjud. Ulardan asosan ko'p marta murojaat qilinadiganlarini quyida keltirilgan:

append(element): **element** - ni ro'yxat oxiriga qo'shish;

insert (indeks, element): ro'yxatga **element**ni **indeks** – element qilib qo'shish;

extend (items): ro'yxat oxiriga **items** to'plamini qo'shish;

remove(item): ro'yxatdan **item** elementini olib tashlaydi. Ro'yxatdan birinchi uchragan **item** elementi o'chiriladi. Agar element topilmasa, ValueError xatoligi qaytariladi;

clear(): ro'yxatdagi barcha elementlarni o'chirish;

index(item): **item** elementining indeksini qaytaradi. Agar element topilmasa, ValueError xatoligi qaytariladi;

pop([indeks]): **indeks** indeksdagi elementni olib tashlaydi va qaytaradi. Agar indeks ko'rsatilmay faqat pop() holida foydalanilsa, oxirgi elementni olib tashlanadi.

count(item): ro'yxatdagi **item** elementining takrorlanishlar sonini qaytaradi. Agar mavjud bo'lmas, 0 qaytaradi;

sort([key]): ro'yxat elementlarini tartiblaydi. Standart bo'yicha o'sish tartibida saralanadi. Lekin **key** parametr yordamida saralashni qanday tartibda bo'lishini boshqarish mumkin.

reverse(): Ro'yxatdagi barcha elementlarni teskari tartibda qayta joylashtiradi.

copy(): ro'yxatni nusxalaydi.

Bundan tashqari, Python ro'yxatlar bilan ishlash uchun bir qator standart funktsiyalarni taqdim etadi:

len(list): ro'yxat uzunligini aniqlash;

sorted(list, [key]): tartiblangan ro'yxatni qaytaradi;

min(list): ro'yxatning eng kichik elementini topish;

max(ro'yxat): ro'yxatdagi eng katta elementni topish.

Ro'yxatga element qo'shish uchun `append()` , `extension` va `insert` metodlari, ro'yxatdan elementni olib tashlash uchun **`remove()`, `pop()` va `clear()`** usullari qo'llaniladi.

Foydalanish usullari:

```
list1= ["Timur", "Bobur"]

# ro'yxat oxiriga qo'shish
list1.append("Elmurod")  #["Timur", "Bobur", "Elmurod"]

# ikkinchi pozitsiyaga qo'shish
list1.insert(1, "Bilol") #["Timur", " Bilol", "Bobur",
"Elmurod"]

# elementlar to'plamini qo'shish ["Malik", "Samat"]
list1.extend(["Malik", "Samat"])
# ["Timur", " Bilol", "Bobur", "Elmurod", "Malik", "Samat"]

# element indeksini olish
index_of_tom = list1.index("Bobur")  # 2

# ushbu indeksda o'chirish
removed_item = list1.pop(index_of_tom)
```

```

# ["Timur", " Bilol", "Elmurod", "Malik", "Samat"]

# oxirgi elementni olib tashlang
oxirgi_element = list1.pop() # "Samat"
print(list1) # ["Timur", " Bilol", "Elmurod", "Malik"]

# "Elmurod" elementini olib tashlash
list1.remove("Elmurod")
print(list1) # ["Timur", " Bilol", "Malik"]

# barcha elementlarni o'chirish
list1.clear()
print (list1) # []

```

IN kalit so'zining ishlatirishi

Agar biron bir element topilmasa, olib tashlash va indekslash usullari xatolik qaytaradi. Bunday vaziyatni oldini olish uchun elementni ishlatishdan oldin **in** kalit so'zidan foydalanib uning mavjudligini tekshirish lozim:

```

list_odamlar = ["Timur", "Bobur", "Samat"]
if "Ali" in list_odamlar:
    list_odamlar.remove("Ali")

print(list_odamlar) # ["Timur", "Bobur", "Samat"]

```

Odamlar ro'yxatida "Ali" elementi bo'lsa, **if "Ali" in list_odamlar** ifodasi "True" qiymati qaytaradi. Shunday ekan bu vaziyatda False qiymati qaytadi va **list_odamlar.remove("Ali")** funksiyasi bajarilmaydi va **list_odamlar** nomli ro'yxat tarkibi o'zgarishsiz qoladi.

DEL metodi

Pythonda ro'yxatdan elementni o'chirish uchun yana bir **del** nomli operatoridan ham foydalaniladi. O'chiriladigan element yoki elementlar to'plami ushbu operatorga parametr sifatida uzatiladi:

```
list_odamlar = ["Timur", " Bilol", "Bobur",  
"Elmurod", "Malik", "Samat"]  
  
# ikkinchi elementni olib tashlash  
del list_odamlar [1]  
print (list_odamlar) # ["Timur", "Bobur", "Elmurod",  
"Malik", "Samat"]  
  
# to'rtinchi elementgacha o'chirish  
list_odamlar [:3]  
print(list_odamlar) #["Elmurod", "Malik", "Samat"]  
  
del list_odamlar[1:]#ikkinchi elementdan oxirigacha  
print(list_odamlar) #["Timur"]
```

COUNT() metodi

Agar element ro'yxatda necha marta borligini bilish kerak bo'lsa, count() metodidan foydalanish maqsadga muvofiq bo'ladi:

```
odamlar = ["Bobur", " Bilol", "Bobur", "Elmurod",  
"Malik", "Bobur"]  
  
odamlar_soni = odamlar.count("Bobur")  
print(odamlar_soni)          # 3
```

SORT() metodi

Sort() metodi standart holatda o'sish tartibida saralash uchun ishlatiladi:

```
odamlar = ["Timur", "Bilol", "Bobur", "Elmurod",  
"Malik", "Samat"]  
odamlar.sort()  
print(odamlar)  
  
#['Bilol', 'Bobur', 'Elmurod', 'Malik', 'Samat', 'Timur']
```

Agar ma'lumotlarni teskari tartibda saralash zarur bo'lsa, **sort()** metodidan keyin **reverse()** usulini qo'llash mumkin:

```
odamlar = ["Timur", "Bilol", "Bobur", "Elmurod",  
"Malik", "Samat"]  
  
odamlar.sort()  
odamlar.reverse()  
print(odamlar)  
  
# ['Timur', 'Samat', 'Malik', 'Elmurod', 'Bobur', 'Bilol']
```

Saralash aslida ikkita ob'ektni taqqoslaydi va qaysi biri "kichik" bo'lsa, "katta" ob'ektdan oldin joylashtiriladi. "katta" va "kichik" tushunchalari albatta nisbiydir. Va agar ro'yxatning barcha elementlari raqamlardan tarkib topgan bo'lsa, unda barchasi oddiy - raqamlar o'sish tartibida joylashtiriladi. Agar ro'yxatda satrlar va boshqa ob'ektlar mavjud bo'lsa va saralash kerak bo'lsa, unda vaziyat yanada murakkabroq bo'ladi. Bunday holda harflar sonlardan oldin joylashtiriladi. Albatta sonli satrlar haqida gap ketmoqda. Chunki butun tipli ma'lumot bilan string tipli ma'lumot solishtirish xatolik keltirib chiqaradi. Demak, string tiplarni solishtirishda, agar ularning birinchi belgilar teng bo'lsa, ikkinchi belgilar solishtiriladi va hokazo. Bunda raqamli belgi alifbodagi bosh harfdan "kichik" deb hisoblanadi va o'z navbatida bosh harf ham kichik harfdan kichik hisoblanadi.

Shunday qilib, agar ro'yxat tarkibida katta va kichik harflar mavjud bo'lsa va saralash amali bajarilsa, unda unchalik to'g'ri bo'lmagan natijalarni olinishi mumkin, chunki asosan "bob" matni "Tom" matnidan oldin kelishi kerak. Bunday holda sort metodining standart tartiblashini o'zgartirish uchun unga parametr sifatida funktsiyani beriladi:

```
odamlar = ["Timur", "bilol", "Bobur", "elmurod",  
"Malik", "Samat"]  
  
odamlar.sort()      # standart tartiblash  
print(odamlar)      # ['Bobur', 'Malik', 'Samat', 'Timur',  
'bilol', 'elmurod']  
  
odamlar.sort(key=str.lower)  
# kichik katta-kichik tartiblash  
  
print ( odamlar )  
# ['bilol', 'Bobur', 'elmurod', 'Malik', 'Samat', 'Timur']
```

SORTED metodi

Saralash usuliga qo'shimcha ravishda ikkita shaklga ega bo'lgan tartiblangan standart funksiyadan foydalanish mumkin:

sorted(list): list ro'yxatini tartiblaydi

sorted(list, key): list ro'yxati elementlariga **key** funksiyasini qo'llash orqali ro'yxatni tartiblaydi

```
list_per = ["Tom", "bob", "alis", "Sem", "Bill"]  
  
sorted_per = sorted(list_per, key=str.lower)  
print (sorted_per)  
# ["alisa", "Bill", "bob", "Sem", "Tom"]
```

Ushbu funktsiyadan foydalanganda shuni yodda tutish kerakki, bu funksiya tartiblangan ro'yxatni o'zgartirmaydi, ammo u barcha tartiblangan elementlarni yangi ro'yxatga joylashtiradi va natijani qaytaradi.

MIN va MAX funksiyalari

Pythonning **min()** va **max()** funksiyalari mos ravishda minimum va maksimum qiymatlarni topishga imkon beradi:

```
raqamlar = [9, 221, 12, 10, 3, 125, 18]

print (min(raqamlar))      # 3
print (max(raqamlar))      # 221
```

COPY() funksiyasi

Ro'yxatlarni nusxalashda, ro'yxatlar o'zgaruvchan tur ekanligini yodda tutish kerak. Shuning uchun ikkala o'zgaruvchi ham bir xil ro'yxatga ishora qilsa, bitta o'zgaruvchini o'zgartirish boshqa o'zgaruvchiga ta'sir qiladi:

```
list_per = ["Timur", "Bobur", "Alisey"]
list_per2 = list_per
list_per2.append("Samat") # elementni ikkinchi ro'yxatga
qo'shish

# kishi1 va odamlar2 bir xil ro'yxatga ishora qiladi
print(list_per)      # ["Timur", "Bobur", "Alisey", " Samat"]
print(list_per2)      # ["Timur", "Bobur", "Alisey", "
Samat"]
```

Bu yuqorida keltirilgan dasturdagi bir o'zgaruvchining qiymatini ikkinchisiga yuklab qo'yish holati nusxalash deb atalmaydi. Nusxalash maqsadida bunday hatti harakatni qilmagan ma'qul. Elementlarni nusxalash uchun chuqurroq nusxalash talab qilinadi. Shundagina bir vaqtning nusxalangan va aslnusxadagi ro'yxatlar qiymatlari bir xil bo'lsada, aslida turli xil ro'yxatlarni ifodalaydi. Buning uchun oddiygina **copy()** metodidan foydalanib nusxalash kerak bo'ladi:

```
list_per = ["Tom", "Bob", "Alis"]
list_per2 = list_per.copy() # elementlarni list_per
dan list_per2 ga nusxalash

# elementni FAQAT ikkinchi ro'yxatga qo'shish
list_per2.append("Sem")

# list_per va list_per2 turli ro'yxatlarni bildiradi
print(list_per)      # ["Tom", "Bob", "Alis"]
print(list_per2)     # ["Tom", "Bob", "Alis", "Sem"]
```

Ro'yxatning bir qismini nusxalash

Agar butun ro'yxatni emas, balki uning ma'lum bir qismini nusxalash kerak bo'lsa, unda quyidagi shakllarni qabul qilishi mumkin bo'lgan maxsus sintaksisdan foydalanish kerak bo'ladi:

list[:end]: ro'yxat boshidan ***end*** elementigacha bo'lgan oraliqni tanlash.

list[start:end]: ro'yxatning ***start*** va ***end*** elementlari oraliq'ini belgilaydi

list[start:end:step]: ro'yxatning ***start*** va ***end*** elementlari oraliq'ini ***step*** qadam bilan belgilaydi. Odatiy bo'lib, bu ***step*** parametr 1 ga teng bo'ladi.

```
list_per = ["Tomson", "Bobi", "Elison", "Semmi",
"Timati", "Billi"]

slice_people1 = list_per [:3] # 0 dan 3 gacha
print(slice_people1)      # ['Tomson', 'Bobi', 'Elison']

slice_people2 = list_per [1:3] #1 dan 3 gacha
print(slice_people2)      # ["Bobi", "Elison"]

slice_people3 = list_per[1:6:2] #1 dan 6 gacha 2 ga oshib
print(slice_people3)      #["Bobi", "Semmi", "Billi"]
```

Manfiy indeks

Ro'yxatning oxirgi elementidan boshlab murojaatni boshlash kerak bo'lganda manfiy indeksdan foydalanish kerak bo'ladi. Masalan, -1 – element oxirgi element, -2 – element oxiridan bitta oldingi va hokazo.

```
list_per = ["Tomson", "Bobi", "Elison", "Semmi",
"Timati", "Billi"]

# nolinchidan oxirgisigacha, oxirgi xisobga kirmaydi
slice_people1 = list_per[:-1]
print(slice_people1)

# ['Tomson', 'Bobi', 'Elison', 'Semmi', 'Timati']

# oxirgi uchinchi elementdan oxirgisigacha, oxirgisini olmaydi
slice_people2 = list_per [-3:-1]
print(slice_people2)      # ["Semmi", "Timati"]
```

Ro'yxatlarni ulash

Qo'shish (+) operatori ro'yxatlarni birlashtirish uchun ishlatiladi:

```
people1 = ["Tomson", "Bobi", "Alison"]
people2 = ["Tomson", "Semmi", "Timati", "Billi"]
people3 = people1 + people2
print (people3)

# ["Tomson", "Bobi", "Elison", "Tomson", "Semmi", "Timati",
"Billi"]
```

Ichma-ich Ro'yxatlar

Ro'yxatlar satrlar, raqamlar kabi standart ma'lumotlardan tashqari, boshqa ro'yxatlarni ham o'z ichiga olishi mumkin. Bunday ro'yxatlarni jadvallar bilan bog'lash mumkin, bu yerda ichki ro'yxatlar qatorlar vazifasini bajaradi. Masalan:

```
odamlar = [
    ["Tomson", 29],
    ["Elison", 33],
```



```

        ["Bobi", 27]
    ]

    print(odamlar[0])           # ["Tomson", 29]
    print(odamlar[0][0])       # "Tomson"
    print(odamlar[0][1])       # 29

```

Ichki ro'yxatning elementiga murojaat qilish uchun bir juft indeksdan foydalanish kerak: `odamlar[0][1]` - birinchi joylashtirilgan ro'yxatning ikkinchi elementiga murojaat.

Umumiy ro'yxatni, shuningdek, ichki ro'yxatlarni qo'shish, o'chirish va o'zgartirish oddiy (bir o'lchovli) ro'yxatlar kabi amalga oshiriladi:

```

odamlar = [
    ["Tomson", 29],
    ["Elison", 33],
    ["Bobi", 27]
]

# ichki ro'yxat yaratish
odam = list()
odam.append("Billi")
odam.append(41)

# ichki ro'yxat qo'shish
odamlar.append(odam)
print(odamlar[-1])    # ["Billi", 41]

# ichki ro'yxatga qo'shish
odamlar[-1].append("+79996543210")

```

```

print(odamlar[-1]) # ["Billi", 41, "+79996543210"]

# oxirgi elementni o'rnatilgan ro'yxatdan olib tashlash
odamlar[-1].pop()
print(odamlar[-1]) # ["Billi", 41]

# oxirgi kiritilgan ro'yxatni butunlay o'chirish
odamlar.pop(-1)

# birinchi elementni o'zgartirish
odamlar[0] = ["Semmi", 18]
print (odamlar)
# [ ["Semmi", 18], ["Alison", 33], ["Bobi", 27]]

```

Ichki ro'yxatlar ustida takrorlash:

```

odamlar = [
    ["Tomson", 29],
    ["Elison", 33],
    ["Bobi", 27]
]
for odam in odamlar:
    for item in odam:
        print(item, end=" | ")

```

Konsol chiqishi :

```
Tomson | 29 | Elison | 33 | Bobi | 27 |
```

Nazorat savollari:

1. Ro'yxat deb nimaga aytiladi?
2. Ro'yxatlarni qanday yaratiladi?

3. Ro'yxatlarning elementlari bir turga mansub bo'lishi shartmi?
4. ro'yxat elementlariga murojaatlar qanday amalga oshiriladi?
5. Ro'yxat uzunligini qanday aniqlanadi?
6. Ro'yxat elementini biro songa ko'paytirish nimani beradi?
7. Manfiy index tushunchasi nimani anglatadi?
8. `tomson`, `bobi`, `semmi` = `odamlar` – ushbu ifodaning ma'nosi nima?
9. Ro'yxatga ma'lumotni sikl orqali kiritish qanday amalga oshiriladi?
10. Ro'yxat elementlarini saralash uchun qaysi metodni ishlatiladi?
11. Ro'yxatga ma'lumot qo'shish uchun qaysi metodni ishlatiladi?
12. `remove()` , `pop()` va `clear()` matodlari qo'llanilishi haqida gapirib bering.
13. `if "Alison" in people` - ifodasining manisini tushuntirib bering.
14. `del` – metodi qanday ishlatiladi?
15. `count()` metodining vazifasi qanday?
16. `reverse()` metodini nima maqsadda foydalaniladi va uning parametrlari qanday?
17. Ro'yxatlarni nusxalash qanday amalga oshiriladi?

4.2.§. Pythonda kortej tushunchasi va u bilan ishlovchi metodlar bilan tanishish

Kortej xuddi ro'yxat kabi elementlar ketma-ketligini ifodalaydi. Bundan tashqari kortej o'zgarmas tur hisoblanadi. Shuning uchun kortejdagi elementlarni qo'shish, olib tashlash yoki uni o'zgartirishning ilojisi yo'q.

Tuple yaratish uchun turli qiymatlarni vergul bilan ajratib oddiy qavslar ichiga joylab qo'yish kifoya:

```
tommi= ("Tomson",23)
print (tommi)          # ("Tomson", 23)
```

Shuningdek, kortejni yaratish uchun qavslarsiz vergul bilan ajratilgan qiymatlarni ro'yxatga olish kerak bo'ladi:

```
tommi = "Tomson", 23
print (tommi)          # ("Tomson", 23)
```

Agar mabodo kortej bitta elementdan iborat bo'lib qolsa, u holda kortejning yagona elementidan keyin vergul qo'yish kerak:

```
tommi = ("Tomson")
```

Ro'yxat kabi boshqa elementlar to'plamidan kortej yaratish uchun ro'yxatni **tuple()** funksiyasiga uzatiladi, bu esa ro'yxatni kortejni aylantiradi va kortej qaytaradi:

```
data = ["Tomson", 37, "Google"]
tommi = tuple (data)
print(tommi)          # ("Tomson", 37, "Google")
```

LEN() funksiyasi

Pythonning standart len() funktsiyasidan foydalanib, kortej uzunligini aniqlash mumkin:

```
tommi = ("Tomson", 37, "Google")
print (len(tommi))     # 3
```

Kortej elementlariga murojaat

Kortejdagi elementlarga murojaat ro'yxatdagi kabi indeks bo'yicha amalga oshiriladi. Indeksflash, shuningdek, ro'yxat boshidan elementlarni olishda noldan va ro'yxat oxiridagi elementlarni olishda -1 dan boshlanadi:

```
tommi = ("Tomson", 37, "Google", "software  
engineereeng")  
print(tommi[0])          # "Tomson"  
print(tommi[1])          # 37  
print(tommi[-1])         # "software engineereeng"
```

Lekin kortej o'zgaras tur bo'lgani uchun uning elementlarini o'zgartirishning ilojisi yo'q. Ya'ni, quyidagi yozuv ishlamaydi:

```
tommi[1] = "Tomsonjon"
```

Agar zarurat tug'ilsa, kortejni alohida o'zgaruvchilarga ajratish mumkin:

```
name, age, company, position = ("Tomson", 35,  
"Google", "software engineereeng")  
print(name)              # Tomson  
print(age)               # 35  
print(position)          # software engineereeng  
print(company)           # Google
```

Subtuplarni olish

Ro'yxatlarda bo'lgani kabi, kortejning bir qismini boshqa kortej sifatida olish mumkin:

```
tommi = ("Tomson", 35, "Google", "software engineereeng")  
  
# 1 dan 3 gacha bo'lgan elementni olish, 3-element xisoblanmaydi  
print(tommi[1:3])        # (35, "Google")
```

```
# 0 dan 3 gacha bo'lgan elementni olish, 3-element xisoblanmaydi
print(tommi[:3])      # ("Tomson", 35, "Google")

# 1 dan oxirgi elementgacha subtuple olish
print (tommi[1:])
# (35, "Google", "software enginireeng")
```

Kortejni funksiyaga parametr sifati berish va natijasi sifatida qabul qilish

Funktsiyadan bir vaqtning o'zida bir nechta qiymatlarni qaytarish kerak bo'lganda, kortejlardan foydalanish qulaydir. Funktsiya bir nechta qiymatlarni qaytarsa, u aslida kortejni qaytaradi:

```
def get_user():
    name="Tomson"
    yosh = 25
    kompaniya = "Google"
    return name, yosh, kompaniya

foydalanuvchi = get_user()
print(foydalanuvchi)      # ("Tomson", 25, "Google")
```

* operatori yordamida funksiyaga kortejni uzatishda uni funktsiya parametrlariga o'tkaziladigan individual qiymatlarga ajratish mumkin:

```
def print_person(ism, yosh, kompaniya):
    print(f"Ism:{ism}Yosh:{yosh} Kompaniya: {kompaniya}")

tommi = ("Tomson", 25)
print_person(*tommi, "Microsoft")
# Ism: Tomson   Yosh: 25   Kompaniya: Microsoft

bob = ("Bobi", 41, "Apple")
print_person(*bob)
# Ism: Bobi    Yoshi: 41   Kompaniya: Apple
```

Kortejda sikldan foydalanish

To'plam elementlariga murojaat qilishda ularning har bir elementiga alohida murojaat qilish, o'ta noto'g'ri bo'ladi. Buning uchun standart for va while sikllaridan foydalaniladi. **for** tsikli orqali kortej elementlarini olish quyidagicha amalga oshiriladi:

```
tom = ("Tomson", 22, "Google")
for element in tom:
    print (element)
```

While siklidan foydalanib kortej elementlariga murojaat qilish:

```
tom = ("Tomson", 22, "Google")

i = 0
while i <len(tom):
    print (tom[i])
    i += 1
```

Kortej tarkibida element mavjudligini tekshirish

Ba'zi masalalarda elementga murojaat qilish kerak bo'ladi va murojlat natijasi xatolik bilan tugashi mumkin. Bunday hollarda elementni mavjudligini birinchi tekshirib olish kerak bo'ladi va keyin biror amal bajarilsa, maqsadga muvofiq bo'ladi. Ushbu murojaat amalga oshiriladigan elementni mavjudligini tekshirish uchun **in** operatoridan foydalaniladi va u quyidagicha foydalaniladi:

```
foydalanuvchi = ("Tomson", 22, "Google")
nomi="Tomson"
if nomi in foydalanuvchi:
    print("Foydalanuvchi nomi Tomson")
else:
    print("Foydalanuvchining nomi boshqacha ekan!")
```

Nazorat savollari:

1. Kortej deb nimaga aytiladi?
2. Kortejlarning ro'yxatlardan farqi nimada?
3. Kortejlarning yaratilishi usullari qanday?
4. Kortejdagi ma'lumotlarga qanday murojaat qilinadi?
5. Kortejdagi ma'lumotlarni qanday qilib o'zgartirish mumkin?
6. Parametr va funksiya natijasi sifatida kortejni qanday qo'llash mumkin?
7. Kortej elementlarini tsikl orqali chiqarish qanday amalga oshiriladi?
8. Kortejni parametr sifatida uzatishda "*" (yulduzcha) belgisining ro'li qanday?

4.3.§. Pythonda lug'at tushunchasi va u bilan ishlovchi metodlar bilan tanishish

Pythondagi lug'at (dictionary) elementlar to'plamini saqlaydi. Lug'at tarkibidagi har bir element o'ziga xos, yagona kalitga va u bilan bog'liq bo'lgan qiymatga ega bo'ladi.

Lug'at ta'rifi quyidagi sintaksisga ega:

```
dictionary={kalit1: qiymat1, kalit2: qiymat2, ....}
```

Lug'atning barcha elementlari jingalak qavslar ichiga joylanadi. Har bir element bir-biri bilan vergul orqali ajratildi. Har bir elementning yagona kaliti va unga bog'langan qiymat mavjud bo'ladi. Elementning kaliti birinchi keladi va undan keyin qiymat keladi. Kalit va qiymatini ikki nuqta ajratib turadi.

Lug'atni aniqlash:

```
users = {1: "Tomson", 2: "Bobi", 3: "Billi"}
```

`users` lug'ati raqamlarni kalit sifatida va satrlarni qiymat sifatida ishlatadi. Ya'ni, 1-kalitga ega bo'lgan element "Tomson" qiymatiga teng, 2-kalitga ega element "Bobi" qiymatiga ega va hokazo.

Lug'atdagi elementlar tarkibi bir xil tipli ma'lumotlar bo'lishi ham mumkin. Masalan:


```
e_emails = {"tomson@gmail.com": "Tomson",  
"bobi@gmail.com": "Bobi", "semmi@gmail.com": "Semmi"}
```

`e_emails` lugʻatida kalit sifatida satrlardan foydalanildi - foydalanuvchi elektron pochta manzillari va qiymat sifatida ham satrlardan - foydalanuvchi nomlaridan foydalanildi.

Lekin kalitlar va satrlar bir xil turdagi boʻlishi shart emas. Ular turli xil turlarni koʻrsatishi mumkin:

```
objects = {1: "Tomson", "2": False, 3: 150.64}
```

Bundan tashqari, umuman elementsiz, boʻsh lugʻatni aniqlash ham mumkin:

```
dict_object = {}
```

yoki shunga oʻxshash:

```
dict_object = dict()
```

Roʻyxatlar va kortejlarni lugʻatga aylantirish

Lugʻat va roʻyxat tuzilmaviy jihatdan bir-biriga oʻxshamaydigan turlar boʻlsa-da, standart **dict()** funksiyasi yordamida maʼlum turdagi roʻyxatlarni lugʻatga aylantirish mumkin. Buning uchun roʻyxat ichki roʻyxatlar toʻplamini saqlashi kerak. Har bir ichki roʻyxat ikkita elementdan iborat boʻlishi kerak - lugʻatga aylantirilganda birinchi element kalitga, ikkinchisi esa qiymatga aylanadi:

```
user_list=[  
    ["+554513455", "Tomson"],  
    ["+002549557", "Bobi"],  
    ["+445454512", "Elison"]  
]  
user_dict = dict(user_list)  
print(user_dict)    # {"+554513455", "Tomson:", "+002549557",  
# "Bobi", "+445454512": "Elison"}
```

Xuddi shunday, ikkita kortej lugʻatga aylantirilishi mumkin, oʻz navbatida lugʻat ikkita kortejni oʻz ichiga oladi:

```
user_tuple = (  
    ("+554513455", "Tomson"),  
    ("+002549557", "Bobi"),  
    ("+445454512", "Elison")  
)  
  
user_dict = dict (user_tuple)  
print (user_dict)
```

Lugʻat elementlariga murojaat qilish va oʻzgartirish

Lugʻat elementlariga murojaat qilish uchun uning nomidan keyin element kaliti kvadrat qavs ichida koʻrsatiladi:

```
Dictionary_name[key]
```

Lugʻatdagi elementlarni olish va oʻzgartirish uchun:

```
dict_users = {  
    "+11002255": "Tomson",  
    "+15245999": "Bobi",  
    "+15415154": "Elison"  
}  
  
# "+11002255" kaliti bilan elementni olish  
print (dict_users["+11002255"])           # Tomson  
  
# element qiymatini "+15245999" tugmasi bilan belgilash  
dict_users["+15245999"] = "Bob Smit"  
print (dict_users["+15245999"])          # "Bob Smit"
```

Agar bunday kalit bilan elementning qiymatini belgilashda lugʻatda hech qanday element boʻlmasa, u qoʻshiladi:

```
dict_users["+5555555"] = "Semmi"
```

Ammo agar lugʻatda mavjud boʻlmagan kalit bilan qiymat olishga harakat qilinsa, Python **KeyError** xatoligini chiqaradi:

```
foydalanuvchi=foydalanuvchilar["+4444444"] # KeyError
```

Bunday holatni oldini olish maqsadida elementga murojaat qilishdan oldin **"key in dict"** ifodasi yordamida lugʻatda element mavjudligini tekshirish mumkin. Agar kalit lugʻatda boʻlsa, u holda bu ifoda **"True"** ni qaytaradi:

```
key = "+00000044"
if key in dict_users:
    dict_user = dict_users[key]
    print(dict_user)
else:
    print ("Element topilmadi")
```

Shuningdek lugʻat elementlariga murojaat qilish uchun **get** metodidan ham foydalaniladi. **get()** metodidan ikki usulda foydalanish mumkin:

get(key): lugʻatdan kalit tugmasi boʻlgan elementni qaytaradi. Agar ushbu kalitga ega boʻlgan element mavjud boʻlmasa, u holda **"False"** qiymati qaytariladi;

get(key, default): lugʻatdan key kaliti boʻlgan elementni qaytaradi. Agar bunday kalitga ega element boʻlmasa, u **default** qiymatni qaytaradi.

```
foydalanuvchilar = {
    "+11002255": "Tomson",
    "+15245999": "Bobi",
    "+15415154": "Elison"
}
```

```

user1 = users.get("+15415154")
print (user1)      # Alison
user2=users.get("+15245999", "Noma'lum foydalanuvchi")
print (user2 )     #Bobi
user3=users.get("+0000000", "Noma'lum foydalanuvchi")
print(user3)       # Noma'lum foydalanuvchi

```

DEL funksiyasi

Del funksiyasi elementni kalit bilan olib tashlash uchun ishlatiladi :

```

foydalanuvchilar = {
    "+11002255": "Tomson",
    "+15245999": "Bobi",
    "+15415154": "Elison"
}

del foydalanuvchilar["+15415154"]
print (foydalanuvchilar)
# { "+11002255": "Tomson", "+15245999": "Bobi"}

```

Ammo shuni yodda tutish kerakki, agar bunday kalit lugʻatda boʻlmasa, `KeyError` xatoligi qaytariladi. Shuning uchun, oʻchirilishi kerak boʻlgan elementni oldin, berilgan kalitli element mavjudligini tekshirish tavsiya etiladi.

```

dict_user = {
    "+11002255": "Tomson",
    "+15245999": "Bobi",
    "+15415154": "Elison"
}

key = "+99990000"
if key in dict_user:
    del dict_user[key]

```

```
print(f"{key} kaliti element olib tashlandi")
else:
    print (f"{key} kalitli element topilmadi")
```

POP() metodi

O‘chirishning yana bir usuli - **pop()** usuli. Pop metodidan ikki xil ko‘rinishda foydalanish mumkin:

- 1) **pop(key):** *key* kalit berilgan elementni olib tashlaydi va olib tashlangan elementni qaytaradi. Agar berilgan kalit bilan element topilmasa, **KeyError** xatoligi qaytariladi.
- 2) **pop(key, default):** *key* kalitli elementni o‘chiradi va olib tashlangan elementni qaytaradi. Agar berilgan kalit bilan element bo‘lmasa, **default** qiymati qaytariladi.

```
foydalanuvchilar = {
    "+11002255": "Tomson",
    "+15245999": "Bobi",
    "+15415154": "Elison"
}
key = "+11002255"
user1 = users.pop (key)
print (user1)          # Tomson

user1 = users.pop("+00000000", "Noma'lum
foydalanuvchi")
print (user1)          # Noma'lum foydalanuvchi
```

Agar barcha elementlarni olib tashlash kerak bo‘lsa, ya’ni bir so‘z bilan aytganda tozalash kerak bo‘lsa, unda **clear()** metodidan foydalanish kerak bo‘ladi:

```
per.clear()
```

Lugʻatlarni nusxalash va birlashtirish

copy() usuli lugʻat tarkibini nusxalab, yangi lugʻatni qaytaradi:

```
foydalanuvchilar={"998888":"Tomson","998771": "Bobi"}
talabalar = users.copy()
print (talabalar) # {"998888": "Tomson", "998771": "Bobi"}
```

update() metodi ikkita lugʻatni birlashtiradi:

```
foydalanuvchilar={"998888":"Tomson","998771": "Bobi"}

users2 = {"222222": "Semmi", "666000": "Kail"}
users.update(users2)

print(users)      # {"998888": "Tomson", "998771": "Bobi",
"222222": "Semmi", "666000": "Kail"}

print(users2)     # {"222222": "Semmi", "666000": "Kail"}
```

Bunday holda, `users2` lugʻati oʻzgarishsiz qoladi. Faqat foydalanuvchilarning lugʻati oʻzgartiriladi, unga boshqa lugʻat elementlari qoʻshiladi. Ammo agar ikkala manba lugʻati ham oʻzgarishsiz qolishi kerak boʻlsa va birlashmaning natijasi uchinchi lugʻat boʻlsa, avval bitta lugʻatni boshqasiga nusxalanishi kerak:

```
users3 = users.copy()
users3.update(users2)
```

Lugʻatni takrorlash

Lugʻat tarkibini sikldan foydalanib olish uchun `for` siklidan foydalanish mumkin:

```
users = {"998888": "Tomson", "998771": "Bobi"}
for a in users:
```

```
print(f"id: {a}    Foydalanuvchi: {users[a]} ")
```

Takrorlash orqali elementlarga murojaat qilishda joriy elementning kalitini elementning o'zini olish uchun ishlatiladi.

Elementlarni olishning yana bir usuli - **items()** metodidan foydalanish:

```
users = {"998888": "Tomson", "998771": "Bobi"}  
for key, val in users.items():  
    print(f"id: {key}    Foydalanuvchi: {val} ")
```

items() usuli kortejlar to'plamini qaytaradi. Har bir kortej elementning kaliti va qiymatini o'z ichiga oladi, ularga murojaat qilishda **key** va **val** o'zgaruvchilarini chaqirilsa yetarli.

Shuningdek, kalitlar ustida takrorlash va qiymatlarni takrorlash uchun alohida usul ham mavjud. Lug'atdagi kalitlarni olish uchun **keys()** metodidan foydalaniladi:

```
for key in dict_users.keys():  
    print(key)
```

Faqat qiymatlarni takrorlash uchun lug'atning **values()** metodidan foydalaniladi:

```
for val in dict_users.values():  
    print(val)
```

Murakkab lug'atlar

Raqamlar va satrlar kabi eng oddiy ob'ektlardan tashqari, lug'atlar murakkabroq ob'ektlarni ham saqlashi mumkin. Masalan bir xil ro'yxatlar, kortejlar yoki boshqa lug'atlarni o'z ichiga element sifatida joylashtirishi mumkin:

```
dict_users = {  
    Tomson: {  
        "id": "745",  
        "e_main": "tomson12@gmail.com"  
    },  
}
```

```
Bobi: {  
    "id": "444",  
    "e_mail": "bobi@gmail.com",  
    "skype": "bob123"  
}  
  
}
```

Bunda lugʻatning har bir elementining qiymati oʻz navbatida alohida lugʻatni ifodalashi mumkin.

Ichki lugʻat elementlariga kirish uchun mos ravishda ikkita kalitdan foydalanish kerak:

```
old_e_mail = dict_users["Tomson"]["e_mail"]  
dict_users["Tomson"]["e_mail"] =  
"supertomson@gmail.com"  
  
print (dict_users["Tomson"])  
# { "id": "745", "e_main": "supertomson@gmail.com" }
```

Ammo agar lugʻatda boʻlmagan kalit orqali qiymat olishga harakat qilinsa, Python **KeyError** xatoligini chiqaradi:

```
tom_skype = dict_users["Tom"]["skype"]      # KeyError
```

Xatolikka yoʻl qoʻymaslik uchun lugʻatda kalit mavjudligini tekshirish kerak:

```
key = "skype"  
if key in dict_users["Tomson"]:  
    print(dict_users["Tomson"]["skype"])  
else:  
    print("skype lugʻatdan topilmadi!")
```

Boshqa barcha jihatlarda murakkab va ichki lugʻatlar bilan ishlash oddiy lugʻatlar bilan ishlash kabi bajariladi.

Nazorat savollari:

1. Pythonda tugʻat tushunchasiga tarif bering.
2. Pythonda lugʻatlar qanday usullar bilan yaratilishi mumkin?
3. Roʻyxatlar va kortejlarni lugʻatga aylantirish qanday amalga oshiriladi?
4. Lugʻat Elementlariga murojaat qilish va oʻzgartirish qanday amalga oshiriladi?
5. **get()** metodining vazifasi qanday va uning qabul qiluvchi parametrlari qanday?
6. **del** metodining vazifasi va ishlash prinsipi haqida gapirib bering.
7. **pop()** metodining vazifasi qanday va uning qabul qiluvchi parametrlari qanday?
8. **clear()** metodining vazifasi va ishlash prinsipi haqida gapirib bering.
9. **copy()** metodining vazifasi qanday va uning qabul qiluvchi parametrlari qanday?
10. **upgrate()** metodining vazifasi va ishlash prinsipi haqida gapirib bering
11. **items()** metodining vazifasi va ishlash prinsipi haqida gapirib bering
12. **keys ()** metodining vazifasi va ishlash prinsipi haqida gapirib bering
13. Murakkab lugʻatlar qanday hosil qilinadi?

4.4.§. Toʻplamlar bilan ishlash

Toʻplam (set) shunday toʻplamki, uning har bir elementi toʻplamda yagona, takrorlanmas hisoblanadi. Bunday turdagi toʻplamni aniqlash uchun jingalak qavslardan foydalaniladi:

```
set_list= {"Tomson", "Bobbi", "Alisey", "Tomson"}  
print(set_list)    # {"Tomson", "Bobbi", "Alisey"}
```

Eʼtibor berish kerakki, print funksiyasi yordamida toʻplamni ekranga chiqarilganda "Tomson" elementini bir marta chiqarildi. Ammo eʼlon qilinganda "Tomson" ham, toʻplam taʼrifi ushbu elementni ikki marta oʻz ichiga oladi.

Buning sababi shundaki, toʻplam faqat yagona qiymatlarni oʻz ichiga oladi.

Bundan tashqari, to‘plamni aniqlash uchun elementlar ro‘yxati yoki to‘plami uzatiladigan **set()** funktsiyasidan foydalanish mumkin:

```
set_persons = ["Maya", "Billi", "Teddi"]
set_users = set(set_persons)
print(set_users)      # {"Maya", "Billi", "Teddi"}
```

set() funktsiyasi bo‘sh to‘plam yaratish uchun quyidagicha foydalaniladi:

```
set_users= set()
```

Boshqa turdagi toplamlar singari **set()** to‘plamida ham uzunligini aniqlovchi **len()** funktsiyasi mavjud:

```
foydalanuvchilar = {"Tomson", "Bobi", "Alison"}
print (len(foydalanuvchilar))      # 3
```

add() metodi

To‘plamga bitta element qo‘shish uchun **add()** metodidan foydalaniladi:

```
set_users = set()
set_users.add("Sammi")
print (set_users) # "Sammi"
```

remove() metodi

Element olib tashlash. Bitta elementni olib tashlash **remove()** metodi yordamida amalga oshiriladi va olib tashlanadigan element qaytariladi. Ammo shuni yodda tutish kerakki, agar bunday element to‘plamda bo‘lmasa, unda xatolik yuzaga keladi. Shuning uchun, o‘chirishdan oldin, **in** operatori yordamida element mavjudligini tekshirish kerak:

```
set_users = {"Tomson", "Bobi", "Alison"}
set_user = "Tomson"
if set_user in set_users:
    set_users.remove(set_user)
print(set_users)      # {"Bobi", "Alison"}
```

discard () metodi

Elementni olib tashlash uchun discard () usulini ham qo'llash mumkin.

Ammo element mavjud bo'lmasa, xatolik qaytariladi:

```
set_users = {"Tomson", "Bobi", "Alison"}

set_users.discard("Tim") # element "Tim" mavjud emas va metod
hech narsa qilmaydi
print(set_users)        # {"Tomson", "Bobi", "Alison"}

set_users.discard("Tomson") # "Tomson" elementi mavjud va
usul elementni olib tashlaydi

print(set_users)        # {"Bobi", "Alison"}
```

CLEAR() metodi

Barcha elementlarni olib tashlash uchun **clear()** usuli chaqiriladi :

```
set_users.clear()
```

To'plam elementlariga sikl yordamida murojaat qilish

Elementlarga murojaat qilish uchun for siklidan foydalanish mumkin :

```
set_users = {"Tomson", "Bobi", "Alison"}

for user in set_users:
    print (user)
```

Takrorlash paytida har bir element **user** o'zgaruvchisiga uzatiladi va u o'z navbatida ushbu elementlarni konsolga chiqaradi.

COPY () metodi

To'plamlarni nusxalash. Copy () usulidan foydalanib, bir to'plam elementlarini boshqa o'zgaruvchiga nusxalash mumkin:

```
set_users = {"Tomson", "Bobi", "Alison"}
talabalar = set_users.copy()
print (talabalar)          # {"Tomson", "Bobi", "Alison"}
```

UNION() metodi

To‘plamlarni birlashtirish. **union()** usuli ikkita to‘plamni birlashtiradi va yangi to‘plamni qaytaradi:

```
set_users = {"Tomson", "Bobbi", "Alison"}
users2 = {"Semmi", "Keyt", "Bobbi"}

users3 = set_users.union (users2)
print (users3)
# {"Tomson", "Bobi", "Alison", "Semmi", "Keyt"}
```

INTERSECTION() metodi

To‘plamlarning kesishishi. To‘plamlarning kesishishi faqat ikkala to‘plamda bir vaqtning o‘zida bo‘lgan elementlarni olish imkonini beradi. **Intersection()** metodi to‘plamlar kesishish funksiyasini bajaradi va kesishmadan hosil bo‘lgan yangi to‘plamni qaytaradi:

```
set_users = {"Tomson", "Bobbi", "Alison"}
users2 = {"Sammi", "Keyt", "Bobbi"}

users3 = set_users.intersection (users2)
print (users3)      # {"Bobbi"}
```

AND opratori

Kesishish usuli o‘rniga mantiqiy ko‘paytirish operatsiyasi (**and**) dan foydalanish ham mumkin:

```
set_users = {"Tomson", "Bobbi", "Alison"}
users2 = {"Sammi", "Keyt", "Bobbi"}
```

```
print (set_users and users2)    # {"Bobbi"}
```

Bu ikkala holda ham, bir xil natijaga erishiladi.

INTERSECTION_UPDATE() metodi

intersection_update() metodi birinchi to'plamni kesishish elementlari bilan almashtiradi:

```
set_users = {"Tomson", "Bobbi", "Alison"}
users2 = {"Sammi", "Keyt", "Bobbi"}

set_users.intersection_update(users2)
print(set_users)    # {"Bobbi"}
```

DIFFERENCE() metodi.

Ushbu metod to'plamlar orasidagi farqlanuvchi elementlarni qaytaradi. Ya'ni birinchi to'plamda bo'lgan, ammo ikkinchisida yo'q bo'lgan elementlarni qaytaradi. To'plamlar farqini olish uchun **difference** usuli yoki ayirish operatsiyasidan foydalaniladi mumkin:

```
set_users = {"Tomson", "Bobbi", "Alison"}
users2 = {"Sammi", "Keyt", "Bobbi"}

users3 = set_users.difference(users2)
print(users3)    # {"Tomson", "Alison"}
print (set_users - users2)    # {"Tomson", "Alison"}
```

SYMMETRIC_DIFFERENCE() metodi

To'plam farqining alohida turi - simmetrik farq **symmetric_difference()** usuli yoki $\wedge$ operatsiyasi yordamida amalga oshiriladi. U ikkala to'plamning umumiy elementlardan tashqari barcha elementlarini qaytaradi:

```

set_users = {"Tomson", "Bobbi", "Alison"}
users2 = {"Sammi", "Keyt", "Bobbi"}

users3 = set_users.symmetric_difference(users2)
print(users3)      # {"Tomson", "Elison", "Sammi", "Keyt"}

users4 = set_users ^ users2
print(users4)      # {"Tomson", "Alison", "Sammi", "Keyt"}

```

ISSUBSET metodi

issubset metodi joriy to‘plam boshqa to‘plamining kichik to‘plami (ya’ni qismi) ekanligini aniqlash imkonini beradi:

```

set_users = {"Tomson", "Bobbi", "Alison"}
users2={"Sammi", "Keyt", "Bobbi", "Alison", "Tomson"}

print (set_users.issubset(users2))      # True
print(users2.issubset(set_users))      # False

```

ISSUPERSET metodi

Agar joriy to‘plam boshqa to‘plamning super to‘plami (ya’ni o‘z ichiga olgan) bo‘lsa, **issuperset** usuli **True** qiymatini qaytaradi:

```

set_users = {"Tomson", "Bobbi", "Alison"}
users2={"Sammi", "Keyt", "Bobbi", "Alison", "Tomson"}

print(set_users.issuperset(users2))      # False
print (users2.issuperset(set_users))      # True

```

FROZENSET matodi

Muzlatilgan to‘plam (*frozen set*) turidagi to‘plamlar o‘zgartirib bo‘lmaydigan to‘plamlar turiga kiradi. Uni yaratish uchun **frozenset** funksiyasidan foydalaniladi:

```
set_user= frozenset({"Tommi", "Bobbi", "Elison"})
```

Frozenset funksiyasiga elementlar to‘plami - ro‘yxat, kortej boshqa to‘plamlar uzatiladi.

Bunday to‘plamga yangi elementlarni qo‘shish, undagi mavjud elementlarini o‘chirib tashlash kabi amallarni bajarishning ilojisi yo‘q. Aynana shuning uchun **frozenset** to‘plami operatsiyalarning cheklangan to‘plamini qo‘llab-quvvatlaydi:

len(s): to‘plam uzunligini aniqlash;

x in s : agar **s** to‘plamida **x** elementi mavjud bo‘lsa, True qiymatini qaytaradi

x not in s: agar **x** **s** to‘plamining tarkibida bo‘lmasa, True qaytaradi

s.issubset(t): agar **t** **s** to‘plamini o‘z ichiga olgan bo‘lsa, True qiymatini qaytaradi

s.issuperset(t): agar **t** **s** ichida bo‘lsa, True qiymatini qaytaradi

s.union(t): **s** va **t** to‘plamlarning birlashuvini qaytaradi

s.intersection(t): **s** va **t** ning kesishishini qaytaradi

s.difference(t): **s** va **t** to‘plamlari orasidagi farqni qaytaradi

s.copy(): **s** to‘plamining nusxasini qaytaradi

Nazorat savollari:

1. To‘plamlarni yaratishning qanday usullarini bilasiz?
2. set() metodining vazifasi qanday?
3. To‘plamlarni lug‘atlardan farqi nimada?
4. To‘plamga element qo‘shish qanday amalga oshiriladi?
5. To‘plamga element olib tashlash qanday amalga oshiriladi?
6. **discard()** metodining vazifasi qanday?

7. **clear()** metodining vazifasi qanday?
8. **copy()** metodi nima maqsadda ishlatiladi va uning afzallik hamda kamchiliklari qanday?
9. **union()** metodi haqida gapirib bering.
10. **intersection()** qanday metod?
11. **intersection_update()** metodining vazifasi va **intersection()** metodidan farqi haqida gapirib bering.
12. **symmetric_difference()** metodi haqida gapirib bering.
13. **issubset()** – metodining ishlash prinsipi qanday?
14. **issuperset()** – metodidan qanday foydalaniladi?
15. **frozenset()** funksiyasining vazifasi nima?

FOYDALANILGAN ADABIYOTLAR RO‘YXATI

1. Sh.A. Mengliyev, O.A. Abdug‘aniyev. “Python dasturlash tili”. Termiz - 2021.
2. Франсуа Шолле. “Глубокое обучение на Python”. — СПб.: Питер, 2018. — 400 с.: ил. — (Серия «Библиотека программиста»)..
3. Пол Бэрри. “Изучаем программирование на Python”; [пер. с англ. М. Райтман]. – Москва: Эксмо, 2017 год.
4. Н.А.Прохоренок, В.А.Дронов. “Python3 и PyQt5. Разработка приложений”. СПб.: БХВ-Петербург, 2016. – 832 с.: ил.
5. Чан, Уэсли. “Python: создание приложений. Библиотека профессионала”, 3-е изд. [Пер. с англ. - М. : ООО "И.Д. Вильямс"], Москва: Санкт-Петербург • Киев 2015.
6. Sh.R.Sapayev, A.A.Abidov, “Dasturlash texnologiyalari” o’quv qo’llanma. Toshkent 2022. 123b
7. Марк Саммерфилд. “Программирование на Python 3. Подробное руководство” [Пер. с англ. – СПб]. - Москва: Санкт-Петербург–2009 год.